

THE IDENTITY TOKEN SYSTEM · VERSION 3



POLARIS

Fixus inter mutabilia

The map of the system

A reference implementation of an issuer-unlinkable, duress-aware identity-token system, signed with ML-DSA-65 under an audited algorithm-migration path, read from the centre outward, at version 1.0.0-rc.7

Egor Khaklin

SETON HILL UNIVERSITY · SEPTEMBER 2026

About this version

Version 1 of this report (`polaris_project_report.pdf`, preserved unchanged beside this file) described Polaris as a database-design project: twelve tables around one credential, an entity-relationship model, a lifecycle state machine, and the argument that identity data must be post-quantum from the first day. Version 2 (`polaris_project_report_v2.pdf`, also preserved) described the system as it stood at tag `v9.345`, thirty-six tables, a federation protocol, a transparency layer and an institutional exchange fabric, and was revised at `v9.355` when the holder-side work it had recorded as missing was built. Version 3 describes the system at version `1.0.0-rc.7` (commit `4e74376`, 20 September 2026), re-measured whole rather than patched, and it is written for a reader who has never opened the repository. It was first written at `1.0.0-rc.1` four days earlier and re-measured in place; fourteen of the twenty-six counts in Appendix C did not move, and the ones that did are marked there.

What changed, in one paragraph. Between Version 2 and this one the repository stopped counting its own progress and started counting witnesses. The four standalone packages a relying party installs are on public registries. A wallet written by somebody else, run unmodified, presented a credential to the verifier and was accepted, after refusing it once for a defect every internal instrument had passed over. The OpenID Foundation’s hosted conformance suite ran the verifier profile across the open internet. The verification layer itself was attacked from inside: the checks, the constraints, the triggers, the unique indexes, the stored procedures, the zero-knowledge witnesses, the conformance contract and both reference verifiers were each mutated and required to go red, and many did not, at first. The physical layer became an object somebody else can implement. And two words on the front door were weakened, *post-quantum* to *algorithm agility under an audited migration path* and *compulsion-resistant* to *duress-aware*, because the evidence did not support the stronger ones. Appendix F lists every claim of Version 2 that this version withdraws or narrows.

Three dates, stated plainly. Version 2’s body was measured at `v9.345`; its holder-side sections at `v9.355`. Everything in this version, every count, every mark and every figure, was measured at `1.0.0-rc.7` by the methods in Appendix C. Where a number is older than that, because the repository has not re-run the instrument that produced it, the paper says so beside the number. Nothing that ran at `v9.355` stopped running.

The paper keeps what Version 1 got right, the token-centred geometry of the schema, used here as the centre of every map, and the lifecycle machine, and what Version 2 got right, the concentric reading of the system, one ring per section, each ring introduced by the failure of the ring inside it.

Five marks

Every capability carries one of five marks, so that code that runs is never confused with an intention, and so that a thing the author has tested is never confused with a thing somebody else has.

IMPLEMENTED A path that runs at `1.0.0-rc.7`, exercised by a test or a drill in continuous integration, and pinned by an invariant check that fails the build if it stops being true.

EXPERIMENTAL A path that runs but is limited in a way the repository itself names: not covered by CI, notional in scale, or dependent on a backend that CI simulates.

EXTERNAL VALIDATION REQUIRED A property the repository implements and tests internally but that no external party has audited, attacked, deployed or independently re-implemented.

EXTERNALLY EXERCISED A path that a named party outside the repository has actually exercised, on a date, with a result that can be pointed at: a wallet, a conformance service, a published test corpus. The mark is deliberately rare, it is given only where the repository’s scoreboard

has a filled row, and it never means audited, certified or piloted. Three things carry it in this paper.

PROPOSED A proposal: a roadmap row, a design note, or an idea that exists only in this paper. It is not code.

A row carrying two marks runs today and still requires external validation for the stronger claim, or has been exercised from outside on one path and not on the others.

Numbers were measured on the tree at 1.0.0-rc.7 by the methods in Appendix C, not copied from earlier documents. Where the repository's own documents disagree with its code, the code wins and the disagreement is recorded in the author's notes that accompany this version.

The analogy, and its limit

The paper explains Polaris through one sustained analogy: a walled town. The credential record is the town hall's register; the constraints are the wall; the endpoints are gates; the transparency logs are watchtowers; independent witnesses stand outside the wall watching the towers; other authorities are other towns, joined by roads that run one way; above the whole town stand laws that bind even the town itself; and, new in this version, travellers from elsewhere have now come to the gate, and what they found there is the only report the town did not write about itself. The analogy describes arrangement and sight lines, not political centralisation: Polaris is federated per authority, with no central database, root or verification service, and every recurrence of the analogy returns to the actual mechanism by its real name.

One thesis, and one rule

The system is an attempt to build identity infrastructure in which important claims do not have to be trusted merely because a central application says they are true. Authority, state, history, privacy boundaries and the system's own behaviour are progressively exposed to independent verification and constrained by layers that do not share each other's assumptions. Each layer exists because the layer inside it could be entirely correct and still fail to prove what a society needs to know. That progression is the spine of the paper.

The rule, adopted on 13 September 2026 and applied to everything since, is that the unit of progress is a new external dependency survived: a named wallet, a named conformance profile, a named relying party, a defect filed by somebody who is not the author. The count of invariants, versions and lines is not progress and the repository's scoreboard refuses to record it. This paper follows the rule. Where it reports internal evidence, it says so, and it keeps that evidence out of the sentences that report the other kind.

Contents

About this version	i
1 The problem	1
2 How to read this map	2
3 The core: one credential, one signature, one person	5
4 The walls: the schema as the security boundary	10
5 Authenticating truth: signatures, custody, two witnesses and one migration	16
6 Verification without the issuer: the packaged verifiers, the conformance contract and the first strangers	20
7 Verification is not authorization: status, revocation and the offline trade	25
8 The privacy boundary: what a verifier is allowed to learn	29
9 Using the credential: holders, relying parties and the login that keeps no record	33
10 Federation: independent authorities that recognise one another	36
11 Watching the watchers: logs, witnesses, the split view and the public report	39
12 Institutions: exchange, time and signatures without a message log	42
13 Surviving reality: the stack, failure, load, the standing witness, and the day a pilot ends	46
14 How Polaris observes and understands itself	51
15 Errors become new senses: the history of self-correction	54
16 What Polaris refuses to become	61
17 AI agents, proof of personhood and the future internet	63
18 Future research and engineering, under a rule that forbids most of it	67
19 Current limitations and the external validation still required	71
20 Conclusion: the whole organism	74
A Capability status	76
B The metacognitive map	79
C Counts at 1.0.0-rc.7, and how they were measured	82
D Glossary	84
E From Version 1 to Version 2	87
F From Version 2 to Version 3	88

1 The problem

A person in the United States carries six to eight credentials that do not talk to each other: a driver's licence, a passport, a Social Security card, a Real ID, a voter registration, an insurance card. Each is a separate artifact, signed by a separate authority to a separate standard, with no shared revocation path and no shared audit trail. Version 1 of this report set out to model their consolidation into one active credential record per person, verified through context-scoped events. That is the easy half of the problem, and it is still the problem's shape.

The hard half is what happens when the system works. An identity system that succeeds becomes the doorway to everything, and a doorway is a place where power concentrates. Three failure modes follow, and every layer of Polaris exists to answer one of them.

The system lies about the past. An operator, or an insider with database access, revokes a token and later erases the record; issues a credential under procedures the public was never told of; or shows one history to an auditor and another to a court. A national identity system cannot be allowed to retroactively rewrite what it did.

The system watches. Every verification is a fact about where a person was and what they were doing. A log of those facts is a surveillance backbone whether or not anyone intended one, and it stays one after the operator's intentions change.

The system coerces. A holder can be compelled: a border officer, an employer, or a thief can force a credential to be presented, and a doorway that every service requires can force a population to enrol. An identity system is also the most efficient instrument of exclusion ever built, if the list of who is not in it can be produced on demand.

The conventional answer to all three is policy: rules about retention, access, audit and consent, enforced by the people who run the system. Polaris's constitution (MISSION.md) takes the position that policy is worthless as the only thing holding a promise up, because the people who run the system are exactly the ones the promise is meant to bind. The promise has to be enforced where it cannot be talked around: in the schema, in triggers, in unique indexes, in check constraints, in signatures that anyone can verify without asking the system, and in logs that other people watch. The application is not the security boundary. The database is, and the layers around the database exist to make even the database's claims checkable by strangers.

1.1 Why post-quantum from the first day, and what the phrase is allowed to mean

Version 1 argued, through Mosca's inequality, that identity data has a secrecy horizon measured in generations, that migration of a national system takes a decade, and that a credential system deployed on classical cryptography in 2026 is therefore a deferred compromise rather than a working system. That argument is unchanged and is summarised in Appendix E. At 1.0.0-rc.7 the default signing algorithm is ML-DSA-65 (FIPS 204), ML-DSA-87 is accepted beside it, the zero-knowledge proof system is hash-based with no trusted setup, and the public TLS edge and one of the two internal hops negotiate a hybrid post-quantum key exchange.

What the repository claims about that has narrowed since Version 2, and the narrowing is one of this version's subjects. *Post-quantum* is accurate about the algorithms. It is not a claim the repository can make about security, because ML-DSA's security rests on lattice assumptions that could fall to mathematics rather than to hardware, and because the default development path signs with a labelled placeholder rather than a lattice signature at all. The defensible claim is *algorithm agility under an audited migration path*: the algorithm is a row in the schema and never a constant, a population can be re-signed under a new one on a path that runs on every

push, and what a break would cost is written down. Section 5 gives the argument and the cost; an invariant check refuses any outward surface that says more.

1.2 The progression this paper follows

Each of the following sentences names a layer, and each is true of the layer before it.

- A credential's authenticity is not its current authorization. A signature stays genuine forever; the authority under it can be withdrawn this afternoon.
- Current authorization is not privacy. A verifier can learn that a credential is valid and still learn far more than it needed to.
- Privacy is not freedom from coercion. A system can know almost nothing about a person and still be the doorway every service demands.
- Federation is not institutional independence. Two instances of the same code run by the same author prove that the protocol works, not that independent institutions run it.
- A transparency log is not protection from a split view. One observer can be shown a private fork of history and never know.
- A green test is not proof that the test asked the right question.
- A system that has checked itself is not a system that anyone else has checked. Every instrument in the repository passed over a defect that the first outside implementation refused on sight.

The last two sentences are the ones the project has learned most expensively. Section 15 is about the sixth, and Section 6 carries the seventh.

2 How to read this map

Polaris is not one enormous table or one enormous server. It is a very small credential core with deliberately bounded structures arranged around it, each of which constrains, protects, observes, verifies, records or limits something about the structure inside it. The paper reads outward from the core, ring by ring (Figure 1), and the reader should be able to zoom: from the person, to the credential, to the cryptography that authenticates it, to the schema rules that bound it, to the authority that issued it, to the parties that rely on it, to the federation of authorities, to the witnesses that watch them, to institutional exchange, to the operations that keep it alive, to the society it sits inside, to the environment of software agents it now serves, and finally to the outside: the software written by strangers whose acceptance or refusal is the only evidence the repository did not produce about itself.

2.1 The questions asked of every layer

For each major component the paper answers the same questions, in a card of the form used throughout. What is this? Why does it exist? What problem does it solve? What does it trust? What does it refuse to trust? What can it see? What is it deliberately prevented from seeing? What happens if it fails? What part of Polaris checks it? Is it needed now, or is it preparation for a future environment? How does it connect to the layer inside it and the layer outside it? A reader who can answer those questions for every ring has the mental model this paper exists to build.

2.2 The town

IN THE TOWN The credential record is the register kept in the town hall: one entry per person, the claim that everything else is organised around. The wall is the schema: triggers, check constraints and unique indexes that decide which states may exist at all, and which bind every visitor, every official and every restore from backup alike. The gates are the endpoints through which a claim enters or leaves: issuance, verification, login, exchange. The watchtowers are the transparency logs, and the witnesses stand outside the wall, cosigning what the towers publish and shouting when two towers disagree. Other towns are other authorities, each with its own hall and its own key, joined by roads that run in one direction and for one purpose. Above everything stand laws that the town itself cannot repeal by a vote of its officials. And at the main gate, since this version, a traveller from a town nobody here built has presented papers and been let through, after first being turned back for a flaw in the town's own seal that none of its clerks had noticed.

The map of the town, with the technical name beside every civic one, is Figure 5 in Section 4. The analogy will recur in a shaded box like the one above, and every recurrence is followed by the mechanism it stands for. The traveller is `walt.id`'s wallet, the gate is the OpenID4VP verifier, and the flaw in the seal was a request-signing certificate with no `digitalSignature` key usage; Section 6 gives all three by name.



Figure 1: The concentric map of Polaris. The core is one credential record and its signature. Each ring names the actual components on it, and the reading order of the paper follows the rings outward. The outermost ring is dashed because most of it is not code: it is what parties outside the repository have done with the rings inside it, and at 1.0.0-rc.7 that is one wallet, one hosted conformance run and one published test corpus.

2.3 Trust and refusal

The most useful habit for reading Polaris is to ask, of every layer, what it refuses to take on faith from the layer below. The schema refuses to trust the application. The detached verifier refuses to trust the issuer's server. The second witness refuses to trust the first implementation. The relying party refuses to trust a transitive chain of authorities. The witness refuses to trust the log's own head. The check layer refuses to trust a check that cannot detect its own violation. The mutation drills refuse to trust a test that has never been shown to fail. And the scoreboard refuses to count anything the author ran against the author's own code. Appendix B tabulates the refusals as a ladder; Sections 3 through 13 explain each one.

3 The core: one credential, one signature, one person

3.1 What the credential is

At the centre of Polaris is one row of the table `IdentityToken` and at least one row of `TokenSignature`. The token row records a unique `token_value`, a physical serial, the person it belongs to, the agency that issued it, the algorithm it was issued under, its predecessor if it replaced an earlier token, and its status. The signature row records the issuing agency's ML-DSA signature over the SHA3-256 digest of the token value, together with the public key that produced it, so that verification is self-contained and survives the retirement of the signing key. A token without a signature row cannot exist: a database trigger refuses any write that would leave one without a non-deprecated signature. The credential is therefore not a document but a signed claim: *this authority issued this token to this person*. Since v9.349 the claim may also bind a key the holder controls, recorded in `HolderKeyEvent` under the same append-only discipline as an authority key, so that possession of the file stops being possession of the credential (Section 17). **IMPLEMENTED**

What the holder holds is the authenticity pack: the token value, the signature and the issuer's public key, exported by the application and kept as a file by the reference wallet (`scripts/polaris-wallet.py`). Anyone can check the pack with a standard ML-DSA library and no Polaris code (Section 6).

3.2 The physical token, as an object somebody else can implement

Version 2 recorded the hardware card as modelled and not manufactured. That is still true of the silicon, and false of everything around it. Since v9.366 the card is an encoding with a reference codec and published vectors (`polaris_card/card_profile.py`), a software token that speaks ISO 7816-4 with real status words, two PIN slots, a retry counter and a PUK (`emulator.py`), a personalization step in which the card generates its own keys so that the authority never holds them, and a reference verifier device that reads a card over NFC or a presentation over QR and reports three findings separately, possession, authenticity and authorization, because a revoked card still signs a challenge and a device that says only *accepted* teaches its operator nothing. The profile states its one shaping constraint plainly: certified secure elements that sign FIPS 204 in hardware are not something an authority can buy in volume today, so a card carries two issuer signatures over one body, a classical one that today's silicon can produce and an ML-DSA one, and the reader decides which it trusts. Every one of these runs in a drill on every push. **IMPLEMENTED** for the profile, the emulator, the vectors and the device; **PROPOSED** for a manufactured card, which no drill can establish.



Figure 2: The core entity geometry, unchanged from Version 1. Seven entities around **IdentityToken**: the person who holds it, the agency that issues it, the algorithm that signs it, the contexts it may be verified in, and the two append-only event logs it generates. The many-to-many relationships resolve to the junction tables **AgencyAlgorithmAuth** and **TokenPermission**. This geometry is the constant of the paper: Section 4 draws the other thirty-eight tables as rings around it.

3.3 What the credential is not

The constitution’s tenth constraint, C10, states that identity attestation never carries spending authority: there is no monetary claim in the schema, and a check fails the build if one appears. The reason is stated as a failure mode, not a preference. A credential that is also money inherits programmability: every constraint anyone wants to place on spending accretes onto the identity token, until an administrative paperwork error becomes an existential bank-balance error, and until the system can be told that a person may not buy fuel. The same refusal covers reputation, behaviour and profiles. Polaris answers one question, *is this token valid for this context right now*, and it declines to answer *should this person be allowed to do X*. That second question belongs to the relying party, outside the boundary. **IMPLEMENTED**

3.4 One active token per person

A person may hold several provisioned tokens but only one may be **ACTIVE**. Two active tokens for one person would let one token authorise a transaction and the other repudiate it. The rule is C3, and it is enforced by a partial unique index, `uq_one_active_per_person`, on the person's identifier where the status is active. Two operators activating two reserve tokens for the same holder at the same moment find that exactly one of them succeeds, and the test that proves it runs with real threads against a live database (C9). The rule is also modelled in TLA+ and the model is checked on every push, under every interleaving of concurrent issue and revoke, with a counterpart configuration it is required to fail. One more thing is true of it, and Section 15 tells it in full: when the index was dropped to see what would notice, the suite that exists to prove C3 turned out to be violating it, committing a second active token before failing. **IMPLEMENTED**

3.5 The lifecycle



Figure 3: The token lifecycle machine as the trigger `trg_token_state_machine` enforces it: six legal transitions, each labelled with the lifecycle event it writes automatically. **ACTIVE** is the only operational state. A token enters at **RESERVE** and is activated, or is revoked before it ever is; an active token leaves through one of three terminal states or into **DORMANT** when a successor takes over. Version 1's figure left out the revocation of a reserve token, which the trigger has always allowed.

Version 1 recommended enforcing the transitions in a trigger and left it as future work. The trigger exists, and a second trigger writes a `TokenLifecycleEvent` row for every status change, including a change made by hand from a database shell. Succession is by reference, never by overwrite: a replacement token points at its predecessor and the old token stays in the database in its terminal state. Lost tokens stay lost, and their data is not erased. **IMPLEMENTED**

3.6 Entering, leaving and returning

Three mechanisms surround the core so that the schema cannot be turned against the holder at the edges of the lifecycle, and each has grown a record of what it rested on since Version 2.

Entering: tiered enrolment, and what an enrolment rests on. Every person has an enrolment status drawn from five values, and **EXEMPT** is a first-class one: a recognised way to take part in civic life without a token, whether for biometric incompatibility, religious exemption or an alternative path. Recording an exemption is one row; enumerating the

not-enrolled is possible but unhelped by any function, and it is audited when done. Until v9.371 the system could issue a credential and had no way to say how the person had been proven to be who they claimed. `EnrollmentProofing` now records one proofing event and `EnrollmentEvidence` what it rested on, both append-only, and the assurance level is *derived* from the evidence rather than typed by an operator; there is deliberately no column for the document itself, and a named set of fields is refused outright, so the record cannot reconstruct a passport. What it does not establish is stated in the review packet as a limitation: whether the evidence was genuine is taken on the operator's word. **IMPLEMENTED**

Every combination of evidence a standard names starts from documents, and a person with none, a care leaver, a refugee, an adult who never held a passport, fails all of them by omission. The trusted referee (`RefereeVouching`) is the answer to that, and it is also the easiest way to mint an assurance level out of nothing, so the repository treats the anti-exclusion mechanism and the forgery channel as the same table: a vouching is bounded, co-signed past a threshold, and invisible on the credential itself, because a person who needed one should not carry a mark for it at every counter. An enrolment code, the last piece that needs no hardware, establishes only that somebody who could reach a channel returned a secret, and the level it can contribute is capped accordingly. **IMPLEMENTED**

Leaving: issuer discretion, as a decision that survives being changed. The worst failure of an identity system is not a forged token but an authority revoking a population's tokens at scale. A trigger bounds how fast one agency can revoke its own tokens (five percent of its outstanding base in thirty days by default) and above that a co-signer from a different agency is required, recorded in the audit trail. The bound converts a surprise into a trend that reporting can see; it does not stop slow abuse under the ceiling, and says so. Since Version 2 the settings around this power have become records: the ceiling itself (`IssuerDiscretionPolicy`) can only be changed with a stated justification and leaves a row saying who changed it and from what; a quota on how much of the population an agency may touch is a decision with a reason rather than a value that can be overwritten; the row at the root of the hierarchy, `Agency`, carries a trigger that records every change to an authority's level and marks whether it widened; and a relying party's bar, which could be lowered with nothing recorded anywhere until v9.425, is now recorded with the same discipline, weakenings flagged and reasons required. **IMPLEMENTED**

Returning: the recovery ceremony. A fire takes the wallet and the reserve together. Recovery is mandatory and is also an impersonation path, so it is made expensive without being made impossible: a two-phase ceremony with a 48-hour cool-down, three independent out-of-band channels, an approver who cannot be the requester, and admin-only completion. The decision fields move one way under a trigger since v9.347, and the four-eyes rule, the cool-down and the three channels are refusals a stored procedure makes that a mutation drill now proves the tests would notice losing (Section 4). The holder is without identity for the cool-down; the substitute credential a production system would want for that window is not built. **IMPLEMENTED** for the ceremony, **PROPOSED** for the interim credential.

3.7 Duress, and what the word is allowed to mean

The vocation above the ten constraints is anti-coercion: no person may be compelled to renounce, transfer or surrender their identity against their will. The mechanism that gives that sentence something to stand on is old and comes from banking: a second secret that looks exactly like success to whoever is watching, while an alert goes out through a channel the watcher cannot see. A holder may enrol a duress code. When it is presented, the verification proceeds identically on every operator-visible surface, the same page, the same flash message, the same event row, and a `DuressEvent` row is appended to a table the operator's screens never join, from which a pager

fires at the highest severity. The comparison is constant-time and the work is paid on both paths. On the card the same mechanism is a second PIN of the same length as the first, entered the same way, unlocking a second key slot whose public key the card object does not carry, and a drill measures the response in time as well as in bytes, because a coercer at a reader observes how long the card took. **IMPLEMENTED** as a tested property of the modelled flow.

Version 2 called the system compulsion-resistant. Version 3 does not, because the repository's own assessment of the mechanism against the three coercers the operating contract names concluded that the word was not supported, and the vocabulary was weakened to *duress-aware* on every outward surface, with a check that refuses the strong word's return. Against a casual coercer, one who does not know duress codes exist, the mechanism works: the screen is identical and the alert is silent. Against an informed coercer standing beside the holder it does not: the code is typed on the verifier's terminal, keystrokes are not constant-time, and because enrolment is opt-in, *I have no duress code* is a claim a coercer can press on and the holder cannot prove. Against post-hoc lawful or institutional access it is worse than nothing: `DuressEvent` is append-only with no purge path, which is C1 working as intended, so where the coercing party is the institution, or can later compel it, the mechanism manufactures indelible evidence that the holder resisted. The assessment proposes no mechanism change and this paper proposes none; it records the boundary. The side-channel claim, that nothing observable distinguishes the two paths, has been measured by the repository and by nobody else. **EXTERNAL VALIDATION REQUIRED**

Layer card: the credential core

What it is	One <code>IdentityToken</code> row and its <code>TokenSignature</code> rows, optionally bound to a holder key: a signed claim that an authority issued this token to this person, in a lifecycle with one active state, with the proofing it rested on recorded beside it.
Why it exists	Every other structure in Polaris organises itself around this claim. It is kept as small as possible so that the rest can be bounded.
What it trusts	The issuing agency's signing key, held behind a custody interface (Section 5); the schema's constraints; the operator's word that presented evidence was genuine.
What it refuses	Application code as a source of truth: the state machine, uniqueness and audit are decided by the database. An assurance level typed rather than derived. Money, reputation and profiles: C10.
What it can see	The holder's legal name, date of birth and country; the issuing agency; the algorithm; the lifecycle; the kind of evidence an enrolment rested on.
What it cannot see	A biometric template (never stored), the document behind an enrolment (no column exists for it), a verification graph (Section 8), a spending history (never modelled).
If it fails	A forged core is a forged signature, which the two-witness verification and the detached verifier refuse; a duplicated core is refused by the unique index; a rewritten core is refused by the append-only triggers; a coercer who is the institution is not stopped, and the record says so.
Checked by	<code>check_c1c10_objects_resolve</code> , the C3 property tests and the C3 model, the concurrency suite, <code>check_card_profile</code> , <code>check_card_emulator</code> , <code>check_verifier_device</code> , <code>check_duress_on_card</code> , <code>check_duress_claims_are_aware</code> , the attack suite's forgery and tamper cases.
Now or later	Now. It is the part of Polaris that has existed since Version 1; the card is an object now and silicon is later.
Inside and outside	Inside: the person, who exists outside the software. Outside: the wall of constraints (Section 4).

IN THE TOWN The town hall keeps one register. Each entry says who a person is, which official entered it, under which seal, and what papers the official was shown, though never the papers themselves. The register cannot be edited, only appended to; a second entry for the same person is refused at the desk; and the seal is one that every reader can check against the official's published mark without asking the hall.

4 The walls: the schema as the security boundary

4.1 Why the database is not plumbing

In most systems the database stores what the application decided. In Polaris the database decides, and the application is one client among several: the operator's web application, the command-line tool, a bulk loader, a database shell, a restore from backup. A rule that lives in application code binds only the clients that run that code. A rule enforced by a trigger, a check constraint or a unique index binds every client, survives every restore, and cannot be bypassed by the next caller, because the application role has no data-definition privilege with which to remove it. The governing sentence of the architecture is *by construction*: the system cannot be operated in a way that violates the constraint, because the constraint is enforced before any application code runs. **IMPLEMENTED**

The design principle behind it is older than Polaris: make invalid states impossible to represent rather than asking developers not to create them. A verification event that records a token identifier at the zero-knowledge disclosure level is not an application bug to be caught in review. It is a privacy violation that must never be storable, so it is a check constraint on the row.

4.2 The ten constraints and where each lives

The constitution names ten hard constraints in two tiers. Five are constitutional, rights guarantees that may not be relaxed without an amendment recorded in the changelog; five are engineering invariants, the disciplines that keep the first five true under load and attack. The hierarchy is one line deep, vocation, then constitutional constraints, then engineering invariants, and the constitution states that no deeper apparatus governs them.

Table 2: The ten constraints at 1.0.0-rc.7, their tier, and the object that enforces each. The table is kept honest against the code by `check_c1c10_objects_resolve`, which fails the build if a named object no longer exists.

#	Constraint	Tier	Enforced by
C1	The audit trail is append-only; history is never rewritten or deleted	Constitutional	Triggers reject UPDATE and DELETE on every audit table (<code>reject_audit_modification</code>)
C2	A zero-knowledge verification stores no token identifier	Constitutional	Bidirectional CHECK <code>chk_disclosure_token_consistency</code>
C3	One person holds at most one ACTIVE token	Constitutional	Partial unique index <code>uq_one_active_per_person</code>
C4	Failed-login counting is atomic	Engineering	Single-statement UPDATE <code>...RETURNING</code>
C5	No inline scripts; the content security policy is <code>script-src 'self'</code>	Engineering	Response header, verified per route
C6	Disclosure level is enforced server-side; a client cannot upgrade what it learns	Constitutional	Server code paired with redaction tests; the C2 CHECK behind it
C7	No hardcoded cryptography; algorithms are rows in a registry	Engineering	Foreign key to <code>CryptographicAlgorithm</code>
C8	Every map and API aggregate is bounded	Engineering	Hard caps in the SQL functions and routes
C9	Concurrency claims are tested with real threads	Engineering	Threaded suites against a live database
C10	Identity is not money; the schema carries no monetary claim	Constitutional	Structural absence, pinned by a check

The repository also records a structural claim about the ten: that the set is closed, so that an eleventh constraint requires either replacing one or extending the framework, and that the constraints depend on one another, so that removing any one ripples through the rest. The

dependency walk (remove C10 and every other constraint protects a financial-surveillance system instead; remove C1 and every other constraint's enforcement becomes retroactively deniable) is the argument for treating them as a graph rather than a list. Athena, in Section 14, holds the map from each constraint to the exact live mechanism that enforces it, and a check fails the build if a mechanism drifts. Three of the ten now also have a formal model checked on every push: C1's purge coverage, C2's unlinkability across two pooling relying parties, and C3 under concurrent issue and revoke, each with a counterpart configuration it must fail, so none of the three is vacuous.

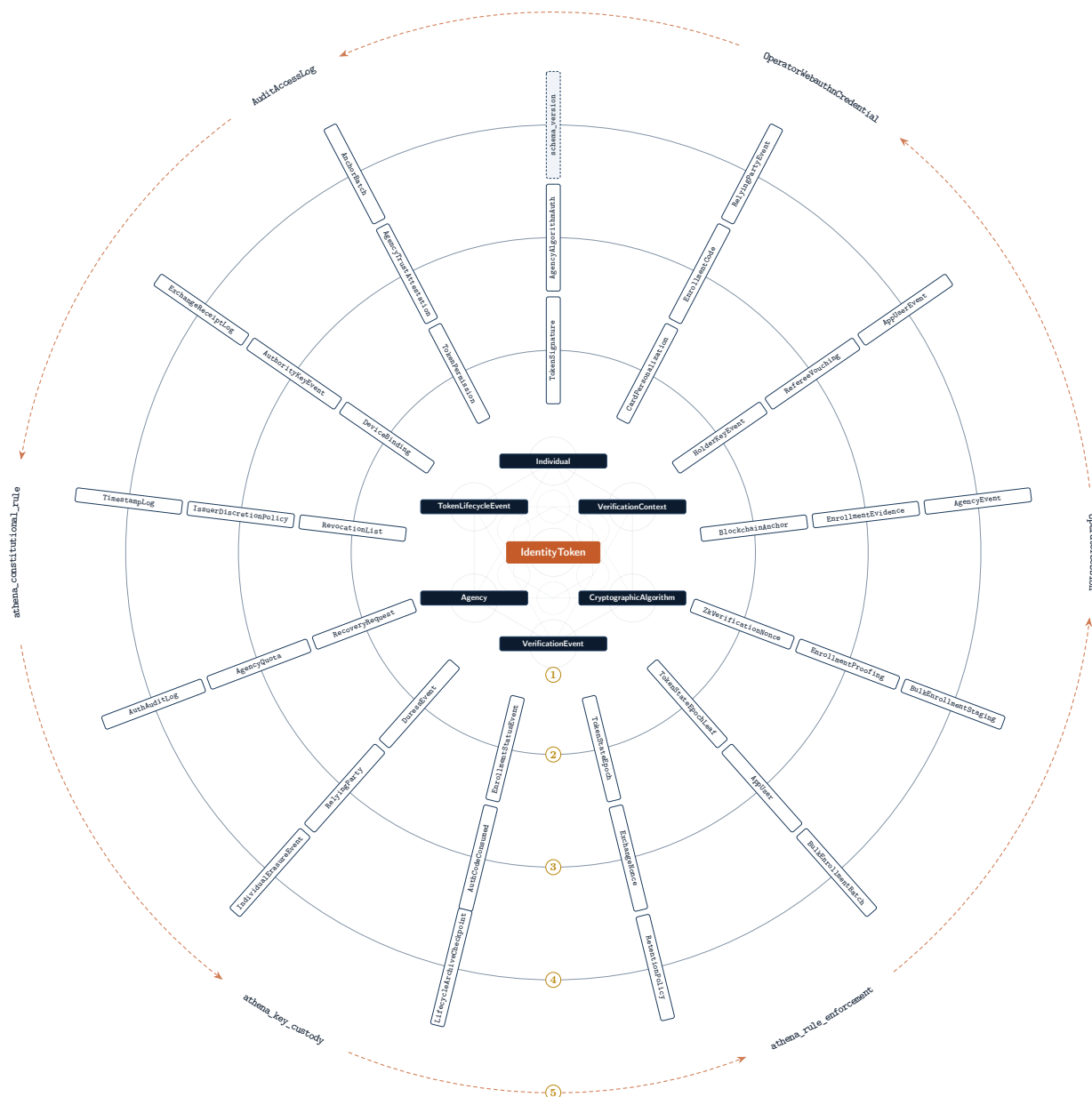


Figure 4: The 45 canonical tables of `01_schema.sql` at `1.0.0-rc.7`, as rings around the Version 1 nucleus. The faint lattice at the centre is the nucleus geometry of Figure 2 at the same proportions. Ring two is the credential machinery: signatures, permissions, epochs, recovery, duress, the holder’s key, the card. Ring three is authority, trust and the people at the counter: who may sign under which algorithm, who trusts whom, revocation ceilings, quotas, relying parties, the replay registers, and what an enrolment rested on. Ring four is the archives, watchtowers and decisions kept as records: append-only logs, anchors, the retention decision as data, and the events that record every change to an authority, an operator account or a relying party. A migrated deployment holds seven more tables, drawn dashed: the migration registry closes the fourth ring, so rings two, three and four have the same thirteen spokes, and the other six (operator sessions, their hardware keys, the audit-access log and Athena’s three curated tables) form a fifth, hexagonal ring that echoes the nucleus. Its arrows all turn one way: migrations applied one after another, round the canonical schema. The five partition children are not drawn.

4.3 The audit of record

Fourteen tables follow one named pattern, the audit of record: the row and its history are the same object. There is no separate event log beside the table that could be edited while the table is locked. Mutation is either forbidden outright or bounded to one field that moves one way, such as a signature's deprecation date or an attestation's revocation date. Every one of the fourteen is enforced by a trigger that raises `insufficient_privilege`, and a check fails the build if any trigger stops doing so. Beyond the fourteen the design record names, thirty-one tables in all carry an append-only trigger at 1.0.0-rc.7, the new ones being the records that decisions about authorities, operators, relying parties and enrolments now leave behind, and since v9.412 C1 is tested once per table that claims it. Of the thirty-one, twenty-four refuse every edit, and a table-driven suite tests each of them for both UPDATE and DELETE, cross-checked against the catalogue; the other seven permit bounded mutation, one field moving one way, and the trigger mutation drill is what answers for them. The cross-check finds tables by their guard function, and the list of those functions was written by hand, so a new table under a new guard would have gone untested without anything saying so. Since 23 September 2026 every such guard must be declared strict or bounded, which is deciding which instrument answers for it, and an undeclared one fails the build. **IMPLEMENTED**

The corollary is that no foreign key anywhere in the schema cascades a delete. A cascade would let the lifecycle events of a deleted token vanish with it. Every reference defaults to NO ACTION, so a principal referenced by any audit row is effectively undeletable, and a check scans every SQL file for both cascade forms with no allowlist. The one record that deliberately carries no foreign key at all, so that ending a relying party's contract cannot erase its history, taught the repository something about its own test database at 1.0.0-rc.7: the seed that reloads the sample world restarted the party's identifiers from one and reached that record with neither, so the first party of each reload inherited a dead party's events. The seed names the record now, and a check computes from the schema's own foreign keys which records a reload would leave behind.

The application role, `polaris_app`, is revoked UPDATE and DELETE on the append-only tables and holds no data-definition privilege. The only path that deletes audit rows is the archive-then-purge procedure, which verifies the archive against its manifest first, refuses any cutoff inside the retention window, and appends its own append-only checkpoint row. The retention window itself is data with a floor: no policy row can express a retention shorter than a year, because a check constraint refuses it, and shortening the floor is a schema change answered for in review. The vocation refuses unbounded retention; it equally refuses retention short enough to erase the record. **IMPLEMENTED**



Figure 5: The walled town, with the real component beside every civic name. The register in the hall is the credential core; the wall is the schema; the four gates are the routes through which a claim enters or leaves; the watchtowers are the transparency logs and the witnesses outside the wall watch them; the road to the second town is a directional, per-context attestation; the laws above the town bind the town itself. The plan describes structure. Polaris is federated: every authority is its own town.

4.4 The check layer, and the drills that attack it

Above the schema sits a flat layer of 313 plain functions, each of the form `check_*(repo_root)`, that read the repository and return a finding. They gate continuous integration: any failure fails the build. Each check is paired with a detection test that breaks a fixture in the specific way the check claims to catch and asserts that the check turns red, and, since v9.405, also asserts that the check passes on the same fixture unbroken, because a test that only ever asserts failure has not established that it detects anything. A check that cannot detect its own violation is treated as broken. This is the mechanism by which prose claims are turned into gates: a count stated in the README, a route that must return a header, a migration that must not cascade, a drill that CI must run. **IMPLEMENTED**

Version 2 described the checks and stopped there. Between v9.398 and v9.437 the repository asked the layer the question it asks everything else, not *does it pass* but *would it notice*, and the answer was often no. Commenting out the lines carrying a check's search strings, which is exactly what a temporarily disabled line looks like, left 71 of 75 checks green, because the helper every check reads through handed it the comments along with the code. Dropping each of the 46 CHECK constraints found every one load-bearing, which was the good news; dropping each of the 37 triggers found 14 that nothing noticed, twelve of them the append-only guards that C1 *is*; dropping each of the 17 unique indexes found twelve uncovered, and stopped on the second, because with `uq_one_active_per_person` absent the suite that exists to prove C3 had inserted and committed a second active token for one person and the index could not be rebuilt. Deleting each of the 59 refusals the sixteen stored procedures make found 27 that no test would miss, concentrated exactly where a trigger cannot see: the four-eyes rule, the cool-down, the three channels and the third-person witness of the recovery ceremony, the preconditions on an irreversible erasure. Every one of these is now covered. The drill selected procedures only, and three of the use cases, issuance, reserve activation and device binding, are functions, because they return the id they create; their eleven refusals sat outside it until 23 September 2026. Among the ones no test noticed were the check that an agency is authorized for the algorithm it issues under and the check that a reserve token belongs to the same person as the credential it replaces. Nine are covered now. One of them, the reserve check the routine repeats under its lock, can fire only if another transaction moves the reserve while the routine waits, so its test holds the lock itself, waits until the routine is seen blocked on it, revokes the reserve and lets go. The two that remain are the copies of its two status checks made before the lock, which on one connection refuse exactly when their twins do. Every one of these measurements is now a drill that runs on every push with a negative control, so that a result of zero survivors is distinguishable from a harness that ran nothing. Twelve such drills exist at 1.0.0-rc.7: over the checks, the constraints, the triggers, the procedures, the zero-knowledge witnesses, the conformance contract, the two reference verifiers, the OpenID4VP verifier, the application, the constitutional constraints, the surfaces a check quantifies over, and the moves of code between modules. Section 15 carries the record. **IMPLEMENTED**

Layer card: the walls

What it is	PostgreSQL 16: 45 tables, 46 triggers (42 of them in the trigger file), sixteen stored procedures and four use-case functions that implement every state-changing use case, the append-only audit tables, the retention policy as data.
Why it exists	A promise enforced only by the people who run the system is not a promise. The schema binds every client, every restore and every insider with a database connection short of the owner.
What it trusts	The database engine and its owner; the correctness of the constraints as written, which is why each is now mutation-tested.
What it refuses	Application code, review process and operator discipline as the last line: the application never enforces a constitutional guarantee on its own. A test that has never been shown to fail.
What it can see	Everything the system stores. This is why the privacy layer bounds what is stored, not who may read it.
What it cannot see	Biometric templates, holder-side keys, enrolment documents, the content of exchanged messages or timestamped documents: none enter the database.
If it fails	A dropped trigger fails the build: deleting each of the 42 in the trigger file in turn, a check notices 38 by reading the schema, and the trigger mutation drill notices all 42 by running the suites; a lowered retention floor or a cascade fails the checks; a constraint whose loss the suite would not notice fails the mutation drill; a database owner with a shell has larger options than any trigger, which the threat model states rather than hides.
Checked by	<code>check_aor_append_only_triggers</code> , <code>check_aor_privilege_boundary</code> , <code>check_no_fk_cascade</code> , <code>check_retention_engine</code> , <code>check_append_only_tables_are_tested_exhaustively</code> , <code>check_constraint_suite_is_mutation_tested</code> , <code>check_triggers_are_mutation_tested</code> , <code>check_unique_rules_are_tested_exhaustively</code> , <code>check_procedure_refusals_are_mutation_tested</code> , <code>check_seed_restart_resets_dependent_records</code> , the SQL self-tests.
Now or later	Now.
Inside and outside	Inside: the credential core. Outside: cryptographic evidence, because a constraint binds only the database that holds it.

A constraint binds the database, not a copy of the claim that has left it. The moment a credential is handed to a stranger, or an authority's history is read by an auditor, nothing in the schema travels with it. The next layer makes claims carry their own proof.

5 Authenticating truth: signatures, custody, two witnesses and one migration

5.1 What a signature is for

A digital signature is a seal that only the holder of a private key can make and that anyone with the matching public key can check. Polaris signs the SHA3-256 digest of a token value under the issuing agency's ML-DSA key and stores the signature beside the public key that made it. The claim a signature carries is narrow and permanent: *this key signed these bytes*. It says nothing about whether the token is still authorized, which is the next section's problem, and nothing about who holds the key, which is custody's problem. **IMPLEMENTED**

The algorithm is ML-DSA-65 (FIPS 204, security category 3) by default and ML-DSA-87 (category 5) beside it; ML-DSA-44 is refused everywhere as below the floor. No signed body hardcodes its algorithm: the custodied key decides it before signing, because `algorithm` is itself a signed field, and the registry advertises the accepted set. SLH-DSA-128s and SLH-DSA-256s, the hash-based alternatives that do not rest on lattice assumptions, are registered in the algorithm table precisely because they rest on hash functions alone, so that a rotation away from lattices is a row update; no SLH-DSA signer is wired, and the posture ledger carries the gap. **IMPLEMENTED** for ML-DSA; **PROPOSED** for a hash-based signer.

5.2 Custody

The private key never sits in application code. It sits behind a custody interface with three drivers: a file (development), PKCS#11 (a hardware security module or a software token; the key is generated inside the token and never leaves it), and a cloud key-management service. Continuous integration exercises the PKCS#11 driver against a software token that implements ML-DSA, not a hardware module, and rotates a key inside it: a token signed under the old in-token key still verifies after the switch while the new key signs. The KMS driver's envelope cryptography is real and its service is a stand-in. One production profile makes the module the sole signer: the application refuses to boot if a file key is present or real signing is unavailable. Which driver, which module, and who holds the key are decisions the readiness ledger assigns to a deploying organisation. **IMPLEMENTED** for the drivers and the drill; **EXTERNAL VALIDATION REQUIRED** for hardware custody, since no hardware module has been used.

The placeholder deserves a paragraph, because it is where an honest system is most easily misread, and Version 3 says it more bluntly than Version 2 did. Without the real-signing flag and a lattice library present, the signing module produces a deterministic SHA3-256 binding labelled DETERMINISTIC-PLACEHOLDER-SHA3-256: thirty-two bytes that verify against no key, where a real ML-DSA-65 signature is 3,309. That is the default in development and in most of continuous integration. It is guarded, since v9.280 a production boot fails closed unless real signing is actually available, the placeholder is a named development profile, and it warns loudly when used unnamed, and it is still the reason the repository no longer calls itself post-quantum without saying so in the same breath. The history is recorded rather than hidden: at v9.58 the headline post-quantum claim was, at the data level, a hardcoded string, and the real signing module was an unused island until it was wired into issuance. **IMPLEMENTED**

5.3 Two witnesses at issuance, one at use, sampled, and one outside corpus

The two-witness principle states that any cryptographic verdict Polaris ships must be checkable by a second, independent implementation, written separately, differently represented, sharing no code, and agreeing on the verdict or abstaining explicitly. A verifier that agrees with itself has established nothing. At issuance the signature just produced is verified by two independent FIPS 204 implementations, a lattice library and an OpenSSL-backed one, and is refused persistence unless both accept it; a missing witness at issuance is a refusal, not a downgrade. **IMPLEMENTED**

Verification at use runs one witness, because a stored signature is already known to verify under both, and one witness detects tampering at about ten times the rate: on the reference machine roughly 7,950 verifications per second per core against roughly 740 two-witness. The fast path earns its speed by being continuously checked: a mandatory random fraction of successful checks is replayed through the second witness, a disagreement increments a counter and pages at the highest severity, and every response names which witness set ran. **IMPLEMENTED**

One thing about the witnesses is not self-referential. On every push, the Project Wycheproof ML-DSA-65 verification vectors, a third-party corpus pinned to a commit in which accepting a signature the corpus calls invalid is a failure, run under both witnesses: 58 of 58 agree. That is evidence that Polaris's verification of the primitive agrees with an outside authority on 58 cases. It is not a wallet, not an interoperability result, and it says nothing about the credential protocol; the repository's scoreboard records it in exactly those words. **EXTERNALLY EXERCISED** for the primitive.

The same principle governs the zero-knowledge verdict (Section 8): a Python re-implementation with plain integers modulo a prime, sharing no code with the Rust prover, re-derives the membership statement and must agree, and it abstains, in the open, on the one axis it cannot model, the integrity of the proof bytes. What the principle catches is implementation divergence. What it does not catch is a shared misreading of the specification, and the repository

says so beside every two-witness claim. Since v9.419 the witnesses are also mutation-tested: the claim that two implementations must agree is now a claim about the tests as well as the code.

5.4 Canonical signing: the same bytes on both sides

Every signed artifact in Polaris is a JSON statement with sorted keys and no whitespace, digested with SHA3-256, then signed. The application builds the statement; the detached verifier (Section 6) rebuilds it from the artifact to check the signature. The two constructions must be byte-identical or a genuine artifact fails to verify, and a test, the canonical-equivalence oracle, pins every signed type on both sides. The one bug the oracle is there to catch was found in review: an artifact whose `algorithm` field was set after signing carried a signed field the signature did not cover. **IMPLEMENTED**

5.5 Keys have a lifecycle

The key register is an append-only table of events, registered, retired, or compromised from an instant that may predate the discovery, from which a signed trust list is published. A verifier holding a trust list decides a key's status at any instant, so a credential under a compromised issuer key is rejected independently of the issuer's own manifest, which a thief holding the key could forge, and a document signed and timestamped before the compromise stays valid while one signed after does not. The two-instance drill records a compromise on one authority and watches the other's decision flip over the wire, with no cooperation from the compromised party. **IMPLEMENTED**

5.6 Agility under an audited migration path, and what a break would cost

This subsection is new, and it replaces a word. Version 2 called Polaris post-quantum on its cover. An outside argument, that the phrase is empty because nobody knows how large quantum computers will get, attacked the wrong thing, and the part it got right was sharper than its framing: ML-DSA-65's security does not rest on a hardware prediction, it rests on Module-LWE and Module-SIS being hard against the best known classical and quantum algorithms, which could be wrong for mathematical reasons. The two real risks are a cryptanalytic advance against the lattice assumptions and an implementation flaw, and the design already answers both, the hash-based hedge in the registry and the two witnesses at issuance. So the engineering was not the problem; the wording was. What the repository can demonstrate is *algorithm agility under an audited migration path*: C7 makes the algorithm a row, `AgencyAlgorithmAuth` authorizes it per authority, and a population can be re-signed under a new parameter set on a path that runs on every push. **IMPLEMENTED**

That path is measured rather than planned. `polaris migrate-population` re-signs the whole active population under a new parameter set, built out of the same constraints the per-token procedure relies on: the unique constraint is the serialization point, so two runners racing one credential produce one row, and sixty-four runners divide the population without coordinating. The number that matters is not throughput but how many holders hold a credential that verifies under nothing, and the drill samples it at every batch boundary rather than at the ends, because a gap that opens and closes between two endpoints is invisible to a before-and-after check. It is zero throughout. The old signature keeps verifying until its deprecation date, and that interval *is* the migration: deprecating the superseded algorithm is refused while any active credential is unmigrated, because a holder would otherwise discover the gap at a border while the console reported progress. Under real ML-DSA-87 with one custodied key the drill re-signs about 345 credentials per second on one runner, 91 percent of it signing, so a population of 350 million is about twelve days on one runner and four and a half hours on sixty-four; buying database capacity for it buys almost nothing. **IMPLEMENTED**

What a break in Module-LWE would cost is written down in the readiness ledger, and this paper repeats it. It would not require a schema change, a change on the verification path, or a new trust model. It would cost every credential already issued under the broken algorithm, and one thing no agility buys back: during a cutover a credential carries a classical signature beside the post-quantum one, and the classical half is protected by nothing this design provides, so a credential harvested today is readable on the day its classical signature falls, whatever the system migrates to afterwards. That asymmetry is why signing post-quantum now is worth doing, and it is the limit of what doing it achieves.

The repository's own lab then asked whether the agility is on the side that needs it, and found that it is issuer-side only. The algorithm's `deprecation_date` reaches no signed artifact: the registry publishes a bare list of algorithm names, the trust list carries per-key status only, and the detached verifier's accepted set is a dictionary in code. So a verifier cannot learn that an algorithm has been deprecated, retiring one is a software release to every relying party rather than a migration, and key revocation does not substitute, because if the assumption breaks an attacker forges under any key the trust list calls active. Rollback and the mixed window during a partial migration are unmeasured. No change is proposed; the finding is recorded as a limitation for a deploying organisation. EXTERNAL VALIDATION REQUIRED

5.7 What remains classical

The token signature, the digests, the Merkle commitments, the zero-knowledge proof, the client-to-edge key exchange for modern clients and, since v9.404, the hop from the application to the connection pooler are post-quantum, each of the last two read off a real handshake rather than inferred from a base image, which is how the posture ledger had once called one of them classical while it was not. The hop from the pooler to the database, the certificate signatures, and the operator's WebAuthn credentials remain classical, each gated on a third party (a PostgreSQL release, the public certificate ecosystem, authenticator hardware) and each stated in the posture ledger with its threat and its exposure. None of the lattice library, the proof system or the circuit has had an external cryptographic audit. EXTERNAL VALIDATION REQUIRED

Layer card: cryptographic evidence

What it is	ML-DSA signatures over SHA3-256 digests, stored with their public keys; a custody interface with file, PKCS#11 and KMS drivers; two independent verifying implementations and one outside corpus; a canonical statement format; an append-only key register and a signed trust list; a population migration that runs.
Why it exists	A claim that leaves the database needs to carry its own proof, a proof only one program can check is a promise, and an algorithm that cannot be replaced is a deadline.
What it trusts	FIPS 204 and 202 as standardised; the two libraries not sharing a bug; the custody backend the operator chose.
What it refuses	A verdict from a single implementation at issuance; an algorithm named in code rather than by the key; a signature stored without its key; a placeholder passing as a signature; a deprecation while any credential would be left verifying under nothing.
What it can see	The bytes it signs, which are digests: the signing path never sees a person.
What it cannot see	Whether the signed claim is still authorized (Section 7); whether a verifier in the field knows an algorithm has been retired.
If it fails	A forged signature fails both witnesses; a compromised key is retired from an instant by the register and every verifier holding the trust list rejects what it signed afterwards; a broken algorithm costs every credential under it and is survived by re-signing, at a measured rate.
Checked by	<code>check_pqc_signing_wired</code> , <code>check_pqc_second_witness</code> , <code>check_verify_witness_sampling</code> , <code>check_real_pqc_default_boot</code> , <code>check_trust_lifecycle</code> , <code>check_algorithm_agility</code> , <code>check_quantum_event_readiness</code> , <code>check_zk_witnesses_are_mutation_tested</code> , <code>check_post_quantum_claims_are_agility</code> , the canonical-equivalence oracle, the attack suite.
Now or later	Now. The hardware module and the external audit are later, and belong to a deploying organisation; the signed algorithm status a verifier could read is not built.
Inside and outside	Inside: the schema, which stores the signature rows. Outside: the independent verifiers that check them without the issuer.

A signature the issuer's own server checks proves nothing to a stranger, because the stranger has to trust the server. The next layer takes the verification out of the issuer's hands entirely, and, for the first time, into the hands of software the issuer's author did not write.

6 Verification without the issuer: the packaged verifiers, the conformance contract and the first strangers

6.1 The detached verifier, as a thing a stranger installs

`polaris-verify` verifies a credential's authenticity pack with only a standard ML-DSA library: no Polaris code, no database, no network unless asked to fetch status. Version 2 described it as a script in the repository. It is now a package on a public registry, at `1.0.0-rc.3`, a release candidate, with `0.1.0` beneath it since 15 September 2026, published by the repository's own workflow under trusted publishing so that no long-lived credential exists anywhere, and the script that Version 2 named is a shim onto it. It refuses to start until the run says what cryptography it is doing: `-pqc-provider oqs, cryptography` or `auto`, or `-dev-placeholder`, with no default and no environment variable, because a verifier that quietly stopped verifying still answers every question. Every machine-readable verdict carries a `crypto` field, and under the development flag the real backends are turned off, so a development run cannot produce a true verdict at all; building that gate found that the first version of the flag declared the mode without entering it, which is a label, not a mode. **IMPLEMENTED**

It is the one Polaris component whose input is chosen by an adversary, because a relying party runs it on whatever a stranger presented, and it is the component through which every later protocol artifact is checked: manifests, epoch checkpoints, revocation feeds, status bundles,

status assertions, transparency heads, receipts, timestamps, signed documents, registries, trust lists, presentations, holder proofs, agent grants. Three independent ML-DSA implementations agree on the published vectors: a lattice library, OpenSSL, and a third in the TypeScript SDK. It is held *total* against hostile input by a metamorphic fuzzer, and an attack suite of executable adversaries runs beside it; a grep proves a line exists, an attack proves a defence holds. **IMPLEMENTED**

Whether a stranger can actually install it is measured, not assumed. On every push, and before every publish, a drill builds the wheel, installs it into a throwaway environment with none of Polaris present, and runs it from a directory that is not the repository with every `POLARIS_*` variable stripped: the forbidden runtime surface (the web application, the database driver, the check layer) must be absent, the command must refuse to start without a declared crypto mode, a genuine ML-DSA-87 credential must verify and its tampered twin must not. It carries a negative control, a second wheel with a database import bolted onto the verifier, which the harness has to catch. This matters because every other suite in the repository runs from inside the repository with the repository on the path; an import that would have stayed green there fails on the first stranger's machine. **IMPLEMENTED**

6.2 The SDKs and the conformance contract

A relying party installs one of two verify SDKs, `polaris-sdk-python` and `polaris-sdk-ts`, each answering one narrow question about a presented credential, is it authentic and is it authoritative right now, and never returning a person's data. Both are on their registries. The TypeScript one is a lesson in what a registry teaches: its first packaging pointed `exports` at TypeScript source, which Node refuses to strip inside `node_modules`, so the package could not have been imported by a single consumer on any Node version; the repository's tests ran inside the package directory, where stripping is allowed, and never noticed. It compiles to a distribution now, and the drill installs the packed tarball into a bare project. **IMPLEMENTED**

Both are held to a language-agnostic conformance suite: a runner drives any verifier over standard input and output through the published cases, 118 at 1.0.0-rc.7 across twenty-five artifact types, including the composite federation trust decision, the holder proof, the agent grant and its revocation. Passing the suite is what *conformant* means, and the normative wire specification (RFC 2119 language, twenty-seven format strings, the exact signed-field list of every artifact) is what an implementation importing no Polaris code would build against. **IMPLEMENTED**

The contract was then asked what it did not ask, by measurement, and the answer bounds what conformance means. Three artifact types had no case in which the signature was bad, so a verifier that never checked theirs conformed. Five of the published valid vectors had expired months earlier and no case noticed, because none asserted freshness. Asking about freshness surfaced two divergences between the shipped reference verifiers: neither bounded a holder proof's age, so an integrator following one got no replay protection on presentations, and neither could report whether an artifact's issuer was trusted at all, so a genuine signature by a stranger passed. All are fixed and the three verifiers agree on every case. What the contract still does not prove is stated where an integrator will read it: 44 verdict fields are unconstrained by any published case, and none can be closed by writing one, because each needs a conforming verifier to compute something it does not. One of them is not a decision at all: the window a trust attestation carries, echoed back beside the `expired` verdict that is the decision. The contract constrains what its weakest conforming implementation computes. Then the reference verifiers themselves were mutated: every refusal in each was inverted in turn, and eighteen of eighteen in the Python implementation and nine of fourteen in the TypeScript one accepted what they exist to reject with both that SDK's tests and the conformance runner green, including an agent grant's exhausted use count and, in the TypeScript SDK, a constant-time comparison whose length guard inverted

made a five-byte value compare equal to a 32-byte root. All are closed in the SDKs' own tests, and the drill inverts all 32 refusals on every push with a per-SDK negative control. **IMPLEMENTED**

6.3 Versioning, and the frozen version 1

Protocol majors live in the format string (`polaris-registry/1`), minors are advertised by the registry and are never needed to verify, and the negotiation rule is normative: reject an unknown major, accept any minor, never emit an unadvertised version, answer an unsupported one with a specific error and the supported list. Version 1 of the protocol is frozen under checksums, with the detached verifier of `v9.317` vendored beside it, and a compatibility suite runs both directions on every push: today's verifiers must accept everything version 1 published (44 of 44 cases hold), and the pinned older verifier must never accept what today's suite rejects. **IMPLEMENTED**

6.4 The outside: a verifier for wallets Polaris has never met

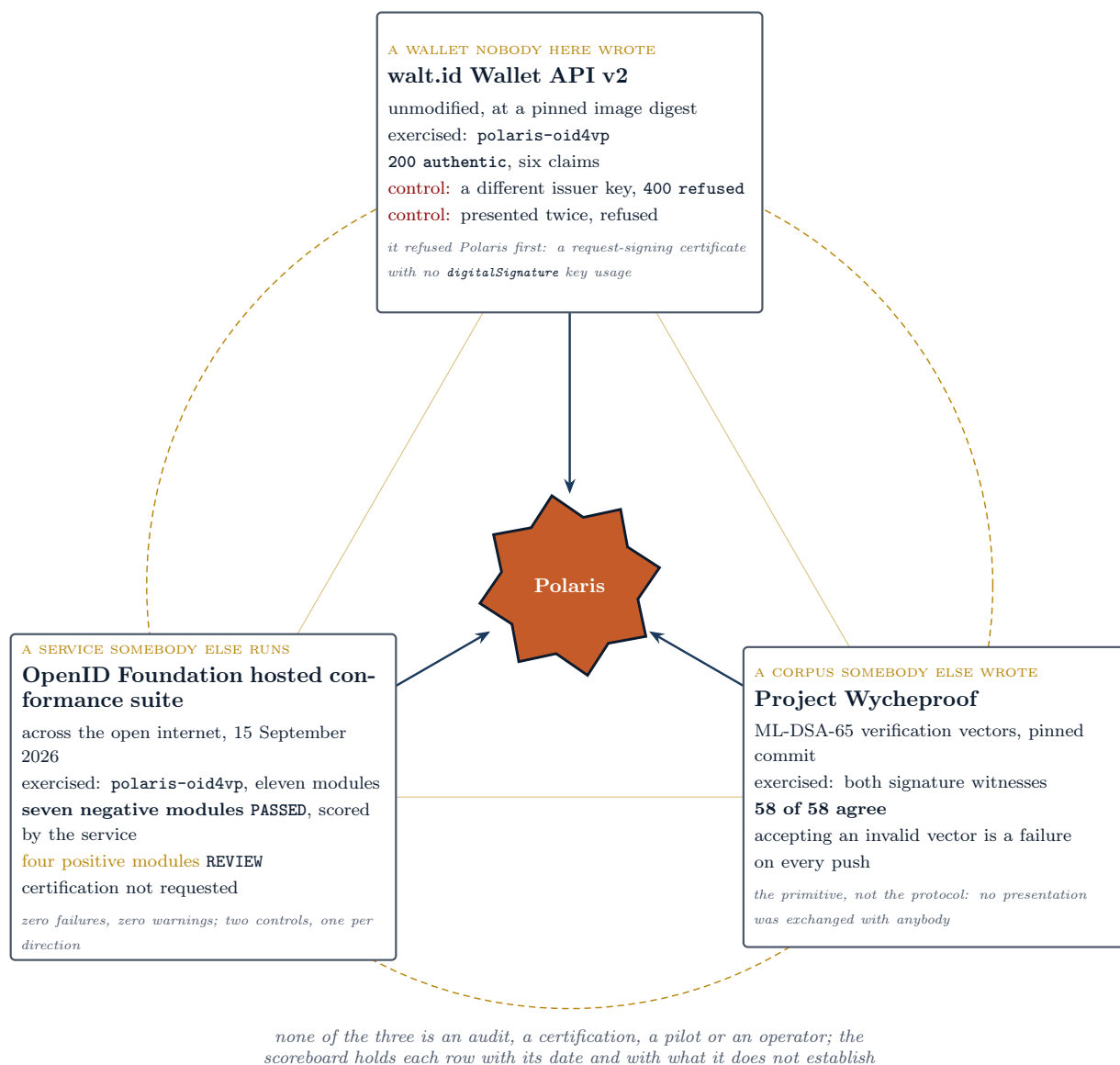


Figure 6: The three things outside the repository that have exercised it, and the two controls without which the first would mean nothing. A wallet nobody here wrote presented a credential and was accepted; the same wallet, with the verifier trusting a different issuer key, was refused; and a re-presentation against an answered request was refused. The Foundation’s hosted suite scored seven refusals itself and left four acceptances for a human reviewer. A published corpus agrees with both signature witnesses on the primitive. None of the three is an audit, a certification or a pilot.

Everything above is Polaris verifying Polaris, and Version 2’s honest boundary was that no external implementation had interoperated. That sentence is different now, on one path, and the path was chosen because it is the one an outside wallet actually speaks. Under the operating contract the first interoperability target was an OpenID4VP 1.0 verifier under the High Assurance Interoperability Profile, and the OpenID Foundation’s conformance suite has a test plan for exactly that role, `oid4vp-1final-verifier-haip-test-plan`, in which *the suite signs the credential*, so Polaris’s own ML-DSA issuance is not involved. `polaris-oid4vp` is that verifier: it checks an SD-JWT VC signed with ES256, with holder key binding, delivered as an encrypted `direct_post.jwt` response, and it refuses with a named reason, one per negative module of the plan,

issuer_signature, sd_hash, kb_signature, nonce, audience, kb_freshness. It is a separate package because polaris-verify promises to open no socket and this one must listen. One of its fixtures is not self-referential: a complete response produced by the conformance suite's own wallet, built by a Java library this project does not use, is committed, decrypts, and verifies, and is refused a year later, under another nonce, under another audience, and with one disclosure rewritten. **IMPLEMENTED**

Then two things happened that the repository did not do to itself. On 15 September 2026 the OpenID Foundation's hosted conformance service ran all eleven modules of the plan against the verifier across the open internet, fetching the request object and posting the response from outside: zero failures, zero warnings. The seven negative modules, where the suite's wallet sends a presentation broken in one specific way, carry the service's own result: **PASSED**, scored automatically on the verifier's refusal, which makes them the closest thing to an external adversary this project has been offered. The four positive modules finished in **REVIEW**, the state the suite reserves for a human to attest that the verifier displayed a successful verification; the screenshots are attached and a Foundation reviewer sees them only at certification submission, which has not been requested. **REVIEW** is not **PASSED**, and Polaris is not certified. The first write-up of the run said eleven of eleven clean, and it was false in a flattering direction: one fixed alias had been used for every module, the suite interrupts a test when another starts under the same alias, and the four positives had been killed by their successors, which the suite had already recorded inside the very log downloaded to corroborate the claim. It was re-run one module at a time. The run carries two negative controls, one per direction: a verifier patched to refuse everything is noticed by the positive modules, one patched to accept everything by all seven negatives. **EXTERNALLY EXERCISED** for the profile; **EXTERNAL VALIDATION REQUIRED** for certification.

And on the same day an unmodified walt.id Wallet API v2, the published image at a pinned digest, presented an SD-JWT VC to the verifier over OpenID4VP and was accepted: it fetched the signed request object, verified it against the registered trust anchor, matched the credential to the query, signed a key-binding token with a P-256 key it generated and this repository has never held, encrypted the response, and polaris-oid4vp verified the chain. It refused Polaris first, and it was right. The verifier's own key generator had produced a request-signing certificate with no key-usage extension at all, so the certificate whose entire job is signing request objects was not marked as usable for signing; eleven conformance modules, 142 package tests, a mutation drill over every refusal and every check in the invariant layer had passed over it, and the first implementation from outside declined to proceed. Fixed the same day, with a test that is red on a leaf missing it, and counted on the scoreboard as the first of the fixes caused by an external finding; the second, two days later, was an outside reviewer's. The exchange stands on the same two controls, because a verifier that accepts everything prints the same success line. What it does not establish is stated in the same place: one wallet, one credential format, one presentation path, ES256 throughout, which is what the profile pins; nothing about any other wallet, about ISO 18013-5, or about the post-quantum path, and no claim that the verifier is interoperable in general. Anyone can repeat the exchange from a clean machine in about ten minutes without cloning the repository; the page that says how was executed before it was written. **EXTERNALLY EXERCISED** for this path.

The native protocol, the polaris-*/1 formats that everything else in this paper is built on, remains where Version 2 left it: the specification, the cases and two SDKs exist, and no implementation by an external team from the documents alone has happened. **EXTERNAL VALIDATION REQUIRED**

Two format bridges make the boundary explicit. A Polaris credential renders in the ISO/IEC 18013-5 mobile-document structure, and a reader that speaks the standard parses it, walks its namespaces and verifies every disclosed element's digest against the signed security object, with an independent CBOR implementation proving it rather than the one that wrote the bytes; the reader cannot verify the issuer signature, because it is ML-DSA-65 and the standard mandates

classical algorithms, and the repository calls that the design rather than a limitation, since a post-quantum credential carrying a classical signature to earn a green tick would be a classical credential. A verification result also renders as a W3C Verifiable Credential, deliberately not an identity credential: its subject vocabulary is closed and refuses identity fields by name, and its subject identifier is the per-verifier handle or absent. Both are formats, not trust models. **IMPLEMENTED**

Layer card: independent verification

What it is	Four packages on public registries: a standalone verifier, two SDKs and an OpenID4VP verifier; a conformance runner over 118 published cases; a normative wire specification; a frozen version-1 set with a cross-version suite; a fuzzer, an attack suite and three mutation drills; two format bridges.
Why it exists	A verification the issuer performs asks the relying party to trust the issuer's server. Authenticity has to be decidable by anyone, anywhere, with the network off, and by software the author did not write.
What it trusts	The published issuer keys the relying party chose to trust (its anchor set); a standard ML-DSA library, or for the OpenID4VP path, a registered X.509 trust anchor and ES256.
What it refuses	Any Polaris code or database; any input it cannot parse; any object presented to the wrong type's verifier; any version it was not told about; to start without a declared crypto mode; a nonce or audience it did not itself ask for.
What it can see	The artifact and the anchor set. For an SD-JWT VC, exactly the disclosed claims. Nothing else.
What it cannot see	Whether the credential is still authorized, unless a status artifact is presented with it; the 44 verdict fields the published cases do not constrain.
If it fails	A wrongly accepted mutation fails the fuzzer; a refusal that stops refusing fails the SDK mutation drill; a drift between specification and signer fails <code>check_wire_spec_matches_code</code> ; a package that cannot be installed by a stranger fails the boundary drill; a defect every internal instrument missed is what the outside is for.
Checked by	<code>check_detached_verifier</code> , <code>check_conformance_suite</code> , <code>check_typescript_sdk</code> , <code>check_verifier_fuzz</code> , <code>check_attacks_run</code> , <code>check_protocol_versioning</code> , <code>check_sdk_refusals_are_mutation_tested</code> , <code>check_conformance_asks_the_relying_party_question</code> , <code>check_oid4vp_verifier_boundary</code> , <code>check_documented_verifier_commands_run</code> .
Now or later	Now, for the software and for one outside wallet on one path. Certification, a second wallet and an implementation of the native protocol by someone else are later and not ours to do.
Inside and outside	Inside: the signatures. Outside: the status layer, which answers the question a signature cannot.

Signing a credential proves who signed it. It does not prove whether that credential is still authorized: the seal on a revoked passport is as genuine as the seal on a valid one. Polaris therefore needs status.

7 Verification is not authorization: status, revocation and the offline trade

7.1 Two results, never one

The whole product is visible in one object. A relying party asks the issuer about a credential and receives, for a revoked token:

```
POST /api/v1/verify -> {
  "authentic": true,           // the ML-DSA signature is genuine
  "currently_authoritative": false, // but this token has been revoked
  "status": "REVOKED",
  "usable": false,           // authentic AND authoritative -> false
  "decision": "reject"
}
```

Authenticity is permanent and cacheable. Authorization is fresh and revocable. A relying party needs both, and the API refuses to collapse them into one flag, because a signature stays genuine forever while the authority under it can be withdrawn at any moment. **IMPLEMENTED**

The relying-party endpoint authenticates the caller as an organisation, not a person, by an OAuth2 client-credentials grant; it accepts a possession proof rather than a sequential identifier, so it cannot be used to enumerate tokens; it returns a uniform verdict on a mismatch, so it is not an existence oracle; and it keeps no record of who verified whom. **IMPLEMENTED**

7.2 Fresh state, not a stale replica

The verify endpoint is routed to a read replica for throughput, and the repository's history records the mistake that split the two results permanently. Authenticity is a property of immutable material, the signed value, the signature bytes and the stored key, so a lagging replica can neither forge it nor deny it. Whether the token is usable *now* is a current-authorization question, and a revocation lands on the primary; a replica inside its staleness window could still show a just-revoked token as active. Since v9.264 the status that decides `usable` is pinned to the primary, the response names its source and carries `as_of` and a zero staleness bound, and a caller may cache authenticity and must re-read authorization. **IMPLEMENTED**

7.3 Revocation

Revocation is a status transition through one sanctioned procedure that also writes to `RevocationList` in the same transaction, so the verifier-facing list can never diverge from the token's state, and a trigger refuses any other path into `REVOKED`. The list carries a canonical reason vocabulary; the audit trail carries the procedural detail and the co-signer when the issuer-discretion ceiling required one. Revocation looks forward: events recorded while a credential was valid are not invalidated, because history is not rewritten. **IMPLEMENTED**

7.4 Authorization with the network off



Figure 7: The offline trade, drawn as a timeline. The issuer signs a status assertion with a window; the holder staples it; a relying party verifying offline inside the window accepts. If the issuer revokes the credential inside the window, the last **ACTIVE** assertion stays valid until it expires, and an offline verifier cannot know. The window is bounded by the issuer’s time-to-live (an hour by default) and by the `max_window` the verifier imposes; a high-risk relying party demands seconds or goes online.

An offline verifier cannot ask the issuer. Polaris answers with a short-lived signed status assertion: the issuer’s signature, under the same key that signed the credential, over the token’s current status and a window. The holder fetches it by presenting the genuine credential, with no bearer and no personal data, and staples it to a presentation. A verifier accepts offline only if the credential is authentic, the assertion is authentic, bound to this credential, fresh on the verifier’s own clock, no wider than the window the verifier will accept, and **ACTIVE**. The rule that a verifier never accepts a credential more than one window after its revocation, and that the window is the one the verifier set rather than the one the issuer minted, is now a formal model checked on every push with a configuration it must fail. **IMPLEMENTED**

The cost is stated as plainly as the benefit. Because verification touches no issuer, the issuer never learns it happened: offline is more private than online. And because it touches no issuer, a revoked credential’s last assertion stays valid until it expires. Issuer non-observation, offline availability and revocation freshness cannot all be maximised at once; a deployment picks two, and the window is a policy number, not a law. One consequence is recorded on the scoreboard as a known limitation rather than fixed: the issuer mints a one-hour window, the reference verifier’s own default bound is unset, and the ceiling the artifact itself carries is not read by the assertion check, so the maximum offline lifetime of a status rests partly on the relying party’s configuration. A relying party that needs seconds sets seconds or goes online.

7.5 Status across authorities, and through networks that are not trusted

Two more signed objects carry status between authorities without a callback the issuer could log. An epoch checkpoint is an authority's commitment to the latest point on its epoch chain, the epoch number, its Merkle root and the prior epoch it extends; two checkpoints prove monotonicity, and two different roots signed at one epoch number are cryptographic proof that the authority equivocated about its own history. A revocation feed is the sorted set of revoked-credential leaves, each the SHA3-256 of a token value, derivable only by the holder; a newer feed that drops a published revocation is a caught rollback, because the underlying table is append-only. At federation scale an aggregator mirrors many authorities' feeds and checkpoints, verbatim and still under each member's own signature, into one short-lived bundle. The aggregator is untrusted for correctness: in full control of the bundle, recomputing the commitment and re-signing the envelope, it still cannot forge a member's status, because it cannot sign as the member. The bundle adds availability, not trust. **IMPLEMENTED**

Signing a status artifact is also what lets an untrusted cache carry it, and a cache introduces the one failure signing does not prevent, which is time. Since v9.358 a cache directive is never a constant: every windowed artifact's `max-age` is its own remaining life, computed from the `expires_at` the issuer signed, so a cache cannot outlive the window the issuer committed to; every artifact that names one credential is `no-store`, because a shared cache holding one would serve one holder's credential to another; and an expired or unparseable window is not cached at all. Distribution at content-delivery scale is thereby specified and drilled; no deployment has it. **IMPLEMENTED** for the artifacts, drills and cache rules; **PROPOSED** for a distribution deployment.

The epoch cadence is the single number that sets three unrelated properties, and a design record now says which: revocation freshness, since a revoked credential drops out of the *next* epoch; the size of the crowd, since the anonymity set is the epoch; and how often a relying party's one-human-once ledger resets. Closing an epoch was once the expensive part, and it was expensive for a bad reason: a fixed-depth tree padded to the national depth of 24 materialised sixteen million leaves whatever the population was, so a root over a thousand members took 10.8 seconds and 2.9 GB. Since v9.357 the padding is one repeated value hashed once per level, and a root over a million members takes 2.1 seconds in tens of megabytes; the binding constraint on cadence is the anonymity floor per context, not the compute. **IMPLEMENTED**

Layer card: verification and status

What it is	The relying-party verify endpoint, the revocation list, the signed status assertion, epoch checkpoints, revocation feeds, the aggregate status bundle, and the cache rules that let an untrusted network carry them.
Why it exists	Authenticity is not authorization. A verifier needs to know not only that a claim was made but that it still stands, and sometimes needs to know it with the network off.
What it trusts	The issuer's signing key, for the assertion; its own clock, for freshness; the primary database, for the online verdict.
What it refuses	A replica's opinion of current status; an assertion wider than the verifier's ceiling; an aggregator's word for a member's status; a feed not bound to the issuer's key; a cache entry that would outlive the window the issuer signed.
What it can see	The status of one credential at one instant; the set of revoked leaves.
What it cannot see	A revocation that happened after the assertion it holds was signed; the active population, which no feed publishes.
If it fails	A stale verdict is bounded by the window, and the bound is the verifier's; a forked epoch chain or a rolled-back feed is non-repudiable evidence against the authority; a lying aggregator is caught at the member's signature.
Checked by	<code>check_relying_party_api</code> , <code>check_offline_verification</code> , <code>check_epoch_revocation_propagation</code> , <code>check_federation_status_bundle</code> , <code>check_status_distribution</code> , the status-freshness model, the verify-load drills under failover.
Now or later	Now, for the artifacts and the rules. Distribution at national scale is later.
Inside and outside	Inside: the verifiers. Outside: the privacy boundary, which decides what a verifier may learn while it checks.

Status proves current authority, but proving it can reveal far more than the verifier needed, and every verification is a fact about a person's day. The next layer bounds what anyone learns, and this version reports what was measured about that bound rather than what was intended.

8 The privacy boundary: what a verifier is allowed to learn

Two things in Polaris use zero knowledge, one thing deliberately does not exist, and the distinction between them is the most important sentence in this section. Version 3 adds a fourth thing: an adversary, written to falsify the claim, and what it found.

8.1 Three disclosure levels

Every verification event is recorded at one of three levels. `ZERO_KNOWLEDGE` proves that a valid token exists without recording which: the event's token identifier is null, and a check constraint refuses any row that says otherwise (C2). `SELECTIVE` records named attributes only. `FULL` records the identity and is logged and rate-limited. The level is decided by the server and cannot be upgraded by the client (C6): a relying party cannot turn a zero-knowledge check into a full one by asking differently, and the Atlas redacts a zero-knowledge event's location server-side. The default level is zero-knowledge. **IMPLEMENTED**

8.2 Issuer-side unlinkable verification records

The first use of zero knowledge is a property of the issuer's database. A default-mode verification stores no token identifier, so the verification graph, who was verified where and when, cannot be reconstructed from the database even by someone holding all of it. This is a structural claim, not a promise: the repository carries a redaction proof that the redacted field is non-derivable under a stated adversary model, property tests exercise every read path, and the claim that two relying parties pooling everything they hold still cannot link a holder is a formal model checked on every push. The property test that measures reconstruction risk runs an adversary against isolated zero-knowledge events and asserts its success rate is no better than chance, and since v9.408 the assertion is shaped so that a weak adversary cannot flatter the result: two weaknesses

in the adversary had been invisible to the old shape. The claim is scoped precisely. It covers zero-knowledge-mode events. Selective and full events do carry a token identifier, because that is what those levels mean. **IMPLEMENTED**

8.3 The membership proof



Figure 8: Holder, relying party and issuer, with what each sees and what each is prevented from seeing. The holder presents a file; the relying party decides offline with the detached verifier or an SDK and learns that a valid credential exists and its value; the issuer records, in zero-knowledge mode, an event with no token identifier, and keeps no who-verified-whom row for the online status check. The three flows form a circle turning one way; at its centre, in red, is the stated limitation: on a plain presentation the credential value is a stable handle across the verifiers it is shown to.

The second use of zero knowledge is a proof object. A Merkle tree behaves like a cryptographic fingerprint of an entire set: instead of handing a verifier the whole set, one can hand it a short proof that a particular item belongs to the set with the committed fingerprint. Polaris commits, per epoch, a Merkle root over the active-token set, and a holder can produce a Plonky2 proof on

their own device, against a published anonymity set, that their token is a leaf under that root, bound to the epoch, a context and a nonce, without revealing which leaf. The verifier sees the proof bytes and the public inputs and cannot recover the leaf; that is the proof system's property, not an API rule. The setup is transparent, with no ceremony, and the commitments are hash-based. The default tree depth of fourteen covers the schema's ten-thousand-leaf epoch cap, so the anonymity set is a full epoch. On the reference machine a proof takes about 25 ms to make and about 12 ms to check, and the verdict is two-witnessed at the statement level. The proof carries a per-verifier nullifier, `Poseidon(secret || scope || epoch)`, so a relying party can enforce *one human, once* within its own scope while another cannot correlate its nullifiers. **IMPLEMENTED**

What the proof does not do is stated with the same care. It proves membership and binding, nothing else: whether a token is active, permitted for the context and unrevoked is decided when the epoch is composed, not inside the circuit. The nonce prevents substituting a captured proof into another context; online, a single-use nonce store refuses a replayed bundle, and offline the verifier keeps no state, so a relying party that needs single use issues a challenge. The nullifier does not hide the holder from the *issuer*, which derives every leaf in order to build the tree and could compute any member's nullifier in any scope; the property is between relying parties. And the circuit over the proof library has had no external cryptographic audit; the repository's soundness ledger says not to protect real identities with it as shipped. The proof library itself was re-evaluated against its successor at v9.361 and kept, with the triggers that would reopen the question recorded. **EXTERNAL VALIDATION REQUIRED** for the audit.

8.4 Bounded, not permanent, and what bounded was found to mean

Polaris is a full-credential presentation system, and until v9.353 the consequence was flat: the login token's subject was the credential's hash, the same value at every relying party, so two of them could join their user tables exactly and forever. The subject and the presentation handle are now derived under the relying party's own scope, stable where an account needs it and unrecognisable at the next verifier; since the subject is the value a relying party *writes down*, and written-down values are the ones pooled, sold, subpoenaed and breached, this is the half that matters most in practice. The other half did not change: a plain presentation still shows the verifier a stable `token_value`, the issuer's signature and the holder's public key, so the guarantee is about what a verifier should *store*, not about what it is *shown*, and the verifier reports which it is giving: *exposed* for a plain presentation, *bounded* for the zero-knowledge path. **IMPLEMENTED**

Version 3 can say what *bounded* was measured to mean, because under the operating contract the repository's lab asked the question directly: given complete transcripts from two presentations, what advantage does a colluding pair of verifiers have at deciding *same holder*? Three findings, in the order they were made.

The verdict did not mean what it said. *bounded* was reported whenever a scoped nullifier was present. It never checked the premise its own documentation stated. A presentation carrying the nullifier *and* the credential's stable token value, issuer key and signature was reported bounded while two verifiers could link it with a single string comparison. Fixed the same day: *bounded* now requires the nullifier and the absence of every field that is identical across verifiers, including the holder's public key, which is stable by construction and is exactly why the pairwise handle is derived from it rather than shown. The fix corrected a verdict; it made no presentation more private than it was.

No advantage beyond the anonymity set, and nothing floors it. An adversary that flattens every scalar in a transcript, drops what is constant across the population and matches on the rest, with a positive control that links the exposed population at an accuracy of one, achieves on the bounded population an accuracy that tracks $1/N$ at every size from two

holders to two hundred. Within an epoch, scoping does what the word implies against this adversary. And N is the epoch's membership. The schema constrains an epoch's committed count to between one and ten thousand and nothing floors it above one, so an epoch that closes with three members gives a colluding pair a one-in-three guess, and one that closes with a single member identifies the holder outright, while the verifier still, correctly, reports the transcript as bounded. The count is readable off the signed checkpoint; nothing requires a relying party to look, and no verdict mentions it. Reporting it would be a new product guarantee, which lab work does not get to create, so it is recorded as a limitation for a deploying organisation.

What the adversary cannot see. It matches on exact equality and nothing else. Shown a population that is perfectly linkable with no equal field in it, it stays at chance. So the null result means no field is *identical* across verifiers, not that none is *correlated*. The one channel it can read is transcript length, which carried the entire above-chance residue in that control; nothing in a bounded Polaris presentation varies in size per holder today, and a deployment where disclosed attribute sets or optional elements made it vary would open it. Presentation size, timing, issuer metadata, status artifacts and transcript structure are unmeasured, and the lab says so rather than extrapolating.

The path a stranger's wallet actually used cuts the other way. An SD-JWT VC presentation over OpenID4VP, measured the same way, leaves nine values identical across two verifiers, the holder's key, the issuer's signature, the disclosure digests, the hash of the presentation and its length, all issuer-signed so the holder cannot vary them. That is a known property of presenting one such credential repeatedly; the ecosystem's mitigation is batch issuance, many credentials with one holder key each, which the repository does not implement. Adopting the ecosystem's format imported the ecosystem's linkability; it was not visible before it was measured, and it is not a reason to stop interoperating. So cross-verifier correlation is bounded on the native zero-knowledge path, measured within the limits above, and exposed on the interoperable one. *Unlinkable by default* still means the issuer's zero-knowledge-mode records. **IMPLEMENTED** for the derivations and the measurement, with the residue stated rather than closed.

8.5 Bounded surfaces

The remaining privacy mechanisms bound what a surface can return rather than what is stored. Every map and API aggregate is hard-capped (C8), so the whole population cannot be extracted through the operator's atlas, and a request for six and a half million cluster bins returns at most five thousand. The single-entity investigation pages are built to refuse cross-individual link analysis and are audited: a meta-audit table records who read which audit surface, with what filter, and how many rows came back, and it is append-only. The authority's most invasive power, pulling one named person's whole verification history under a warrant, left no record of its own use for as long as it existed, because the read sat behind a stored procedure and nothing looking for a **SELECT** found it; since v9.382 it writes the same row, and a structural rule extends that to any read hidden behind a procedure or a view. The authority-structure layer, Athena, is forbidden by **check_athena_no_person** from referencing any natural-person table or column (Section 14). And timestamps, the timestamp authority and the exchange gateway are built to learn nothing about what they touch, which Section 12 describes. **IMPLEMENTED**

Layer card: the privacy boundary

What it is	Three disclosure levels with a CHECK behind the default; server-side enforcement; the redaction proof and its adversary; the epoch membership proof, its nullifier and its second witness; the per-verifier handle; bounded aggregates; an audited single-entity investigation surface.
Why it exists	A verification log is a surveillance backbone unless it is structurally unable to be one.
What it trusts	The C2 constraint; the proof library's soundness; the redaction proof's adversary model, and the linkability adversary's stated blind spots.
What it refuses	A client's claim about its own disclosure level; a token identifier on a zero-knowledge row; an unbounded read; a person node in the authority layer; a verdict of bounded over a transcript with any field identical across verifiers.
What it can see	At zero knowledge: that a valid token was verified at a time in a context. At full: the identity, logged.
What it cannot see	Which token, in zero-knowledge mode, even from the whole database; and the size of the crowd a given proof hides in, which no verdict reports.
If it fails	An identifier on a zero-knowledge row is unstorable; a cap removed fails the build; a person reference in Athena fails <code>check_athena_no_person</code> ; an adversary weakened to flatter the result fails the property suite's own control. Cross-verifier correlation on a plain presentation is not a failure; it is the documented boundary.
Checked by	The C2 CHECK and the C2 model, <code>check_c6_atlas_redacts_zk_location</code> , the redaction property suite, <code>check_redaction_adversary_is_not_flattered</code> , <code>check_pairwise_presentation</code> , <code>check_scoped_nullifier</code> , <code>check_audited_reads_are_logged</code> , the ZK second-witness differential, <code>check_athena_no_person</code> .
Now or later	Now. Linkability-resistant presentation for the interoperable path, and a crowd-size floor, are a different system and later.
Inside and outside	Inside: status. Outside: the holders and relying parties who use the credential.

A credential bounded in what it reveals still has to be used: held, presented, accepted and logged in with. The next layer is where the credential meets software that is not the issuer's, and where, for the first time, that software was not the author's either.

9 Using the credential: holders, relying parties and the login that keeps no record

9.1 The wallet is a protocol, not a product

The reference holder agent, `scripts/polaris-wallet.py`, holds a credential as a file, shows it, verifies it offline through the detached verifier, presents it, proves membership in zero knowledge against a published epoch, signs a document on the holder's behalf, delegates a bounded authority to an agent (Section 17) and logs in to a relying party. Every message it emits is specified so that any client can emit the same. Native mobile and desktop clients and a browser bridge are product engineering that a reference implementation on notional data does not attempt; they would follow the protocol, and never lead it. The physical presentation path, a card that signs a device's challenge, is the emulator and verifier device of Section 3. **IMPLEMENTED** for the protocol and the reference script; **PROPOSED** for native clients.

A presentation is an unsigned wrapper around signed things: the issuer-signed credential, an optional stapled status assertion so that authorization is decidable with no connectivity, an optional membership proof, an optional holder signature over the verifier's nonce, the context and disclosure level presented under, and an opaque presentation code. The verifier decides it offline and reports the code as present or absent without ever interpreting it, because a duress code is matched only by the issuer, out of sight; a duress presentation is byte-for-byte the same shape as a normal one. For transfer by QR or NFC the presentation is compressed and split into frames that each name the digest of the whole; a receiver takes them in any order and refuses, before parsing anything, frames naming different digests, a missing index, a conflicting duplicate, or a

reassembled payload whose digest does not match. The decoder is resource-bounded: at most 9,999 frames, 512 KiB compressed and 2 MiB inflated, so a decompression bomb is refused at the limit rather than decompressed. The bounds were added after an outside review named the gap. **IMPLEMENTED**

9.2 The relying party

The reference relying party (`scripts/polaris-relying-party.py`) decides accept or reject for a presentation by combining offline authenticity with online status, refusing a revoked, tampered or foreign-issuer credential and staying blind to duress, and an end-to-end drill runs the whole wallet-to-relying-party matrix under real signatures every release. A relying party registers with an issuer as an organisation with a scope that a schema constraint bounds to exactly `verify`, `authenticate`, or both, so that a fourth kind of access is a constitutional change rather than a configuration one. Identity never becomes a login product by accident. Since v9.425 a relying party is also an object about which decisions are recorded: registering one, turning off the zero-knowledge step-up its holders had to satisfy, disabling or re-enabling it, each writes an append-only event, a change that weakens what the party must satisfy is flagged as such and refused without a stated reason, and a change made from a database shell is recorded on the same terms as one made through the tool, because the writer is a trigger and not the tool. The first version of the rule that decides what counts as a weakening ranked a capability by the length of its name, so granting a capability the party did not hold read as harmless; it was found by running it against a loaded database, not by reading it. **IMPLEMENTED**

9.3 Logging in without a login record

The auth broker lets a relying party offer *log in with a Polaris credential* through an authorization code with PKCE. The holder presents the credential at its issuing authority's instance by possession, and the relying party exchanges the resulting code for an identity token signed by the issuing agency, which it verifies offline. Four guards keep this from becoming a login record. The token's subject is the per-relying-party handle of Section 8, never a person identifier and, since v9.353, not the same value at every relying party. The token carries no claim beyond the vocabulary: context, disclosure level, the assurance reached, enrolment status, instants. The authorize route writes nothing, and the token endpoint's only write is the code's hash into a register that holds no subject and no relying party. And duress is served identically. **IMPLEMENTED**

An outside review at v9.331 found that the relying party's demands, a required zero-knowledge step-up or a required enrolment status, arrived only in the holder-side request, so nothing bound the authority to what the relying party actually required. Since v9.336 the policy is registered on the relying party's row and applied first: a request may add a requirement and never remove one, and a context other than the registered one is refused. The same review noted that the authorization code was signed but readable; it is now encrypted. And the conformance contract was later found to verify an identity token's signature and never its audience or nonce, so a verifier that ignored both, and therefore accepted a token minted for one relying party at another, passed every case; four cases fixed it. **IMPLEMENTED**

9.4 The stranger's path

Version 2 could tell a reader how to run Polaris from a clone. Version 3 can tell a reader how to reach one accepted presentation from software nobody here wrote, on a clean machine, in about ten minutes, without cloning the repository: install `polaris-oid4vp` from the registry, start the published wallet image at its pinned digest, generate the verifier's keys, give the wallet a credential bound to a key it generated itself, run the verifier, present. The page that says how was executed from an empty directory before it was written down, which caught its own defect on the way (a

buffered pipe that left the verifier's log empty), and it ends with the line that matters: `<- 200 authentic, claims ['cnf', 'family_name', 'given_name', 'iat', 'iss', 'vct']`. If it fails for a reader, the page says, that is the most useful bug the project can receive. Nobody outside the repository has yet reported walking it. **IMPLEMENTED** for the path; **EXTERNAL VALIDATION REQUIRED** for a walker who is not the author.

9.5 What is deliberately not built

There is no session at the authority, no consent screen, no discovery document, and no browser redirect choreography: a relying party integrates from the wire specification and the registry. A WebAuthn browser bridge is not built; the holder-side key it would rest on now exists. No relying party outside the repository has integrated against the native protocol. One outside wallet has presented over OpenID4VP, and no outside relying party has yet run the verifier that accepted it. **EXTERNAL VALIDATION REQUIRED**

Layer card: use

What it is	The wallet protocol and reference script, presentations and QR framing, the reference relying party, relying-party registration with a bounded scope and a recorded history, the auth broker, and the stranger's path to an accepted presentation from an outside wallet.
Why it exists	A credential nobody can hold, present or accept is a database row. This layer is where the credential meets software the issuer does not run.
What it trusts	The signed objects inside a presentation; the issuer's key for the identity token; the relying party's registered policy.
What it refuses	To interpret the presentation code; to write a login record; to accept a scope outside the CHECK; to let a holder's request weaken the relying party's registered demand; to lower a relying party's bar without a reason on record; to decompress an unbounded payload.
What it can see	A relying party sees the credential value and the verdicts, or on the zero-knowledge path a proof and a scoped handle. The authority sees a possession proof and consumes a code hash.
What it cannot see	The authority cannot see who logged in where; the relying party cannot see the person's record.
If it fails	A replayed code hits the consumed register; a wrong presentation, an unmet step-up, a verify-only bearer at the broker, a stale or tampered frame are each refused in the drills; a weakened relying-party bar with no reason is refused by the database.
Checked by	<code>check_holder_wallet</code> , <code>check_wallet_presentation</code> , <code>check_holder_verifier_flow</code> , <code>check_auth_broker</code> , <code>check_broker_policy_bound</code> , <code>check_relying_party_api</code> , <code>check_qr_resource_bounds</code> , <code>check_documented_verifier_commands_run</code> .
Now or later	Now, as a protocol, and now for one outside wallet on one path. Native clients and an outside relying party are later.
Inside and outside	Inside: the privacy boundary. Outside: federation, because a relying party rarely trusts only one authority.

A relying party that trusts one authority is easy. A society has many authorities, none of which should be able to speak for the others, and none of which should hold every person's record. The next layer is how independent authorities recognise one another without collapsing into one.

10 Federation: independent authorities that recognise one another

10.1 Why there is no centre

Two shapes were possible. A central instance holds every person's identity and every agency's tokens under one trust root, with agencies as tenants. A federated shape gives each issuing authority its own instance and its own signing key, records cross-authority trust explicitly, and lets a relying party verify against published keys with no central service in the path. The decision record chooses the second and states that it is not a free choice: the constitution names the federation trust graph as load-bearing, says no agency holds a monopoly, and refuses population-scale aggregation structurally. A central instance is a monopoly by construction and a single store to seize, subpoena or mine. The federated shape is deliberately the more expensive one; its cost is the inter-authority protocol, and the cost is accepted. **IMPLEMENTED**

10.2 Trust is a row, not a chain

An attestation is one row of **AgencyTrustAttestation**: *verifying agency V accepts issuing agency I in context C, until a date*. Verification across agencies succeeds if and only if exactly one active row matches the verifier, the issuer and the context. The lookup is a single non-recursive query. It never computes a closure, so V trusting I never implies V trusts what I trusts; cycles cannot arise because nothing walks the graph; and an operator who wants multi-hop trust declares every hop, so the audit trail holds every decision rather than the inputs to an inference. Self-attestation is refused by a check constraint. Revocation sets a date and a reason together, once, and looks forward: events recorded while the edge was active are not invalidated. **IMPLEMENTED**

One limit is named rather than glossed. The row records which Polaris operator created the attestation; it is not a cryptographic signature from the attesting agency. Agency-level signing of attestations is a migration the schema is shaped for and a key-management problem the readiness ledger assigns to a deploying organisation. **EXTERNAL VALIDATION REQUIRED**



In CI this runs as two independent instances over HTTP, each with its own database and real key, under a mixed pair of parameter sets. Two instances of one code base, run by one author, prove the protocol; they do not prove institutional independence.

Figure 9: Two authorities and one directional attestation. B attests A's key in one context; a relying party that trusts B accepts A's credential in that context, offline, from B's manifest and A's revocation feed. A also trusts C, and that gives B nothing: trust is never a closure. In CI this runs as two independent instances over HTTP under a mixed pair of parameter sets.

10.3 The three signed objects an authority publishes

A relying party needs three things from a foreign authority, and all three are signed views over append-only tables, published at short-lived endpoints and consumed offline by the detached verifier. The federation manifest carries the authority's own anchors, every key it has ever used with its real status, and the attestations it has made, each with the attested key. The epoch checkpoint is its commitment to the latest point on its epoch chain. The revocation feed is the set of revoked leaves. A manifest must be self-consistent, signed by one of the active anchors it declares, so a stranger's key cannot sign one; authentic, under two witnesses; fresh, within a window the consumer caps; and trusted, from an authority whose anchor the relying party chose. The cross-authority decision then reads: accept if and only if the credential's signature is genuine,

the trusted manifest attests to that signing key in the presented context, and the issuer's feed, fail-closed, does not list the credential. **IMPLEMENTED**

A holder can also prove membership in a foreign authority's epoch in zero knowledge. The proof already carries the epoch root as a public input; the root already travels the federation signed in the checkpoint. The decision composes the two: trust the foreign checkpoint through the graph to obtain a trusted root, require the proof to bind to that root, epoch and context, then check the proof by shelling to the prover binary as a local subprocess, with no network. If the binary is absent the verifier abstains, which is never a false accept. The verdict names an authority and a decision, never a token. **IMPLEMENTED**

10.4 One instance, many authorities, and the part policies cannot fix

A Polaris instance can hold many agencies; the schema permits it and the shipped seed demonstrates six. A review at v9.364 asked whether one authority's operators could see another's data in such an instance, and found that they could: an operator account carried no authority binding at all, and sixteen of the seventy-four operator routes read credential or holder data with no issuing-agency filter, the token list plainest among them, selecting every token joined to every holder with no notion of whose it was. Operators are now bound to an agency and every such route filters on it. What the review also recorded is the part no filter fixes: in a shared instance the database owner, the backup and the schema are shared, so isolation inside one instance is a courtesy the application extends, not a boundary the schema holds. The topology the design record chose is that each authority runs its own instance, and a shared instance is a deployment decision whose cost is stated. **IMPLEMENTED** for the binding and the filters; **EXTERNAL VALIDATION REQUIRED** for a boundary between authorities, which is a deployment.

10.5 Two instances, and what they prove

A dedicated CI job boots two independent instances as two authorities, each with its own database and its own real ML-DSA root, talking only over HTTP, and drives the whole matrix: cross-verification from a fetched manifest, attestation revocation flipping the decision, epoch checkpoints and revocation feeds fetched and verified over the wire, a key compromise recorded on one side reversing the other's acceptance, the exchange gateway, the timestamp service read out of a fetched registry, the broker's login, document signing through the wallet, and a mixed pair of parameter sets (ML-DSA-87 on one side, ML-DSA-65 on the other). It is red on any wrong decision. **IMPLEMENTED**

What the job proves is that the protocol works between two instances of this code base run by one author on one machine. It does not prove institutional independence, and it does not prove that an independent implementation, written by someone else from the specification, interoperates on the native protocol. The first requires deployment by institutions that are not the author. The second requires an external team, and has not happened; the one outside implementation that has spoken to Polaris did so over OpenID4VP (Section 6), not over these formats. The repository draws this line itself: protocol independence, independently operated institutions and independent third-party implementations are three different claims, and only the first is demonstrated. **EXTERNAL VALIDATION REQUIRED**

Layer card: federation

What it is	Per-authority roots, directional per-context attestations, the signed manifest, epoch checkpoint and revocation feed, the aggregate status bundle, cross-authority zero-knowledge membership, per-authority operator isolation, and the two-instance drill.
Why it exists	No agency may hold a monopoly, and no single store of every person may exist; yet a relying party must be able to accept a credential issued elsewhere.
What it trusts	The anchors the relying party itself chose; the append-only tables the signed objects are views over.
What it refuses	Transitive trust; a central service in the verification path; a manifest not signed by its own declared anchor; an aggregator's word for a member's status; a callback the issuer could log; an operator reading another authority's holders.
What it can see	Published trust data: keys, attestations, epoch roots, revoked leaves. No person.
What it cannot see	The active population of any authority; who verified whom.
If it fails	A forged manifest fails self-consistency; a forked epoch chain or a rolled-back feed is non-repudiable evidence; a compromised issuer key is retired from an instant by a trust list the issuer does not control; a shared instance's owner is outside what any filter can bound.
Checked by	check_federation_topology, check_inter_authority_protocol, check_epoch_revocation_propagation, check_cross_authority_zk, check_federation_two_instances, check_exchange_trust_directional, check_route_guards_are_derived_not_listed.
Now or later	Now, as a protocol between instances. Independent institutions and implementations are later, and are not ours to supply.
Inside and outside	Inside: use. Outside: transparency, because a signed history can still be told differently to different people.

Signed federation objects prove what an authority said. A dishonest authority can still say different things to different observers, publishing one epoch chain to an auditor and another to a relying party. Polaris therefore needs transparency: a way for observers to prove that history only ever grew, and to compare notes.

11 Watching the watchers: logs, witnesses, the split view and the public report

11.1 The gap after anchoring

Polaris periodically commits a batch of audit rows to a Merkle root and records the root in an append-only table, so that an outside verifier can prove that the history under a root has not been revised since. That leaves one gap. A single root proves the rows under it; it does not prove that the operator did not later discard a root and publish a different one in its place. A transparency log closes the gap by committing to the whole sequence of roots and letting anyone prove that the sequence only ever grew. **IMPLEMENTED**

11.2 The log

The log's entries are the anchor roots in append order; nothing new is stored, because the underlying table is already append-only. Over those entries the application builds a Merkle tree in the style of Certificate Transparency (RFC 6962), with SHA3-256 and distinct prefixes for leaves and interior nodes so that one can never be presented as the other. It publishes a signed tree head, the log's commitment to its entire history at a size; a consistency proof that the size- m tree is a prefix of the size- n tree, which exists only if the first m entries were never reordered, rewritten or dropped; an inclusion proof for any entry; and the entries themselves, so that a monitor can replicate the log and recompute everything. The same machinery publishes two more logs: the hashes of minted exchange receipts, and the hashes of anchored timestamps. There is no personal data in any of it. **IMPLEMENTED**

11.3 The monitor, and what it cannot see

A monitor is a standalone program anyone runs. It keeps one piece of state, the last head it accepted; each cycle it fetches the current head, verifies the signature against a key it was given out of band, fetches a consistency proof from its last head to the new one, and either records the new head or alerts on a rewritten entry, a fork, a shrunk tree, a head signed by the wrong key, or a timestamp that went backwards. What it proves is consistency: the operator cannot rewrite history without every monitor that saw the old head detecting it. What it cannot prove on its own is that two monitors were shown the same head at the same size. A log can show one observer a private fork of history. **IMPLEMENTED**

11.4 Witnesses, gossip and the equivocation proof



The protocol for independent witnesses is built and drilled under real signatures. Institutionally independent witnesses, a bank, a civil-society group, another authority, exist only once the system is deployed; today the witnesses are three processes in a drill.

Figure 10: The log, the monitor, the witnesses, the gossip pool and the equivocation proof. The log publishes a signed head and proofs. A monitor proves the log only appended. Witnesses cosign heads they found consistent, a relying party requires a threshold of them on the exact head, and witnesses gossip the heads they were shown; two log-signed heads of the same size with different roots are a non-repudiable proof that the log lied. Heads can also be published into an external append-only ledger that returns a receipt.

A witness is a monitor that does two more things. It cosigns a head with its own key, only when the head is an append-only extension of the last one it cosigned, and it gossips the log-signed

head it saw into a shared pool. Two uses follow, both verified offline. A witnessed checkpoint is a head carrying cosignatures from at least K distinct trusted witnesses, so a split view needs K independent witnesses to equivocate, not just the log. An equivocation proof is two heads for one log, both validly signed by the log's key, at the same size with different roots; the log signed both, so it cannot repudiate the lie, and two gossiping witnesses produce the proof the moment they compare notes. A witness that is itself shown a second head at a size it already cosigned refuses and writes the proof. **IMPLEMENTED**

An external ledger completes the picture: the log publishes each head into an independent append-only ledger and receives a receipt, the ledger's own signed head plus an inclusion proof, so the log cannot use a head it has not publicly committed and the ledger cannot later drop it. A ledger is itself an append-only log, so this reuses the same machinery and is monitored the same way. The file-backed ledger is implemented and tested; the chain-backed drivers are declared and wait on their interfaces. **IMPLEMENTED** for the file backend; **PROPOSED** for a chain.

11.5 The public report: saying out loud how often the power was used

The logs prove that history was not rewritten. They do not tell the public what the history contains, and one part of it matters more than the rest: an identity authority asks to be trusted with the power to pull one named person's entire verification history under a warrant, and every control around that power, the warrant, the role check, the disclosure model, is invisible from outside. Since v9.382 the authority can publish a transparency report that says how often it used that power, built from the audit-access record the read now leaves (Section 8). Building it began by asking what was recordable and finding the answer was nothing, because the read had left no trace of itself. Small counts are withheld rather than published as zero, because a report with a cumulative total lets a reader subtract one period from the next and recover the cell, so the report carries no running total to subtract from; a drill on every push attempts exactly that recovery and must fail. The report is a surface where the authority makes its own most invasive act countable by strangers. It says nothing about an access that was never recorded, and the review packet names that as a limitation of every authority's own log. **IMPLEMENTED**

11.6 Who runs the witnesses

The strength of this defence in practice is exactly the independence of whoever runs the witnesses. What is built is the protocol: the cosignature format, the threshold rule, the equivocation proof, and a witness daemon anyone can run, all drilled under real signatures every release with three witnesses catching a fork two ways. What cannot exist until deployment is a set of institutionally independent witnesses, a bank, a civil-society group, another authority, actually running it. Today the witnesses are three processes in a drill. The repository states that this is not a software deficiency but the absence of a deployment, and this paper repeats it, because a transparency layer with witnesses run by the party it watches is not one. **EXTERNAL VALIDATION REQUIRED**

Layer card: transparency

What it is	Three RFC-6962-style logs over append-only tables; signed heads; consistency and inclusion proofs; a monitor; witnesses with cosignatures, a threshold and gossip; the equivocation proof; external-ledger publication with receipts; a public report of the authority's own warrant reads.
Why it exists	A signed object proves what was said; only a watched sequence proves that what was said was not later unsaid; only compared notes prove it was said the same way to everyone; and only a published count lets the public ask how often the watched thing happened.
What it trusts	The log's key for the head; the witnesses' keys and their independence, which is a deployment fact; the authority's own record of its reads.
What it refuses	A head with no consistency proof from the last one; a head not cosigned by the threshold; a receipt the ledger did not record; a second head at a size already seen; a report from which a small count can be recovered.
What it can see	Roots and hashes, and counts of warrant reads. Never a person, never a payload.
What it cannot see	On its own, a split view: that is what the witnesses are for. An access that was never recorded.
If it fails	A rewrite fails every monitor; a fork is a non-repudiable proof; a dishonest witness is one of K ; a dropped published head fails the ledger's own consistency; a recoverable small count fails the report drill.
Checked by	<code>check_transparency_log</code> , <code>check_transparency_gossip</code> , <code>check_transparency_publication</code> , <code>check_timestamp_transparency</code> , <code>check_transparency_program</code> , <code>check_audited_reads_are_logged</code> , the canonical-equivalence oracle for the signed head.
Now or later	Now, as a protocol with a drill. Independent witnesses are a deployment, later.
Inside and outside	Inside: federation. Outside: the institutions that exchange, sign and timestamp under this watch.

With authority, status and history checkable by strangers, institutions can build on the credential: exchange requests, sign documents, attach time. The danger is that an exchange fabric becomes the very message log the vocation forbids. The next layer is built as its inversion.

12 Institutions: exchange, time and signatures without a message log

12.1 The inversion

A conventional exchange fabric between institutions makes an exchange provable by logging the exchanged messages: a mediating node signs and timestamps the traffic so that a receipt can later be shown to a third party. The evidence is real, and so is its cost: a store of who asked what about whom, which is exactly the population-scale record the vocation forbids. The Polaris exchange fabric is the inversion. It proves the same three things a message log proves, that an exchange occurred, between these parties, and was authorized, while retaining none of the payload, by committing to the request and the response by hash and never by content. Every primitive in it, signatures, the detached verifier, the transparency logs, the trust graph, already existed; the fabric composes them. **IMPLEMENTED**

12.2 The registry: discovery, not configuration

A consumer learns what an instance offers and trusts from one signed artifact: the protocol formats and versions it speaks and its accepted algorithms, each service's kind, path template and authentication, the transparency logs, every federated authority with its key register, the contexts and the proof each requires, the in-context trust graph, and the relying parties by name and scope only. Every fact is a view over the authority layer; the registry adds no truth of its own. It must be signed by the active key it lists for its own publisher, so an impostor cannot publish one in an authority's name, and it is non-transitive: it describes its publisher's instance,

never another's. The two-instance drill reads a service's path out of a fetched registry and calls it. No token, no holder, no verification record appears: a map of the institutions, never of the people. One thing the registry does not carry is recorded in Section 5: the accepted algorithms appear as a bare list of names with no status, so a verifier reading it cannot learn that one has been deprecated. **IMPLEMENTED**

12.3 The gateway and the receipt

The receipt commits to the requester's key, the responder, the context, the two body hashes, which attestation authorized the requester, and the instant. The mint endpoint accepts only hashes, so the application cannot retain what it is never given. A third party verifies offline that the receipt is authentic and that the requester was authorized in that context, given the manifests it trusts. A party that holds a body can confirm the hash binds; a party that does not still holds the responder's signed account. One correction from the outside review belongs here: a receipt alone cannot prove the requester took part, because a dishonest responder can sign any receipt it likes; the requester's own signed envelope beside it can, and the wire specification names the pair as the evidence. The set of receipts is itself a transparency log, so an institution cannot later deny a receipt existed and anyone can see how many it minted, while each receipt stays in the hands of its parties. **IMPLEMENTED**



Figure 11: An institutional exchange through the gateway. The requester signs an envelope under its registered key; the responder's own instance authenticates it by a key it already knows, authorizes it through its own attestation in the envelope's context before anything leaves the process, consumes the nonce in an append-only replay register, forwards the body only to an upstream the operator configured, and returns the response with a signed receipt. Nothing but the receipt's hash and the consumed nonce is written. The evidence of the exchange is the pair of envelope and receipt, verifiable offline by a third party with no body.

Authorization at the gateway is directional. Since v9.333 the responding agency itself must attest the requester's key in the envelope's context; an attestation by any other agency on the instance authorizes nothing there. Before that fix the gateway accepted an attestation from

anyone on the instance, which was the transitive trust the federation layer refuses, reappearing one layer up. **IMPLEMENTED**

12.4 Time that learns nothing

The timestamp authority binds an arbitrary SHA3-256 digest to an instant under an authority's registered key. It accepts a digest, never content, and keeps no per-request record, because a timestamp authority that logs every request is a store of who timestamped what and when. The evidence is the signed statement in the requester's hands. Since v9.341 a caller may choose to anchor: the timestamp's own hash joins an append-only, witnessed log and the inclusion evidence comes back stapled, so that a timestamp backdated under a stolen authority key is one absent from every witnessed head of its claimed era. What the authority then retains is one digest and one instant, a record of volume and timing, never of content, and the default request still leaves nothing. **IMPLEMENTED**

12.5 Signing a document so that it outlives the key

A holder in Polaris may hold a key (Section 17), and a holder need not, so the document-signing model stays notarial: the issuing authority signs on behalf of a holder who has just proved possession, exactly as it asserts status for one, and records who by hash. The authority's signature says that a holder of this credential, active at this instant, asked it to sign this digest, and the embedded evidence lets anyone check that claim later. **IMPLEMENTED**

Long-term validation is where the outside review and the repository's own drills sharpened the claim. An authentic timestamp is not a trusted one, since anyone can sign one, and a signer's own timestamp is backdatable by whoever holds the key. So the strong claim requires the timestamp to be trusted by the verifier, from a key distinct from the signer's, active at the instant per the trust list, and, if the verifier asks, anchored in a witnessed log or corroborated by a quorum of independent authorities. A verifier given no timestamp anchors reports the facts and claims nothing. Anchor verification is in the detached verifier and both SDKs, so an implementation that imports no Polaris code can decide long-term validity. **IMPLEMENTED**



Figure 12: A signed document and its long-term evidence. The document never leaves its holder; its digest is signed by an institution, or by the holder's issuing authority on the holder's behalf after a possession proof, recorded by credential hash. Evidence fixed at the instant of signing, a timestamp from a distinct key, the signer's manifest, checkpoint and feed, together with the verifier's own trust list, lets a verifier decide validity at that instant long after the signing key has been retired. A signature whose instant falls after a compromise is refused even though its embedded manifest says the key was active.

Layer card: institutions

What it is	The signed registry, the exchange gateway, the exchange receipt and its log, the timestamp authority with optional anchoring, document signing with long-term validation, the trust list.
Why it exists	Real software ecosystems need discovery, mediated requests, signatures and time. Each of those is conventionally built on a log of content, and the vocation forbids that log.
What it trusts	Registered institutional keys; the in-context trust graph; the verifier's own timestamp anchors and trust list.
What it refuses	A body at the mint endpoint; a URL named by a request; a replayed nonce; an attestation not made by the responder itself; a timestamp from the signer's own key as the strong evidence; a placeholder signature as authentication.
What it can see	Digests, keys, contexts, instants.
What it cannot see	The exchanged messages, the timestamped content, the signed document: none are ever given to it.
If it fails	A forged receipt fails the responder's signature; a dishonest responder's receipt is contradicted by the missing envelope; a stolen timestamp key is caught by the witnessed anchor or the quorum; a compromised signer key invalidates only what it signed after the instant.
Checked by	<code>check_registry</code> , <code>check_exchange_gateway</code> , <code>check_exchange_receipt</code> , <code>check_timestamp_authority</code> , <code>check_document_signing</code> , <code>check_ltv_timestamp_trust</code> , <code>check_broker_policy_bound</code> .
Now or later	Now, between two instances. Real institutions are later.
Inside and outside	Inside: transparency, which watches the receipt and timestamp logs. Outside: the operations that keep all of it running.

Every layer so far is a claim about correctness. None of it matters if the system falls over under load, loses a database, or cannot be deployed by anyone but its author. The next layer is survival.

13 Surviving reality: the stack, failure, load, the standing witness, and the day a pilot ends

13.1 The stack

The production topology is five services: a self-built TLS edge that negotiates a hybrid post-quantum key exchange with modern clients, the application under gunicorn, a connection pooler, PostgreSQL with continuous archiving, and Redis for the shared rate limiter. Every service runs as a non-root user with every capability dropped, third-party images are pinned by digest, and every production container drops the test framework. Four deployment paths share one secrets discipline, one backup path and one threat model: a laptop launcher, a single-host compose profile with a blue-green rolling deploy, a scripted Linux install under systemd exercised on two distributions, and a Helm reference profile with default-deny network policies and the restricted pod security standard, booted in CI on a one-node cluster. **IMPLEMENTED**

13.2 Failure, drilled and measured

The repository's rule is to exercise rather than read: ten of ten defects found in one sweep were invisible to reading. Every operational claim below is a drill that runs in CI and commits a number.

High availability. The database runs under Patroni with a leader lease in a three-member etcd and role-based routing. Under a live write stream the drill loses the leader (the replica promoted after the 20-second lease, in-flight queries failing fast and retried, none lost), cuts the leader off from the lease store (it stood down in 7 seconds), performs a planned switchover (3.3 seconds, no insert failed), and crashes an etcd member (a 0.3-second stall).

The Kubernetes profile runs the same members with the cluster’s API as the lease store and deletes the leader pod. **IMPLEMENTED**

A second region. The HA profile survives a node dying; it does not survive the region, and the tempting fix, a third member in region B, is wrong three ways at once, because it puts the wide-area network inside the lease store’s quorum, on every commit, and inside region A’s blast radius. So since v9.359 region B is a standby cluster: its own lease store, a different scope, streaming asynchronously through region A’s router so a failover inside A does not break replication, electing no primary of its own so two regions can never both accept writes. The recovery point is therefore not zero, and it is measured rather than asserted: the drill cuts region A the way a region goes dark, promotes B under a live write stream, and reports the time to serve (5 seconds) and the rows acknowledged in A and absent from B (zero at the drill’s rate), against ceilings of 90 seconds and 50 rows. It asserts two things that matter more than either number: B holds no row A never acknowledged, and what crossed is a contiguous prefix, because a standby holding rows one to forty and forty-five to sixty has skipped rather than lagged. A returned region A is rebuilt as a standby and never brought back as a primary. **IMPLEMENTED**

Read replicas. Read-only surfaces route to a streaming replica under an explicit staleness contract, at most ten seconds behind by default, falling back to the primary beyond it, with every routed response naming its source and lag. Correctness-critical reads stay on the primary, and the authorization half of a verification verdict is pinned there (Section 7). **IMPLEMENTED**

Partitioning. The four append-only event tables are monthly range-partitioned, and a drill proves that the append-only trigger holds across a partition, an attach and a detach, because the archives’ carve-out must not become a rewrite path. **IMPLEMENTED**

Disaster recovery. Backups are encrypted and a restore fails closed without the key. A monthly drill kills the primary, restores it from the archive and commits the measured numbers against the targets of 300 seconds of data loss and four hours to recover: the last recorded rows show 36 to 42 seconds of loss and under five seconds to service, on the drill’s ephemeral host, not production hardware. **IMPLEMENTED**

Chaos. A weekly drill boots the blue-green stack under traffic, kills one colour with zero dropped requests, stops both until the outage page reaches a real alert manager and webhook, kills Redis and PostgreSQL, partitions the pooler, and commits every recovery time against a ceiling. Findings become checks. **IMPLEMENTED**

Rolling deploys. A deploy under traffic drops zero requests; edge and database recreation are window operations measured on every push against ceilings (0.3 seconds for the edge on a single host, 0.6 seconds for a database crash), and the readiness ledger carries the remaining edge half openly. **IMPLEMENTED**

13.3 Load: what was measured, what was projected, and what a number rests on

Three kinds of number appear in the repository, and the paper keeps them apart. *Measured* numbers come from a named machine, an Apple Silicon laptop with eight cores, and carry a version stamp. *Simulated* numbers come from the national simulation harness, a synthetic country driven through the real procedures at a stated scale factor. *Projected* numbers are arithmetic over measured ones and are labelled as such by the repository itself, under a named assumption. No production cluster has been measured.

Table 12: Performance numbers at the versions that measured them. Every row is single-node on the reference laptop unless marked as a simulation or a projection. The event-ingestion rate and the cryptographic verification rate differ by about 35 times, a gap the repository’s own history once hid (Section 15). Rows the repository has not re-measured since Version 2 carry their original stamp.

Quantity	Number	Kind	Source
ML-DSA-65 sign, one core	about 2,080 per second	measured	dyno, 1.0.0-rc.7
ML-DSA-65 verify, single witness, one core	about 7,950 per second	measured	dyno, 1.0.0-rc.7
ML-DSA-65 verify, two witnesses, one core	about 740 per second	measured	dyno, 1.0.0-rc.7
Membership proof, depth 14	about 25 ms to prove, 12 ms to verify, 76 KB	measured	dyno, 1.0.0-rc.7; the soundness ledger
Closing an epoch at depth 24, one million members	2.1 s, tens of megabytes (was 10.8 s and 2.9 GB for a thousand)	measured	epoch cadence, v9.357
Re-signing a population under ML-DSA-87, one runner	about 345 credentials per second, 91 percent of it signing	measured	quantum-event drill, v9.365
Issuance through the application	40 per second sustained, p95 28 ms	measured	baseline, v9.191
Verification event through the application	80 per second sustained, p95 18.5 ms	measured	baseline, v9.191
Verification latency on the two-member HA topology under failover	a p50 at about 2 requests per second; p95 and p99 withheld, the sample cannot carry them	measured	failover drill, v9.387
Enrolment with real signing, bulk pipeline	about 372 tokens per second	simulated, 1:10,000	national simulation, v9.257
Verification-event ingestion (audit writes, no signature check)	about 25,970 per second	simulated, 1:10,000	national simulation
Atlas aggregates, 8,010 to 80,010 events	every bounded aggregate grew slower than its data (0.72x to 0.89x of linear)	measured	benchmark, v9.402
Single-witness verification across an eight-core node	about 62,800 per second	projection, fan-out assumed	benchmark arithmetic

The roadmap’s planning targets (ten million persons for a state; 350 million, five thousand sustained and fifty thousand peak verifications per second nationally) were validated at v9.384, and both halves of the answer are recorded. Every throughput target clears by more than an order of magnitude: the peak needs about six and a half cores of single-witness verification, the sustained rate under one, and a surge of two hundred thousand enrolments a day is nine minutes of one signer. And the system could not have run for a week at the sustained rate, because the verification event’s identifier was a 32-bit sequence: two billion rows at five thousand a second is five days, twelve hours at the peak, and nothing is slow when a sequence is exhausted, every insert on the path simply fails. Five surrogate identifiers are 64-bit now, a check recomputes the arithmetic from the live schema on every push, and the rule the repository drew from it is that a capacity question answered in cores and seconds is the easy half. The model also refuses to call a core count *measured*: the per-core rate is measured, the multiplication has never been run, and since v9.387 every extrapolation carries its assumption by name. A separate cost model finds that verification throughput is not the cost driver at any realistic national scale, since a hundred million people verified twelve times a year is thirty-eight verifications per second on average, and states which of its inputs are facts about the code and which are guesses about a deployment. A

national workload has been simulated at a scale factor and driven through the real constraints; it has not been run. **EXPERIMENTAL**

13.4 Abuse, secrets, supply chain, and decisions that leave a record

Per-agency quotas on issuance, revocation and verification are opt-in rows enforced by a trigger on every write path, exact under concurrency by an advisory lock, refused loudly on every surface, and keyed by agency identifiers, never by a person; since v9.424 a quota is a decision with a reason that survives being changed rather than a setting that can be overwritten. Velocity alerts compare each agency to its own trailing week. The per-IP rate limiter shares one Redis bucket across workers. Operators authenticate with WebAuthn as a second factor after an enrolment deadline, sessions are registered server-side with per-role caps and origin checks, and a per-role network policy applies. Creating an operator account left no record that it had happened until v9.422, and six of the twenty-one audit event types the schema named had no writer anywhere in the tree; every one has a writer now, pinned by a check, and the first version of that check was wrong about three of them and says so. Secrets are file-mounted, sealed and rotated through one lifecycle, and a database password reaches the database before its file changes, or is not rotated at all: until 23 September 2026 the file changed first, and a live stack the script took for stopped was left presenting a password the database had never been told; the application refuses to boot in production on a default key, and the application log records no form body, cookie value or token value, which was a documented promise kept by discipline alone until a check began reading the logging calls. Every release ships bills of materials with signed provenance, and CVE scans of the dependency surface and of every self-built image gate the build. **IMPLEMENTED**

13.5 The standing witness, and whether it watches

Twenty CI jobs run on every push, and most carry a comment naming the past failure they exist to prevent. They run the product suite and the invariant layer; drive the Atlas console in a headless browser; build and boot the images; build the edge and prove the post-quantum handshake against a real certificate; page a duress alert through a real alert manager to a webhook; validate the tracing wire and the dashboards; kill and restore the primary; install the Linux profile on two distributions; sign in-token through a software PKCS#11 module with two witnesses; roll a deploy under traffic; evacuate a region; fail the HA profile's leader, lease store and etcd member under a live write stream; boot the Kubernetes profile and fail its database over; sign and verify with real ML-DSA-65; federate two instances over HTTP; install `polaris-verify` as a stranger would; type-check, test and conform the TypeScript SDK; scan dependencies and every image for vulnerabilities; and boot the full production stack end to end. **IMPLEMENTED**

Version 3 adds what Version 2 could not: the witness was asked whether it watches. Twenty-two drills were found printing their whole verdict paragraph from zero recorded cases, so a drill now refuses to report a guarantee it did not test, and a check holds every drill to counting its cases. A drill wired into a CI job that lacked the runtime it needed refused to report rather than calling an unverifiable tree clean, which was correct, and the job then failed on its own setup rather than on a finding, which is how a real finding gets waved through; every job now installs what it runs, and a check holds it to that. The local gate a maintainer runs before pushing ran four suites while CI ran eighteen, and shipped a broken change twice before CI said so; it runs what CI runs now, and it names the verification a change needs from the paths and route handlers that moved. **IMPLEMENTED**

13.6 The day a pilot ends, and the day the old credential stops being accepted

Two operations were written down as procedures that run rather than as paragraphs, because each is the promise an institution actually says yes on. A pilot's real promise is not that it will work but that it can be wound back, and that promise fails quietly: erasure becomes a paragraph in a consent form nobody executes. The wind-down (v9.376) revokes every pilot credential and replaces every name with a meaningless marker under the append-only erasure record, the consent language a participant is shown is generated from what the code does rather than written from what would be reassuring, and the whole procedure runs against a real database on every push. The pilot pack that an institution owes before it starts is closed with it. **IMPLEMENTED** for the procedure; **EXTERNAL VALIDATION REQUIRED** for a pilot, since none has run.

A new national credential arrives beside a driving licence and a passport, and for years it is the second thing somebody carries. The dangerous moment in that period is not the cutover but the sunset, the day the old credential stops being accepted, because that is the moment the new one becomes compulsory without anybody having decided to make it so. The coexistence plan (v9.381) treats the sunset as a decision with a floor under it rather than a phase that arrives, and its verdict refuses to be computed from what the issuer can see: whether relying parties accept the credential and whether an alternate path exists and is usable are attestations an operator makes, and the system records them rather than guessing. **IMPLEMENTED**

Two mappings sit beside these. The NIST SP 800-63 mapping states, row by row, what an assurance claim would rest on, every citation resolved and run by a drill so that a row cannot claim evidence that is not there; it is explicitly not a conformance claim and no assessment has been performed. And accessibility is enforced on every push for the third that automation covers, and named as not done for the rest, because an identity system a person cannot operate is one that excludes them from identity. **IMPLEMENTED** for the mechanisms; **EXTERNAL VALIDATION REQUIRED** for an assessment.

Layer card: operations

What it is	The five-service stack on four substrates; HA, a standby region, replicas, partitioning, backup and disaster recovery, chaos, rolling deploys; abuse controls, secrets, supply-chain gates; twenty CI jobs and two scheduled drills; the pilot wind-down and the coexistence plan.
Why it exists	A correct system that cannot survive a lost disk, a lost region, a bad deploy or a hostile client is not infrastructure, and one that cannot be wound back is not one an institution should start.
What it trusts	The drills' ephemeral hosts as a lower bound on production behaviour; the operator's placement of hosts and regions and the custody choice.
What it refuses	A claim without a drill; a number without a stamp; a core count called measured; a drill that reports a guarantee it did not test; a root-running container; an un-pinned image; a fixable critical CVE.
What it can see	Metrics, traces and alerts about the system. The correlation id on every request never touches the audit of record.
What it cannot see	A person: quotas, alerts, metrics and logs are keyed by agency, route and worker.
If it fails	A drill that misses its ceiling fails the build; a chaos or DR finding becomes a check; a region evacuation that invents or skips a row fails; a production incident is what the readiness ledger's nine operator decisions exist to prepare for.
Checked by	check_ha_automation, check_multi_region_dr, check_read_replica_routing, check_event_table_partitioning, check_dr_drill_scheduled, check_chaos_program, check_zero_downtime_deploy, check_capacity_model, check_abuse_controls, check_every_audit_event_has_a_writer, check_logs_exclude_pii, check_container_hardening, check_cve_scanning, check_drills_count_their_cases, check_coexistence_plan.
Now or later	Now, on drills. A hardware module, production hardware and a real pilot are later.
Inside and outside	Inside: everything above. Outside: the system's view of itself, which an operator needs without a view of people.

An operator running all of this needs to see what the system is doing and to understand where authority comes from, and must be able to do both without the system becoming a map of its people. The next layer is how Polaris observes and understands itself.

14 How Polaris observes and understands itself

Two cognitive layers exist, two are proposed, and one line separates all of them from the people the system serves. The design rule is stated in the repository in one sentence: make power legible, not people.



Figure 13: The cognitive layers. Atlas sees what the system is doing; Athena understands the structure of authority; Metis, proposed, would learn the behaviour of Polaris itself; Myrmex, proposed in this paper, would attack it from outside. The constitution constrains all of them. The red line is the person-prohibition: nothing above it models a natural person, and `check_athena_no_person` fails the build if a person-level column or traversal appears in the authority layer.

14.1 Atlas sees

The Atlas is the operator's investigation surface: an analytical console that opens on a bounded, non-geographic overview, a volume time-series with failures overlaid, top-K breakdowns by context, agency, disclosure and outcome, a two-dimensional pivot, a records grid, and a map kept as a tab for what a map is good at, one region's drill or one subject's journey. Every view is a bounded server-side aggregate, never the raw events: at most five thousand cluster summaries

for a viewport, however fine a grid the client asks for, at most 240 time buckets, at most fifty categories, keyset pagination so that a deep page stays cheap. A zero-knowledge verification is counted in volume and in the disclosure tallies and is never located, because it carries no token to attribute it by. The single-entity investigation pages are built to refuse cross-individual link analysis, and every read of an audit surface is itself recorded. A live-simulation mode is driven in a headless browser in CI and the console is asserted to actually stream. Since v9.402 the benchmark times every bounded aggregate at a tenth of the event stream and at all of it, minutes apart on the same hardware, and grades each against the row-count growth, because a timing at one scale says an aggregate was fast once and cannot tell a roll-up that tracks the rows from one going quadratic; every aggregate grows slower than its data, and the benchmark exits non-zero if one ever outruns it. **IMPLEMENTED**

14.2 Athena understands structure

Athena is a read-only semantic and provenance layer over the authority tables: agencies, algorithms, contexts, trust attestations and retention policies, as ten object views and eight edge views, with four functions that answer the governance questions the tree could otherwise answer only by hand. Why may this agency issue under this algorithm? What proof and disclosure policy bounds this context? If this algorithm is deprecated, who is affected? Which mechanism enforces this constitutional rule? The constitution itself is data: C1 to C10 and the vocation are rows, and a table maps each rule to the exact live mechanism that enforces it, a trigger, a partial unique index, a named check constraint, a check function or a procedure. A check fails the build if any named mechanism no longer exists, and a database test proves each is present in the running catalogue, which closes the drift between prose and code that the constraint lattice alone could not. The signed registry of Section 12 is a view over Athena, so the map of the institutions that an outsider fetches is the same map the operator inspects. **IMPLEMENTED**

Athena is safe to exist only because person-legibility is made structurally impossible, not merely discouraged. Five invariants gate it, each with an adversarial detection test: no Athena view, function or table references a natural-person table or column, and no column is a stable per-person surrogate; every function is read-only, so *the graph says revoke, therefore revoked* is impossible; every function bounds its output; every authority edge resolves to an existing authority table, and the current views exclude revoked authority, so Athena describes and never owns; and every rule maps to a mechanism that exists. The same ship that created Athena removed the earlier person-aggregating views. If the prohibition ever obstructs a feature, the repository's own assessment says that is the signal to stop, not to relax it. **IMPLEMENTED**

14.3 What must never be built here

No person, household or relationship object. No subject handle, opaque or otherwise, stable across contexts. No traversal from a zero-knowledge event to a subject. No population segmentation or scoring. No action: the layer reads and never writes. Those are the kill criteria of the design study that endorsed a constrained Athena, and they are the boundary any future cognitive layer inherits.

14.4 Metis and Themis, named carefully

The roadmap carries a captured brief for a future, advisory-only learning subsystem that would learn Polaris itself: infrastructure anomaly detection, capacity forecasting with uncertainty, regression and release-risk intelligence, chaos learning that proposes experiments and never injects them, root-cause assistance over Athena's dependency graph, and aggregate institutional anomalies of the form *this agency normally revokes 0.8 percent a month and now revokes 4.9*. Its central hypothesis is to be falsified before any code: machine learning may infer properties of the system

and must never infer, rank, score or predict the rights of natural persons, and renaming a person to a subject does not satisfy that if the data stays linkable. Its design law is advisory only: observe, model, predict, explain, recommend, and never predict then revoke, deny, issue or change a constitution or a production configuration. The brief proposes the name Metis, because the obvious alternative collides with the metrics stack, and records that a rule that works as well as a model should be preferred to the model. Nothing of it is built; under the operating contract of Section 18 nothing of it will be until an outside party needs it. **PROPOSED**

This paper uses the name Themis for the constitution’s constraining role over all four layers, and the repository’s own brief uses it in the same sentence: Atlas sees, Athena understands structure, Metis learns system behaviour, Themis constrains them all. Themis is not a component. It is the vocation and C1 to C10 as enforced in the schema and pinned by the checks, which already exist. The name is a reading aid and nothing in the code carries it.

Layer card: system cognition

What it is	Atlas, the bounded operational console; Athena, the read-only authority-and-constitution layer; and two proposals, Metis and Myrmex.
Why it exists	An operator must see what the system is doing and understand where authority comes from; a system this large cannot be governed by hand.
What it trusts	The append-only events and the authority tables it reads.
What it refuses	Any person-level object, handle, traversal, score or action; an unbounded read; an aggregate that grows faster than its data; a model where a rule would do.
What it can see	Aggregates of events and the structure of authority, keys, contexts, trust and rules.
What it cannot see	A person. The single-entity investigation page exists outside these layers, gated by role and audited on every read.
If it fails	A person reference anywhere in Athena’s SQL fails <code>check_athena_no_person</code> and stops the build, measured by adding one (the other Athena checks guard other properties and stay green); an unbounded aggregate fails C8; a learning system that scored people would violate the vocation and is refused before it is designed.
Checked by	<code>check_athena_no_person</code> , <code>check_athena_read_only</code> , <code>check_athena_functions_bounded</code> , <code>check_athena_non_sovereign</code> , <code>check_athena_rule_enforcement_resolves</code> , <code>check_atlas_console</code> , <code>check_benchmark_measures_growth</code> , the C8 caps.
Now or later	Atlas and Athena now. Metis after an assessment, and only if an outside party needs it; Myrmex is a proposal in this paper.
Inside and outside	Inside: operations. Outside: the loop by which the system corrects its own understanding.

A system that can observe itself still has to learn from what it observes, and the hardest lesson Polaris has learned is that its own instruments can be green while measuring the wrong thing. The next section is that lesson, and this version has a great deal more of it to tell.

15 Errors become new senses: the history of self-correction

15.1 Formal truth is not intended truth

A check can be perfectly green while proving the wrong proposition. A signature can exist without ever being verified. A benchmark can report thousands of verifications per second while counting database writes. A two-witness guarantee can be certified on the strength of one witness that happened to be absent. Seventy-one of seventy-five checks can pass on a tree where the property each names has been commented out. The suite that proves one identity per person can be the thing violating it. Every one of those sentences describes a state Polaris was actually in, at a version the changelog names, and every one was found by running the system rather than reading it, or by an outside reader who did not share the author’s assumptions. The project’s method for this is a loop: an observation, a failure, a correction of the model of what the mechanism really proves, a new invariant with a detection test, and a permanent new sense that the failure class

cannot recur unseen. Invariants accumulate and are never deleted. At 1.0.0-rc.7 there are 313 of them, and twelve drills whose only job is to attack the layer they belong to.

15.2
Selected corrections, from the record, up to Version 2

Each row is a mechanism that was technically valid and did not prove what was intended, the version that found it, and the sense it left behind. None of these is embarrassing; each is the method working. Version 2 carried this table, and it is kept.



Figure 14: The loop by which an error becomes a permanent sense. Observation is running the system, not reading it; the failure is a green check that proved the wrong thing; the correction is a new statement of what the mechanism proves; the invariant is a check with a detection test; and continuous integration carries it on every push from then on.

Table 15: Corrections recorded in the changelog before Version 2. The right-hand column names the check or test that now fails if the original condition returns.

Version	What was believed	What was true, and what changed	The new sense
v9.58	Tokens were post-quantum signed	The procedure wrote a placeholder string and the signing module was an unused island; signing was wired into issuance	check_pqc_signing_wired

continued on the next page

Version	What was believed	What was true, and what changed	The new sense
v9.113	Signatures were verified	Nothing on a live path ever called verify; issuance now self-verifies and a use-path primitive exists	<code>check_verify_enforced</code>
v9.133	A verdict from one library was a verdict	One library could silently accept a forgery; a second, independent implementation must agree	<code>check_pqc_second_witness</code>
v9.257	The benchmark measured verifications per second; bulk-issued tokens were signed	It measured audit-row ingestion, about 35 times faster than cryptographic verification; bulk rows carried a placeholder literal. Three numbers are now reported separately and bulk issuance refuses an unsigned row	<code>signatures_cryptographically_verify</code> , <code>check_national_simulation</code>
v9.264	Every stored signature was two-witnessed; <code>usable</code> was current	A missing witness at issuance degraded silently to one; <code>usable</code> could be decided on a stale replica. Issuance now requires the witness and status is pinned to the primary	<code>check_pqc_signing_wired</code> (extended)
v9.272	The single-witness fast path was sound	It was sound only while the fast witness stayed trustworthy, which nothing checked; sampling through the second witness is now mandatory and a disagreement pages	<code>check_verify_witness_sampling</code>
v9.302	A transparency monitor caught a lying log	It caught a rewrite and not a split view; witnesses, gossip and the equivocation proof were added	<code>check_transparency_gossip</code>
v9.309	The detached verifier rejected hostile input	It crashed on several classes of it; a totality contract converts every crash into the intended reject	<code>check_verifier_fuzz</code>
v9.313	The canonical oracle pinned every signed field list	Its static key-list pin used a regex that silently matched nothing; the wire specification is now checked against the signer	<code>check_wire_spec_matches_code</code>
v9.333	The exchange gateway authorized by the trust graph	It accepted an attestation from any agency on the instance, transitive trust one layer up; the responder itself must attest	<code>check_exchange_trust_directional</code>
v9.334	Long-term validation trusted a timestamp	It trusted any authentic timestamp, including the signer's own, which the signer can backdate; the verifier now names distinct trusted timestamp authorities	<code>check_ltv_timestamp_trust</code>
v9.335	A receipt alone proved an exchange	A dishonest responder can sign any receipt; the requester's envelope beside it is the evidence. A QR decoder with no bound was also made resource-bounded	<code>check_qr_resource_bounds</code>
v9.336	The broker enforced a relying party's step-up	The demand arrived only in the holder's request; it is now registered on the relying party's row, and the code is encrypted	<code>check_broker_policy_bound</code>

continued on the next page

Version	What was believed	What was true, and what changed	The new sense
v9.341	Anchored time evidence survived a stolen key	A thief could mint a backdated timestamp; anchoring into a witnessed log makes a forgery one absent from every witnessed head	<code>check_timestamp_transparency</code>
v9.345	The maintainer would remember which drills a change touches	A changed verdict shipped without its drills and CI was red for four versions; a tool now names the verification a change needs from the paths and route handlers that moved	<code>check_ship_tool</code>

15.3 Since Version 2: the instruments were asked whether they would notice

Between v9.357 and 1.0.0-rc.1 the loop was turned on the verification layer itself. The question changed from *does it pass* to *would it notice*, and the answers below are the record. Most of these rows are about the tests rather than the system, which is the point: a guarantee is believed because a check says so, and the checks are written by the same hand as the code.

Table 16: Corrections recorded since Version 2. The right-hand column names the check, drill or record that now fails if the original condition returns.

Version	What was believed	What was true, and what changed	The new sense
v9.384	The national throughput targets were the capacity question	They clear by an order of magnitude, and the system could not have run for a week at them: a 32-bit event identifier is five days at the sustained rate. Five identifiers widened	<code>check_capacity_model</code>
v9.387	Six cores would serve the peak	The per-core rate was measured; the multiplication had never been run. Core counts are extrapolated under a named assumption, and a percentile the sample cannot carry is withheld	<code>check_capacity_model</code> (extended)
v9.389 to v9.393	Thirty-five conformance cases passed; 24 of 44 frozen cases held	With no post-quantum library, every pass was a rejection answered by a verifier that rejects everything; a third tool closed with a guarantee it had skipped. All three refuse to report rather than name a cause	<code>check_conformance_suite</code> (executes the runner), the compat suite's void verdict
v9.398, v9.399	The checks read the code	71 of 75 passed on a tree where their property was commented out, because the helper handed them the comments too. Stripped; the exposure is measured on every push	the check-mutation drill
v9.403	A drill that printed its guarantee had tested it	Twenty-two drills printed whole verdict paragraphs from zero recorded cases	<code>check_drills_count_their_cases</code>

continued on the next page

Version	What was believed	What was true, and what changed	The new sense
v9.404	Both internal hops were classical	The posture had been read off base-image versions; one hop had been post-quantum for some time. Read off a real handshake now	<code>check_internal_kex_measured</code>
v9.405	A detection test proved its check detects	Fifteen tests only ever asserted failure; a control on the unbroken fixture is now required	<code>check_detection_tests_have_a_positive_control</code>
v9.407 to v9.415	The database's guarantees were tested	All 46 CHECK constraints were load-bearing; 14 of 37 triggers and 12 of 17 unique indexes could be dropped with the suite green, and with the one-active-token index absent the suite committed a duplicate. All covered; the sweep is a drill	the three mutation drills, <code>check_append_only_tables_are_tested_exhaustively</code> , <code>check_unique_rules_are_tested_exhaustively</code>
v9.417, v9.418	The access-control matrix described the routes	Two hand-written lists named 25 of 101 guarded routes; stripping every guard turned 15 of 654 tests red. The matrix is derived from the code, and a CSRF exemption is read rather than assumed	<code>check_route_guards_are_derived_not_listed</code> , <code>check_csrf_exemptions_do_not_trust_the_session</code>
v9.420 to v9.433	The conformance contract asked what a verifier must check	It verified an identity token's signature and never its audience or nonce; three artifact types had no bad-signature case; five valid vectors had expired; the two SDKs disagreed about replay; the instrument counting all this was wrong three ways	<code>check_conformance_asks_the_relying_party_question</code> , the conformance mutation drill
v9.422 to v9.426	Decisions about operators, quotas, relying parties and ceilings were recorded	Six audit event types had no writer; a quota, a relying party's bar and the revocation ceiling could be changed with nothing recorded anywhere. Each is a trigger-written record now, and the first list of missing writers was wrong about three	<code>check_every_audit_event_has_a_writer</code> , the audit-of-record checks
v9.434 to v9.437	The stored procedures' refusals were tested	27 of 59 could be deleted with the suite green, concentrated where a trigger cannot see; all covered	<code>check_procedure_refusals_are_mutation_tested</code>
v9.440, v9.443	The root of the authority hierarchy and the table of who may sign in were recorded	Neither had a trigger; two operator scripts naming columns that never existed had never run in 410 versions. Found against the live catalogue, not by reading	the schema-drift drill, the audit-of-record checks
v9.452 to v9.454	Polaris was post-quantum	Accurate about the algorithm, a claim about Module-LWE as security, and the default path signed a placeholder. The headline is algorithm agility, and a rule rather than prose holds it there	<code>check_post_quantum_claims_are_agility</code>

continued on the next page

Version	What was believed	What was true, and what changed	The new sense
v9.455, v9.456	The reference verifiers refused what they exist to refuse	18 of 18 refusals in the Python SDK and 9 of 14 in the TypeScript one could be inverted with every suite green; all 32 are inverted on every push	<code>check_sdk_refusals_are_mutation_tested</code>
v9.459 to v9.462	Five concurrency tests proved serialization	None could fail, one claim lived in a comment, and the local runner read half the file; the tests now carry a contention control	<code>check_advisory_locks_have_a_contention_test</code>
v9.465	The verifier's certificates were right	Eleven hosted modules and every internal instrument had passed a request-signing certificate with no key usage; an outside wallet refused it	a test red on a leaf without <code>digitalSignature</code> ; the scoreboard
2026-09-15	Eleven of eleven hosted modules were clean, awaiting a screenshot	One alias for all eleven made each module interrupt its predecessor; four positives were killed, and the suite had recorded it in the log downloaded to prove the claim. Re-run one at a time: 7 PASSED, 4 REVIEW	the drill's unique alias per test; the scoreboard's status vocabulary
1.0.0-rc.1	The sample database reloaded clean	Its reload restarted a relying party's identifiers and reached the party's append-only record with neither, so each reload's first party inherited a dead party's history; a version-stamp check had guarded its comparison so that an unparseable version passed	<code>check_seed_restart_resets_dependent_records</code> , <code>check_roadmap_consistent</code>
2026-09-23	Every refusal a state-changing use case makes was mutation-tested	The drill selected stored procedures, and three use cases are functions because they return the id they create; their eleven refusals were measured by nothing, and several could be deleted with the suite green. Nine are caught now, one by a test that forces the lock interleaving it exists for; two are recorded	<code>check_procedure_refusals_are_mutation_tested</code> (extended to functions)
2026-09-23	A check fails the build if the token state machine's trigger stops being	Deleting each of the 42 triggers in turn left every check green for seven, the state machine and the active-signature guard among them; one check read a trigger's name and was satisfied by the comment beside it. Both core triggers are now pinned, with their legal transitions; four remain the drill's alone	<code>check_token_core_triggers_are_bound</code> , the trigger mutation drill

continued on the next page

Version	What was believed	What was true, and what changed	The new sense
2026-09-23	The checks behind the constitution would notice their mechanism's loss	Each constraint's mechanism was broken in turn on the real tree; seven of ten checks let a break through. C1 took the append-only code anywhere in the schema, comments included; C3's drift checks took a deleted index by its COMMENT ON; C5 took <code>script-src 'self' https;</code> C6 read the centroid's latitude and not its longitude; C7 missed a hardcoded copy beside the table; C8 missed a query whose row cap was removed; C10 saw money only as six words in table names, so a <code>stored_value</code> column on the credential passed. Each is closed and shown red; C2, C4 and C9 held	<code>check_aor_append_only_triggers</code> , <code>check_c1c10_objects_resolve</code> , <code>check_csp_forbids_unsafe_inline</code> , <code>check_c6_atlas_redacts_zk_location</code> , <code>check_crypto_algorithm_is_data</code> , <code>check_c8_atlas_caps</code> , <code>check_c10_no_money_tables</code>

Several corrections in the record are about the project's posture rather than a mechanism. At v9.55 the repository deleted eighteen thousand lines of self-monitoring apparatus and some nine hundred invariants that asserted the apparatus's claims about itself; at v9.453, three hundred and ninety-seven versions later, forty-six references in twenty-four live files still cited the deleted documents as the authority for a rule, one of them in an error message an operator reads. At v9.332 the phrase *zero-knowledge identity system* was split into three claims. At v9.452 *post-quantum* on the front door became *algorithm agility under an audited migration path*, and at the lab's assessment of 13 September 2026 *compulsion-resistant* became *duress-aware*, each with a check that refuses the stronger word. And at 1.0.0-rc.1 the version number stopped counting ships: 9.467 had been bumped by the change that added a check and the change that fixed a comment alike, and under the operating contract a version marks an externally observable change, which is rare.

15.4 Multiple observers of one claim, and the first from outside

The pattern behind the tables is that Polaris increasingly refuses to let one observer decide whether a claim holds. A cryptographic verdict has two implementations and one published corpus. A check has a detection test that proves it can see its own violation, with a control. A drill runs the path instead of reading it, and counts its cases. An attack tries to break the defence with the real code. A fuzzer explores the input space the drills hand-picked. Twelve mutation drills ask whether the tests would notice their subject going missing. An outside review at v9.331 found six things the author's own instruments had not, and five became invariants within eight versions. Continuous integration carries all of it on every push. None of these observers shares the others' assumptions, and that is the point: a green test is not proof that the test asked the right question, but many differently-built observers agreeing is much harder to fool than one.

Version 2 proposed a seventh observer that would live outside the repository altogether, under the name Myrmex, and it remains a proposal (Section 18). What arrived instead was plainer: a wallet somebody else wrote, and a conformance service somebody else runs. The wallet refused a certificate that eleven conformance modules, 142 package tests, a mutation drill over every refusal and every check in the invariant layer had passed over. That is the whole argument for the operating contract in one event. Every observer inside the repository inherits the author's model

of what a request-signing certificate has to contain; the first observer that did not inherit it found the gap in seconds.

16 What Polaris refuses to become

Powerful identity infrastructure carries risks that no amount of correctness removes, because they are risks of what a correct system can be used for. The repository names these as design and governance constraints rather than as politics, and this section states them the same way. Each is a way a working Polaris could be turned into something its constitution forbids, and two of them are stated more narrowly than Version 2 stated them, because the evidence required it.

Identity must not become money. A credential that carries spending authority inherits programmability, and every constraint anyone wants placed on spending accretes onto the identity token, until the system can be told whom to refuse at a fuel pump. C10 forbids a monetary claim in the schema; if a value system is ever built, it lives in a separate database that consumes verification proofs over a boundary, and the boundary is what is load-bearing. This is the single most consequential architectural decision in the repository, and it is the one that never expires with any phase.

Privacy is not merely secrecy. A system can know very little about a person and still be coercive, if every doorway requires its authorization. The zero-knowledge default bounds what is recorded; it does not bound what a society demands. The tiered-enrolment layer exists for this: exemption is a first-class, one-row state, the not-enrolled cannot be enumerated by an accident of schema, and a person with no documents can be enrolled through a referee whose vouching leaves no mark on the credential. The coexistence plan names the sunset of the old credential as the moment the new one becomes compulsory, and refuses to compute that verdict from what the issuer can see. The repository's own threat model names the strongest attack on this layer as making non-enrolment civically impossible from outside, no token, no healthcare, and says plainly that the schema cannot stop it, only make it a named act.

Anti-coercion is a vocation, not a claim. The vocation above the ten constraints says no person may be compelled to surrender their identity. The mechanism that serves it, the duress code, resists the coercer it was built for and not the two others the operating contract named, and against lawful or institutional access its indelible record is evidence against the holder (Section 3). So the front door says *duress-aware* and a check refuses *compulsion-resistant*. The boundary is the honest one: the system notices duress; it does not resist compulsion, and a vocation that pointed at the adversary the mechanism is weakest against would have been the opposite of what the word promised.

Technical federation is not social decentralisation. Polaris is federated in code: each authority its own key, no central store, no transitive trust, and since v9.364 an operator bound to one authority even inside a shared instance. Two instances run by one author on one machine demonstrate the protocol and nothing about who runs it. Whether authorities, witnesses and relying parties are in fact independent is a property of a deployment, not of the repository, and this paper marks every such claim as requiring external validation.

Revocation must not become civil erasure. A revoked token is a terminal state in an append-only record, not a deletion; lost tokens stay lost, and their data is not erased. Succession is by reference. Mass revocation is bounded by a ceiling and a co-signer, and the ceiling itself can only be changed with a reason on record. Erasure, when a jurisdiction requires it, is pseudonymisation with an append-only record of who, when and why, never of the prior

name; a pilot's wind-down is that erasure, executed rather than promised. An authority can end a person's credential; the schema refuses to let it end the person's history.

Relying-party correlation is different from issuer unlinkability. The issuer's database cannot rebuild the zero-knowledge verification graph. Two relying parties can join their logs by whatever stable material a presentation showed them. Both sentences are true at once, and Version 3 can say how much of the second was measured: on the native zero-knowledge path a colluding pair has no advantage beyond the anonymity set against an adversary that matches on equality, and nothing floors that set above one; on the interoperable path a single presentation leaves nine issuer-signed values identical at every verifier (Section 8). A reader who hears *unlinkable* and imagines either path is stronger than that has been misled, which is why the repository states the boundary positively and the verifier reports which kind of presentation it was shown.

Interoperating imports the ecosystem's properties. Speaking the format an outside wallet speaks was the first thing anyone outside used, and it carried the format's linkability with it. The repository records that as a cost of interoperating, not a reason to stop, and it does not let the interoperable path borrow the native path's privacy claims.

No population graph. The Atlas is bounded and never locates a zero-knowledge event; Athena is forbidden a person node, a stable subject surrogate and a subject traversal; the schema carries no attribute to filter a population by; quotas and alerts are keyed by agency, never by person; the transparency report carries no total a small count could be recovered from. The refusal is structural because a population graph, once built, outlives the intentions of whoever built it.

Machine learning must not turn identity infrastructure into scoring. The proposed learning layer may learn how the identity system behaves and must never learn how people behave. Fraud or risk scores for individuals, behavioural profiles, relationship inference, eligibility or likelihood-of-revocation scoring, person-level clustering or embeddings, and model-driven denial of rights are presumptively prohibited, and a recommendation is evidence, never authority.

The repository also lists permanent non-goals that do not expire with any roadmap phase: no payment rails, no central biometric database, no population-scale attribute filtering or analytics, no social scoring, no commercial or advertising use of verification data, and no real personal data before a consent framework exists. Every one of them is a way the system could be made more useful to someone, and every one is refused on sight. Since the operating contract, one more refusal governs the repository itself: no new capability is built because it would be elegant, complete or desirable; a product change needs a named outside party that requires it, or an executable counterexample against something already promised.

IN THE TOWN The laws above the town bind the town. The hall may not mint coin. The register may not be turned into a list of who is absent. The gates may not be closed on a neighbour on the word of a neighbour's neighbour. A name struck from the register stays legible as struck. No clerk, however clever, may be set to guessing which townspeople are likely to cause trouble. And the town does not claim that its walls keep a person safe from a guard who is standing beside them.

17 AI agents, proof of personhood and the future internet

Version 2 first wrote this section as a statement of what did not exist, then rewrote it when the holder-side work was built. Version 3 changes little in it, because little changed: the four mechanisms still run, their bounds are the same, and what this version adds is what the first outside path taught about the residue, and which of the refusals below were found, by mutation, to be refusals in name only until they were fixed.

17.1 The problem that is arriving

When software agents can generate unlimited accounts, content, transactions, messages and apparent participants, the cheap signal that a request came from a person stops being available for free. Many digital systems will need to distinguish, without learning more than they must, between one accountable human; one authorized agent acting for that human or for an institution; anonymous machine activity; duplicated or synthetic identities; and organisational agents acting under an institution's authority. Every naive way of obtaining that signal leaks identity: accounts, phone numbers, payment instruments, biometrics. The identity question changes shape. It is no longer only *who are you*, but *what kind of actor are you, whose authority are you exercising, what are you allowed to do, and how can I verify all of that without learning anything else*. That is the future-facing question this paper considers the most important one Polaris's core could be extended to answer.

17.2 What exists today that points this way

Six ingredients of a privacy-preserving proof of personhood run.

- **Uniqueness at issuance.** C3, one active token per person, is a partial unique index in the database, and since v9.374 a model checked on every push. That is the root of Sybil resistance: a human can hold one active credential, and no application code can bypass the constraint. **IMPLEMENTED**
- **Membership without identity.** The epoch membership proof proves that a valid token exists in an authority's committed set with public inputs of only the root, the epoch, a context, a nonce and a scoped nullifier; a zero-knowledge verification records no token at all; the anonymity set is a full epoch, and, measured, nothing floors it above one. **IMPLEMENTED**
- **Revocation without naming.** A revoked token drops out of the next epoch root, so a membership proof expires with the epoch and no relying party learns who was revoked. **IMPLEMENTED**
- **A key the holder controls.** A credential may carry a holder public key the issuer binds to it, so possession of the file stops being possession of the credential and a verifier can require a signature from the person rather than a copy of their credential. **IMPLEMENTED**
- **Proving on the holder's device.** The epoch's leaf set is published, signed and bounded; every requester receives identical bytes; the holder finds their own leaf and builds the path locally. The issuer is never told which member is proving. **IMPLEMENTED**
- **A per-verifier nullifier.** The proof carries `Poseidon(secret || scope || epoch)`, constant for one person within one relying party's scope and epoch and uncorrelated across scopes. **IMPLEMENTED**

17.3 One human, once: what runs and what it costs

The membership proof stops replay of one proof, but on its own it does not stop the same person producing a thousand fresh proofs to one site and registering a thousand accounts. *One human, once, here* needs a scoped nullifier: one more public input, a hash of the holder's secret, the relying party's scope and the epoch, so a site can refuse a second proof from the same person while two sites cannot link their nullifiers. That is a public input of the circuit, and the claim that two relying parties pooling everything they hold still cannot link a holder is a formal model checked with a configuration it must fail. **IMPLEMENTED**

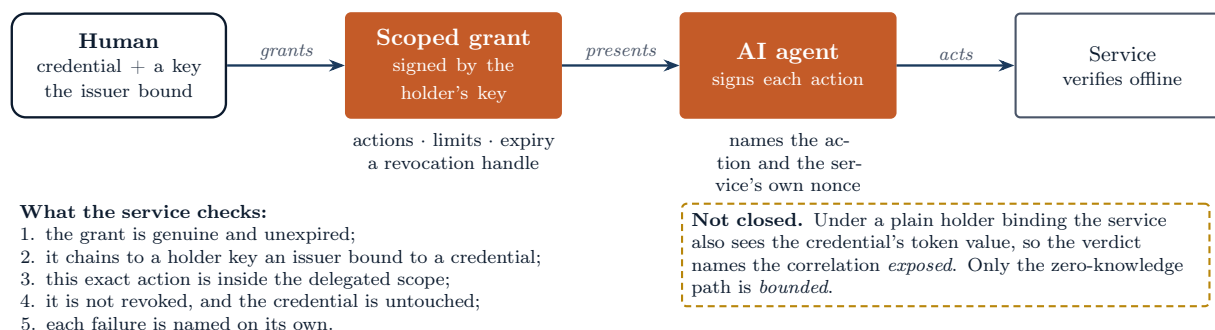
Adding it was not the small change Version 2 once predicted, and the reason is worth keeping because it is the kind of mistake a plan makes from outside the code. The epoch leaf was a SHA3-256 digest computed *outside* the circuit and handed in as an opaque private value. A nullifier beside an opaque leaf proves only that the prover knows some number: nothing ties the two to one secret, so a prover could pair any member's leaf with a nullifier of their own choosing, and every property above would have been a claim rather than a proof. Constraining both to one secret means opening the leaf inside the circuit, and verifying a SHA3-256 preimage there costs thousands of constraints where Poseidon costs about a hundred. So the leaf became `Poseidon(secret || context)`, a commitment the circuit opens, and the old digest became the holder's secret. Epochs closed under the old derivation are re-closed rather than migrated.

The two conditions the vocation imposes were carried into the implementation rather than left as intentions. The nullifier's scope is per relying party and per epoch, never global, because a global nullifier is a persistent identifier and therefore the surveillance outcome the constitution refuses; and it rotates each epoch, so membership never becomes a permanent pseudonym. A relying party's ledger is keyed by its own scope and that epoch, and should not outlive it.

Two bounds remain and are stated rather than closed. The nullifier does not hide the holder from the *issuer*, which derives every leaf in order to build the tree and could therefore compute any member's nullifier in any scope; the property holds between relying parties. And the presentation layer is still full-credential on every path but the zero-knowledge one: a verifier looking at a plain credential sees a stable token value, an issuer signature and a holder key, and a verifier on the interoperable path sees nine issuer-signed values that are the same everywhere. What the pairwise derivation bounds is what a verifier *stores*, which is the material that gets pooled, sold, subpoenaed and breached; what the zero-knowledge path bounds is what it is *shown*, and the lab measured that bound and its blind spots (Section 8). The verifier reports which of the two a given presentation gave, and since the lab's first finding it checks the premise before saying so. The blinded and one-time presentation technologies that would remove the residue are held, not scheduled, in Section 18; none is implemented. **PROPOSED**

17.4 Delegating authority to an agent

The relationship is a chain, and it runs. A human possesses a foundational credential and a key the issuer bound to it. With that key the human signs a grant giving an agent a narrowly scoped authority: named actions, stated limits, an expiry, and a revocation handle belonging to the grant alone. The agent signs each action it performs, naming the action and the service's own nonce. A service verifies, offline, that the grant is genuine and unexpired, that it covers this exact action, that it has not been revoked, and that it chains to a holder key an issuer bound to a real credential. **IMPLEMENTED**



Revoking the grant does not revoke the human. The holder signs the revocation with the same key; the issuer is never contacted and never learns the grant existed.

Figure 15: The delegation chain, as built. The human holds the foundational credential and a key the issuer bound to it. With that key they sign a grant naming actions, limits, an expiry and a revocation handle; the agent signs each action against the service's own nonce; the service verifies the chain offline. The one link drawn dashed is the residue: under a plain holder binding the service also sees the credential's token value, so the correlation it is left with is bounded rather than absent.

What the design is for is visible in what it refuses. Handing an agent the credential itself gives it everything the person can do, forever, revocable only by revoking the person. The grant is the opposite of each: **actions** and **limits** sit *inside* the signed statement, so a grant widened in transit fails rather than passing invisibly; an empty action list authorises nothing and there is no way to write *everything*; a limit key the service does not recognise is refused rather than ignored, since a grant that says `max_transfers: 3` to a service that has never heard of it must not read as unlimited. And the agent must sign: without that link a grant is a bearer token, and whoever copies it in transit becomes the agent. Two of these refusals, an exhausted use count and an amount over its ceiling, are the bound that makes delegation something other than a bearer token, and at v9.455 both were found to be invertible in the reference verifier with every suite green; they are covered now, and the conformance contract, which had published no case in which a grant's or a revocation's signature was bad, has those cases. **IMPLEMENTED**

Revocation is the property worth naming twice. The revocation is signed by the same key that signed the grant, so anyone may publish bytes but only the holder may end it. The issuer is not contacted and never learns the grant existed, and the human's credential is untouched throughout. A person can end their agent's authority without asking permission from, or being observed by, the authority that issued their identity. The revocation carries the grant identifier and the instant, and nothing else. It will not carry a reason: a field recording *why* a grant ended is a field a coercer can demand be filled in or left empty, and either way it turns a revocation into a signal about the person.

What a service learns is bounded, and the bound is reported rather than assumed. Under a plain holder binding the service also sees the credential's token value, so the verdict names the correlation **exposed**; only a chain carried by the zero-knowledge path, whose handle is the scoped nullifier and whose transcript shows nothing identical across verifiers, is named **bounded**. The grant hides the human from the agent's *actions*. It does not hide the credential from the service.

17.5 The future internet, concretely

The term Web3 is used here in one narrow sense, and it has nothing to do with currency: an internet in which people, autonomous agents, institutions, wallets, distributed services and independently operated networks need portable cryptographic trust that does not route through one company or one state. Against that environment the following table marks what Polaris could provide and what each item's status is today.

Table 17: What a future environment would ask of Polaris, and the status of each.

The environment asks for	What Polaris has	Status
Portable credentials	A signed file any standard library verifies; four packages on public registries	IMPLEMENTED
Independently verifiable proofs	The detached verifier, two SDKs, a conformance suite, and an OpenID4VP verifier an outside wallet has presented to	IMPLEMENTED EXTERNALLY EXERCISED EXTERNAL VALIDATION REQUIRED
Institutional federation	Per-authority roots, directional attestations, published feeds	IMPLEMENTED EXTERNAL VALIDATION REQUIRED
Service discovery	The signed registry	IMPLEMENTED
Document signatures with long-term validity	Signed containers with evidence fixed at the instant	IMPLEMENTED
Transparent authority behaviour	Three logs, witnesses, equivocation proofs, a public report of warrant reads	IMPLEMENTED EXTERNAL VALIDATION REQUIRED
Offline verification	Status assertions stapled to presentations; a card that signs a device's challenge	IMPLEMENTED
Post-quantum trust	ML-DSA under an audited migration path, hash-based proofs, a hybrid edge; classical remainders named; what a break would cost written down	IMPLEMENTED
Human, account and agent authorization relationships	A login token whose subject is per relying party, plus the agent grant	IMPLEMENTED
Privacy-preserving uniqueness	Membership proofs carrying a per-verifier, per-epoch nullifier; measured against a colluding pair	IMPLEMENTED
Delegation to agents with bounded scope	A holder-signed grant: actions, limits, expiry, revocation handle; its bounds mutation-tested	IMPLEMENTED
Revoking an agent without destroying the human's identity	A holder-signed revocation; the issuer is not contacted and the credential is untouched	IMPLEMENTED
Presentation that shows a verifier nothing stable	The zero-knowledge path only, within the anonymity set; a plain or interoperable presentation shows stable, issuer-signed values	IMPLEMENTED PROPOSED
Speaking the formats other wallets speak	OpenID4VP with SD-JWT VC verified; an ISO 18013-5 rendering whose issuer signature no standard reader can check; a Verifiable Credential of a verification result	IMPLEMENTED EXTERNALLY EXERCISED

18 Future research and engineering, under a rule that forbids most of it

Version 2's version of this section was a list of things to build. This one cannot be, because on 13 September 2026 the repository adopted an operating contract that changes what *future work* means. A product change now needs exactly one qualifying reason: a named external implementation requires it, a named non-author operator or holder requires it, a finding originating outside the repository requires it, or an executable counterexample shows behaviour the repository already promises to be false. Cleanliness, completeness, elegance and a desirable new guarantee do not qualify, and the contract says not to invent a fifth reason. Everything else goes to the lab, whose job is to falsify the differentiating claims, not to expand the architecture. So the items below are sorted by who would have to act, and everything is marked PROPOSED unless stated otherwise.

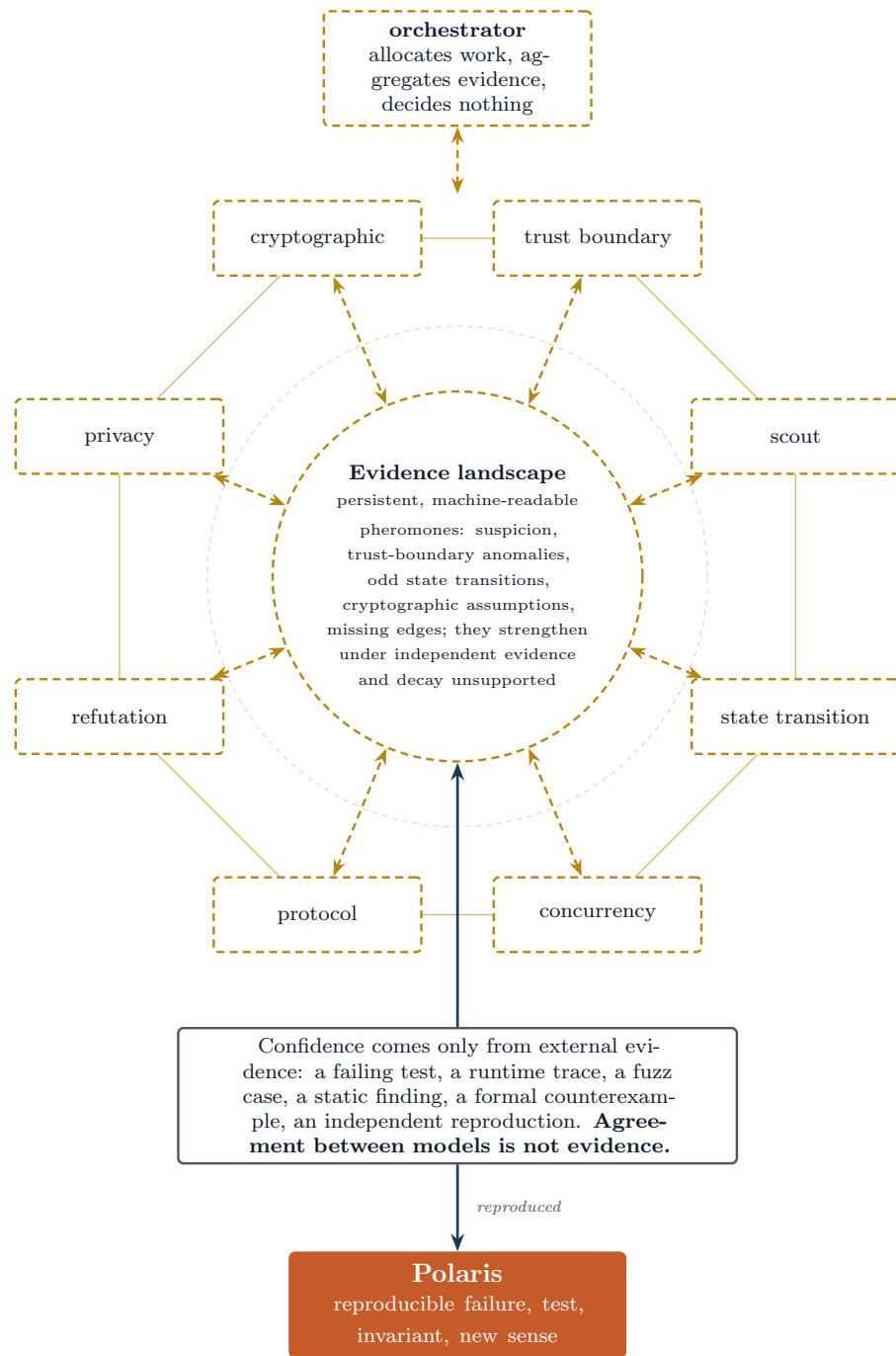


Figure 16: The proposed colony. Small specialised agents traverse the code and the architecture and leave machine-readable pheromones on a persistent evidence landscape: suspicion, a trust-boundary anomaly, an unusual state transition, a cryptographic assumption, a missing edge, an inconsistent claim, an area repeatedly associated with failure. Pheromones strengthen under independent evidence and decay unsupported; agents follow reinforced trails. An orchestrator allocates work and aggregates evidence and decides nothing. Confidence comes only from external evidence, never from agreement between models. What survives reproduction enters Polaris’s own loop as a test and an invariant.

18.1 Gates that are not the repository’s to pass

The contract was adopted on 13 September 2026, and it dates itself. Its ninety-day objective falls due on 12 December 2026 and names seven things; the scoreboard (1ab/EXTERNAL-NOUNS.md) says which hold. Read on 23 September 2026, day ten:

Table 18: The operating contract’s ninety-day objective, item by item, against the scoreboard on day ten.

The contract asks for	Holds	The evidence, or what is missing
polaris-verify independently installable	yes	on PyPI since 15 September 2026; the boundary drill installs it where no Polaris code exists
Real cryptographic behaviour explicit	yes	the verifier refuses to start without a declared crypto mode (Section 6)
A named external wallet completes a presentation	yes	walt.id Wallet API v2, 15 September 2026
An external OpenID conformance suite running	yes	the Foundation’s hosted suite, 15 September 2026: seven PASSED, four REVIEW
A relying party or operator who is not the author has used the verifier	no	none; this is also the one condition between the release candidate and 1.0.0
Repeated outside presentations	no	presentations outside CI: zero
External failures entering the development loop	yes	two fixes caused by outside findings (Section 6)

Five of seven held by day ten, and the two that do not both need a person who is not the author, which is the point of the contract: nothing left on the list can be done by writing more code. Past the objective lie the rows no deadline governs: a second wallet, and one on the native rather than the OpenID4VP path; a certification submission that would put the four modules in REVIEW in front of a Foundation reviewer; an independent security review, penetration test or cryptographic audit; a pilot with consenting users; and institutionally independent witnesses.

The two rows that remain can be filled by a reader of this paper, and the repository says how. The stranger’s path (docs/STRANGER-PATH.md) goes from a clean machine, the published package and an unmodified outside wallet to an accepted presentation in about ten minutes, without cloning the repository, and it is walked end to end before it is changed. Someone who is not the author walking it and reporting the result, in the repository’s issue tracker, is the first missing row; the presentation it ends in is the second. A failure counts as much as a success: it is an external finding, and each of the two recorded so far became a fix. The scoreboard fills a row only with a name and a date, so a report is evidence only when its author is willing to be named. A reader without Docker has a shorter door: each of the other three packages’ READMEs, on the registries from their next release, opens with a first run against the repository’s published vectors, a genuine credential and a tampered one, that takes a minute from the registry, and each was walked as a stranger on 23 September 2026.

EXTERNAL VALIDATION REQUIRED

The contract also names the other end. On 12 March 2027, day 180, the external fields are read. If they remain effectively empty while internal versions, invariants and architecture keep growing, the core is to be frozen, the national-scale architecture stops expanding, the verifier is kept only if external use exists, and the project is archived as a reference implementation. This paper’s subject may be archived by its author’s own rule, and the paper records the rule rather than arguing with it.

EXTERNAL VALIDATION REQUIRED

18.2 Questions the lab still owes an answer

Each of these is a measurement, not a feature, and the lab's discipline is to publish the threat model, the adversary, the harness, the result and the limitations together, with a positive control before any null result counts.

The rest of the linkability threat model. The measured result covers stable fields only. Presentation size, timing, issuer metadata narrowing the anonymity set, status artifacts stapled to a presentation, and transcript structure are unmeasured, and the one channel the adversary can read, transcript length, is known to be live in any deployment where a presentation's size varies per holder.

The interoperable path's nine handles. Batch issuance, many credentials with one holder key each, is the ecosystem's mitigation and is not implemented; whether it would be adopted is an outside party's question, and the cost of not having it is now a number.

The migration path's other half. Rollback, and the mixed window during a partial migration, are unmeasured. A verifier-readable algorithm status, in the registry or the trust list, would be a new product guarantee, so the lab recorded the gap and did not build it.

Duress against the informed coercer. A holder-side panic signal, a deniable enrolment that makes the absence of a duress code unprovable, and a purge path for the duress record are each a mechanism change, and the assessment proposes none of them. What it asks for first is a measurement the repository cannot make: what a coercer standing beside a holder actually observes.

The anonymity-set floor. Reporting the crowd size beside a bounded verdict, or refusing to close an epoch below a floor, would change what the system promises; both are recorded for a deploying organisation to decide.

18.3 Presentation technologies that would remove the correlator, held

Version 2 listed five families. One, the pairwise identifier, is built, with the bound Section 8 states. The other four are held rather than scheduled, and the reason each is not built is the contract's, not a technical one.

Blinded or derived presentations. The holder proves possession of a signature over attributes without revealing the signature itself. The classical constructions rely on pairing-based assumptions that a quantum computer breaks; post-quantum equivalents are an open research area, and the repository's posture would have to be preserved.

One-time presentation tokens. Single-use artifacts minted per presentation, at the cost of issuer contact or a pre-fetched batch, which trades against offline availability and issuer non-observation exactly as the status assertion does.

Anonymous credentials with selective disclosure. Revealing that an attribute satisfies a predicate without revealing the attribute. The benchmark against a mature system measured the gap precisely: Polaris as an issuer has no attribute credential to apply a predicate to, and as a verifier it accepts a pre-computed boolean the issuer had to anticipate.

A native anonymous predicate. The benchmark's first scenario is satisfied by a disclosed attribute, not by an anonymous predicate; that is on the scoreboard as a known limitation, which moves only when an outside party trips over it.

18.4 Metis: learning the system, never the people

The assessment the roadmap requires before any code would have to falsify the central hypothesis (that a model can infer properties of the system without inferring the rights of people) against a synthetic-nation ground truth, compare every model to a deterministic rule, and design the constitutional invariant that would forbid the prohibited uses structurally, in the manner of Athena's six checks. Under the contract the assessment itself waits on an outside party needing what it would produce; nothing about Metis is scheduled, and this paper keeps the brief because the prohibition in it binds any future cognitive layer whether or not Metis is ever built.

18.5 Myrmex: stigmergic debugging as an outside observer

Version 2 proposed, under the provisional name Myrmex, a debugging system that would live outside Polaris and attack it independently, inspired by how an ant colony coordinates without a central plan: specialised agents leaving machine-readable evidence on a shared landscape, confidence drawn only from reproductions and never from agreement between models, and every reproducible failure entering the loop of Section 15 as a test and an invariant. The proposal stands, and the caution with it: an earlier apparatus that lived inside the tree and asserted claims about itself was deleted as scaffolding, so Myrmex would be held to the opposite discipline, outside and adversarial, or not built.

What Version 3 can add is that the thing Myrmex was for has been demonstrated without it. The mutation drills are the colony's cheapest members, built inside the tree: they do not reason, they delete and invert and watch for red, and they found more than any review had. And the observer that found what no drill could, a certificate every internal instrument accepted, was not a colony of agents but one wallet written by strangers. The proposal is kept because a stranger's wallet exercises one path, and the rings inside it are still watched only by their author's instruments.

18.6 Engineering items held under the contract

A hash-based signer behind the same custody interface; the chain-backed ledger drivers whose interfaces are declared; native wallet clients and a browser bridge; status distribution as a deployment rather than a specification; a manufactured card; and the external-implementation path for the native protocol. Each is a roadmap row that the contract has moved from *next* to *when somebody outside needs it*, and the repository's own history is the argument for the move: three hundred and ninety-seven versions were spent building rings that no one outside had asked for, and the first outside party to arrive found a defect in an afternoon.

19 Current limitations and the external validation still required

The repository's readiness ledger opens with one line, and this paper repeats it as its own bound: at 1.0.0-rc.7 Polaris is not production-ready for real identity data. Every engineering gap the ledger enumerated is closed and pinned by a check; what remains is not buildable in the repository. The list below separates what has not been done from what cannot be done by the author, and it is longer than Version 2's in one direction, because the repository spent the interval finding limits rather than removing them.

19.1 Decisions only a deploying organisation can make

Nine decisions must be recorded for a named deployment before real data could be held: a legal basis, impact assessment and regulator approval; the signing-key custody driver and who holds the key; the high-availability topology and its hosts, and whether a second region is run; encryption at rest and its key custodian; the off-site backup target and schedule; the alerting backend, the on-call rotation and who receives the duress page; the right-to-erasure policy; the retention schedule per class, with the counsel who says the number satisfies the statute; and an independent penetration test with a threat-model sign-off. Polaris records each decision and its justification; it does not know the law. EXTERNAL VALIDATION REQUIRED

19.2 What is not built

- The physical token is a specification, an emulator and published vectors; no card is manufactured and no silicon is certified, and the profile states which of its properties an emulator cannot establish.
- No hardware security module has been used; the PKCS#11 driver is proven against a software token, the KMS driver against a stand-in, and the lattice library is not FIPS-validated.
- A second region is a standby cluster with a non-zero recovery point; status distribution is a set of cache rules, not a deployment; and the national workload has been simulated at a scale factor, never run.
- A hash-based signer is registered and not wired. The hop from the pooler to the database, the certificate signatures and today's operator authenticators remain classical.
- A verifier cannot learn that an algorithm has been deprecated, so retiring one is a software release to every relying party; rollback and the mixed window of a partial migration are unmeasured.
- Blinded and one-time presentations are not built, so a plain credential shows a verifier a stable token value, an issuer signature and a holder key, and an SD-JWT VC presentation shows nine issuer-signed values that are the same everywhere. The per-relying-party handle bounds what a verifier *stores*; only the zero-knowledge path bounds what it is *shown*, and that bound is the size of the epoch, which nothing floors above one and no verdict reports.
- Neither the scoped nullifier nor the pairwise handle hides a holder from the *issuer*, which derives every epoch leaf to build the tree. Issuer-blind derivation is not claimed.
- Polaris as an issuer has no attribute credential and no anonymous predicate; as a verifier it accepts a disclosed boolean the issuer had to anticipate.
- The duress mechanism resists the casual coercer and neither the informed coercer beside the holder nor lawful or institutional access, against which its indelible record is net-negative. No holder-side panic signal exists and none is proposed.
- Document authenticity at enrolment is taken on the operator's word; the assurance level is derived from what evidence was presented, not from whether it was genuine.
- Of the identifier columns that grow at the enrolment rate, two remain 32-bit, `Individual.individual_id` and `IdentityToken.token_id`: about twenty-nine years of headroom at the stated surge of 200,000 a day, past the capacity model's horizon; the interim credential during a recovery cool-down is not built; a sunset verdict rests on operator attestations the system cannot verify.

19.3 What has been done by someone else, and what has not

- One unmodified wallet written by someone else has presented a credential and been accepted, on one path, in one format, with one signature algorithm, and it refused Polaris once first. No second wallet, no wallet on the native protocol, and no outside relying party.
- The OpenID Foundation's hosted suite ran the verifier profile: eleven modules, zero failures, seven negative modules passed by the service's own scoring, four positive modules in review awaiting a human the Foundation supplies at submission. Not submitted, not certified.
- A published third-party corpus agrees with both signature witnesses on the ML-DSA-65 primitive. It tests the primitive, not the protocol.
- Four packages are on public registries, each verified by installing from the registry into a clean environment. Nobody outside the repository has reported installing one, and a registry download count is not a person.
- No external penetration test, cryptographic audit, red team, bug bounty or certification has occurred. The scope document and the review packet exist; no engagement has been commissioned. Almost every guarantee in this paper is checked by machinery written by the same author as the guarantee.
- No operator other than the author has run any of it, which is the one condition that separates the release candidate from 1.0.0. No pilot with consenting users has run; the data has always been notional.
- No external team has implemented the native protocol from the specification and conformance suite alone; the suite is the contract and the integration has not happened.
- No institutionally independent witness watches any log; the witnesses in the record are processes in a drill. Two federated instances are two instances of one code base run by one author.
- The circuit over the proof library has had no external review, and the concrete bit security of the shipped proof configuration has not been independently derived.
- The repository's thesis, that a stranger can read it cold and correctly infer what it needs, was never tested by a stranger before its deadline and is recorded as inconclusive, with the strong claim permanently retired; the stranger's path of Section 9 is the narrower test that replaced it, and nobody outside has yet reported walking it.

19.4 What the numbers are

Every performance number in this paper is single-node on one laptop, or a simulation at a stated scale factor, or a projection that the repository labels as one under a named assumption. No production cluster has been measured, and the one measurement on a real failover topology is a median at two requests per second with the tail withheld because the sample cannot carry it. The national planning targets are targets; the arithmetic that clears them is the easy half of the question, and the half that nearly failed was an integer column.

19.5 Documentation that lags the code

Researching this version found places where the repository's own prose is behind its code or its history; they are recorded in the author's notes that accompany this version rather than fixed silently, as Version 2's were. Where this paper and any repository document disagree, the paper followed the code at 1.0.0-rc.7 and says so in the note.

20 Conclusion: the whole organism

A reader who has followed the rings outward should now be able to answer the questions this paper set out to make answerable. Each answer below is one sentence, and each points back to the section that carries the mechanism.

What the central credential is. One signed claim, *this authority issued this token to this person*, held as a file the holder controls and optionally bound to a key the holder holds, in a lifecycle with exactly one active state, with what its enrolment rested on recorded beside it (Section 3).

Why the schema surrounds it. Because a promise enforced only by the people who run the system is not a promise; triggers, check constraints and unique indexes bind every client and every restore, and each is now mutation-tested so the tests that prove them are believed for a reason (Section 4).

How cryptographic trust works. ML-DSA signatures over digests, checked by two independent implementations at issuance and sampled at use, with keys that have a recorded lifecycle and an algorithm that is a row rather than a constant, so that a population can be re-signed when the assumption under it falls (Section 5).

How verification differs from authorization. Authenticity is permanent and cacheable; authorization is fresh and revocable, decided on the primary or by a short-lived signed assertion whose window is the honest cost of going offline (Sections 6 and 7).

How privacy is bounded. By what is stored, not by who may read it: no token identifier on a zero-knowledge event, a membership proof that reveals nothing but membership within an epoch's crowd, bounded reads, and a measured statement of what two colluding verifiers can and cannot do on each presentation path (Section 8).

How relying parties use credentials. Through a wallet protocol, four packaged verifiers, a login broker that keeps no record of who logged in where, and a path a stranger can walk from a clean machine to an accepted presentation (Section 9).

How authorities federate. Each with its own key and no central store, by directional per-context attestations that never form a chain, and by signed feeds a relying party decides on offline (Section 10).

How transparency constrains authority. Append-only logs with signed heads, monitors that prove history only grew, witnesses that cosign, gossip, and produce a non-repudiable proof when a log tells two stories, and a public count of the authority's own most invasive act (Section 11).

How institutional exchange works. Through a signed registry, a gateway that authenticates by key and authorizes by the responder's own attestation, receipts that commit to hashes and never bodies, a timestamp authority that learns nothing, and signatures that outlive their keys (Section 12).

How Polaris survives failures. By drilling them: failover, a lost region, replicas, partitions, backup and restore, chaos, rolling deploys and the end of a pilot, each with a measured number and a ceiling, on every push (Section 13).

What Atlas and Athena do. Atlas sees what the system is doing in bounded aggregates; Athena understands the structure of authority and the constitution as data; neither may model a person (Section 14).

Why AI agents change the identity problem. Because the question becomes what kind of actor is acting, under whose authority, within what scope, verifiable without learning anything else (Section 17).

- How privacy-preserving proof of personhood fits.** On the existing uniqueness constraint and membership proof, with a scoped nullifier, a holder-side key and a prover on the holder's device, all built, and a presentation that carries no stable handle, which exists only on the zero-knowledge path (Sections 17 and 18).
- How delegated agent authority works.** As a signed, scoped, expiring, separately revocable grant from a human's credential to an agent, verified by a service that learns only the five answers it needs, with the bounds that keep it from being a bearer token mutation-tested (Section 17).
- How stigmergic debugging could help.** As an outside colony of specialised agents leaving evidence on a shared landscape, with confidence drawn only from reproductions; it remains a proposal, and the first outside observer arrived instead as somebody else's wallet (Section 18).
- What Polaris can do today.** Issue, sign, verify, revoke, present, federate, log, witness, exchange, timestamp and sign documents, re-sign a population, wind a pilot back, be installed from a public registry, and accept a presentation from a wallet it had never met, all on notional data, all drilled in continuous integration (Appendix A).
- What it cannot do today.** Hold real identity data, run on hardware tokens or a hardware module, prove independence of institutions or implementations, prevent correlation on a plain or interoperable presentation, resist a coercer who is the institution, or claim any property that only one wallet, one hosted suite and one test corpus have tested from outside (Section 19).
- What it is ultimately trying to become.** Identity infrastructure in which no important claim has to be taken on the word of a central application, because authority, state, history, privacy and the system's own behaviour are exposed to independent verification and constrained by layers that do not share each other's assumptions, and, since this version, because the last of those layers is other people's software.

The town's register is small. The walls around it are what make it safe to keep. And the reason the system has so many rings is not that its builders enjoyed rings: each one was added when the ring inside it was found to be correct and still unable to prove what a society would need to know. Credential authenticity is not current authorization. Current authorization is not privacy. Privacy is not freedom from coercion. Federation is not institutional independence. A transparency log is not protection from a split view. A green test is not proof that the test asked the right question. And a system that has checked itself is not a system that anyone else has checked. Polaris is the accumulation of those sentences into mechanisms, and the honest map of where the accumulation stands: at version 1.0.0-rc.7, a release candidate, which is the number for a system whose every internal condition for 1.0 holds and which no operator other than its author has yet run.



Appendix A Capability status

The table answers, for every major subsystem, whether it runs, what limits it, which invariant check fails the build if it stops being true, and, where the mark says so, who outside the repository has exercised it. The marks are those of the front matter. Every row was measured at 1.0.0-rc.7. Rows carrying `EXTERNALLY EXERCISED` point at a filled row of the repository's scoreboard; no row moved from running to proposed since Version 2, and three moved from internal to externally exercised.

Table 19: Capability status at 1.0.0-rc.7. A row carrying two marks runs today and still requires external validation for the stronger claim, or has been exercised from outside on one path and not on the others.

Capability	Status	Limit stated by the repository	Pinned by
Schema constraints, C1–C10	IMPLEMENTED	The database owner is outside the threat model; three constraints carry a checked formal model	check_c1c10_objects_resolve, the TLA+ drill
Append-only audit of record (fourteen named; thirty-one tables)	IMPLEMENTED	Tested once per table that claims it; a reload once reached one record with neither its ids nor its rows	check_aor_append_only_triggers, check_append_only_tables_are_tested_exhaustively, check_seed_restart_resets_dependent_records
Retention as data with a 365-day floor	IMPLEMENTED	Archive and purge remain deliberate operator steps	check_retention_engine
Mutation drills over the verification layer (twelve)	IMPLEMENTED	A shared misreading of a specification survives every one of them	check_constraint_suite_is_mutation_tested, check_triggers_are_mutation_tested, check_procedure_refusals_are_mutation_tested, check_sdk_refusals_are_mutation_tested, check_zk_witnesses_are_mutation_tested
ML-DSA-65 and ML-DSA-87 signing	IMPLEMENTED	The default development and CI path signs a labelled placeholder; SLH-DSA registered, not wired	check_pqc_signing_wired, check_real_pqc_default_boot
Two-witness verification, sampled at use	IMPLEMENTED	Catches divergence, not a shared misreading	check_pqc_second_witness, check_verify_witness_sampling
ML-DSA-65 verification against a published corpus	EXTERNALLY EXERCISED	The primitive, not the protocol: 58 of 58 vectors under both witnesses	the pqc-real job
Key custody: file, PKCS#11, KMS	IMPLEMENTED EXTERNAL VALIDATION REQUIRED	Software token in CI, no hardware module; the KMS is a stand-in	check_key_custody_abstraction
Key register and signed trust list	IMPLEMENTED	Rotation of a live key file or slot is an operator ceremony	check_trust_lifecycle
Algorithm agility, population migration	IMPLEMENTED EXTERNAL VALIDATION REQUIRED	A verifier cannot learn a deprecation; rollback and the mixed window unmeasured	check_algorithm_agility, check_quantum_event_readiness, check_post_quantum_claims_are_agility

continued on the next page

Capability	Status	Limit stated by the repository	Pinned by
Detached verifier, packaged, fuzzed and attacked	IMPLEMENTED	Refuses to start without a declared crypto mode; needs the prover binary for the zero-knowledge path only	check_detached_verifier, check_verifier_fuzz, check_attacks_run, the product-boundary drill
Python and TypeScript SDKs, conformance suite (118 cases)	IMPLEMENTED EXTERNAL VALIDATION REQUIRED	No external implementation of the native protocol; 44 verdict fields unconstrained by any case	check_conformance_suite, check_typescript_sdk, check_conformance_asks_the_relying_party_question
Frozen version 1, cross-version suite	IMPLEMENTED		check_protocol_versioning
OpenID4VP 1.0 + HAIP verifier	IMPLEMENTED EXTERNALLY EXERCISED	One wallet, one format, one path, ES256 throughout; four hosted modules in REVIEW, not certified	check_oid4vp_verifier_boundary, the OpenID4VP mutation drill, the scoreboard
Format bridges: ISO 18013-5 rendering, Verifiable Credential of a verdict	IMPLEMENTED	Formats, not trust models; no standard mDL reader can verify the ML-DSA issuer signature	the mdoc and VC drills
Relying-party verify API	IMPLEMENTED	Client-credentials only; mutual TLS deferred	check_relying_party_api
Signed status assertion, offline authorization	IMPLEMENTED	The revoked-but-valid window is the cost; its maximum rests partly on relying-party configuration	check_offline_verification, the status-freshness model
Epoch checkpoints, revocation feeds, status bundle, cache rules	IMPLEMENTED PROPOSED	Distribution at content-delivery scale is specified and drilled, not deployed	check_epoch_revocation_propagation, check_federation_status_bundle, check_status_distribution
Zero-knowledge membership proof, holder-side prover, scoped nullifier	IMPLEMENTED EXTERNAL VALIDATION REQUIRED	No external audit; the issuer can compute any member's nullifier; the anonymity set has no floor above one	check_zk_tree_depth_synced, check_holder_side_prover, check_scoped_nullifier, the witness differential
Issuer-side unlinkable verification records	IMPLEMENTED	Scoped to zero-knowledge mode; adversary shaped so it cannot flatter the result	the C2 CHECK and model, the redaction suite, check_redaction_adversary_is_not_flattered
Cross-verifier correlation, measured	IMPLEMENTED	Bounded on the zero-knowledge path within the epoch's crowd against an equality adversary; exposed on plain and SD-JWT presentations	check_pairwise_presentation, the linkability lab
Wallet protocol, presentations, QR framing	IMPLEMENTED	No native clients, no browser bridge	check_wallet_presentation, check_qr_resource_bounds
Auth broker with registered policy	IMPLEMENTED	No session product, no consent screen	check_auth_broker, check_broker_policy_bound
Decisions about authorities, operators, quotas, ceilings and relying parties, recorded	IMPLEMENTED	A shell and the tool are recorded on the same terms	the audit-of-record checks, check_every_audit_event_has_a_writer
Identity proofing, trusted referee, enrolment code	IMPLEMENTED EXTERNAL VALIDATION REQUIRED	Evidence authenticity is the operator's word; no proofing has been performed with it	check_enrollment_code, check_trusted_referee, the proofing drill

continued on the next page

Capability	Status	Limit stated by the repository	Pinned by
Federation: roots, attestations, manifests	IMPLEMENTED EXTERNAL VALIDATION REQUIRED	Attestations operator-recorded; two instances by one author	check_federation_topology, check_federation_two_instances
Per-authority isolation of operators	IMPLEMENTED EXTERNAL VALIDATION REQUIRED	A shared instance's owner, backup and schema are shared	check_route_guards_are_derived_not_listed
Cross-authority zero-knowledge	IMPLEMENTED	Abstains without the binary; issuer bound by root	check_cross_authority_zk
Transparency logs, monitor	IMPLEMENTED	A monitor alone cannot see a split view	check_transparency_log
Witnesses, gossip, equivocation proof	IMPLEMENTED EXTERNAL VALIDATION REQUIRED	Witnesses are drill processes; independence is a deployment	check_transparency_gossip
External-ledger publication	IMPLEMENTED PROPOSED	File backend only; chain drivers declared	check_transparency_publication
Public transparency report	IMPLEMENTED	Cannot see an access that was never recorded	check_transparency_program, check_audited_reads_are_logged
Signed registry, exchange gateway and receipts, timestamp authority, document signing	IMPLEMENTED	Bare algorithm names in the registry; notarial signing; anchor verification in verifier and SDKs	check_registry, check_exchange_gateway, check_exchange_receipt, check_timestamp_authority, check_document_signing, check_ltv_timestamp_trust
HA, replicas, partitioning, DR, chaos, rolling deploys	IMPLEMENTED	Ephemeral CI hosts	check_ha_automation and its siblings
Second region as a standby cluster	IMPLEMENTED	Asynchronous; the recovery point is not zero and is measured	check_multi_region_dr
Capacity and cost models	IMPLEMENTED EXPERIMENTAL	Core counts extrapolated under a named assumption; nothing run at scale	check_capacity_model
Post-quantum TLS edge and application-to-pooler hop	IMPLEMENTED	The pooler-to-database hop is classical, gated on a PostgreSQL release	check_edge_pq_kex, check_internal_kex_measured
Abuse controls, session and origin hardening, WebAuthn MFA, logs without PII	IMPLEMENTED	Authenticators classical until hardware ships	check_abuse_controls, check_session_origin_hardening, check_logs_exclude_pii
Atlas	IMPLEMENTED	Every aggregate graded against its data's growth	check_atlas_console, check_benchmark_measures_growth, the C8 caps
Athena	IMPLEMENTED	Read-only; no person node, which one check refuses	check_athena_no_person
Card profile, software token, personalization, verifier device, duress timing	IMPLEMENTED PROPOSED	No card manufactured, no silicon certified	check_card_profile, check_card_emulator, check_verifier_device, check_duress_on_card
Duress code, in the application and on the card	IMPLEMENTED EXTERNAL VALIDATION REQUIRED	Duress-aware, not compulsion-resistant: resists the casual coercer only, and its record is indelible	check_duress_alertable, check_duress_claims_are_aware
Pilot wind-down; coexistence and sunset plan	IMPLEMENTED EXTERNAL VALIDATION REQUIRED	No pilot has run; the sunset verdict rests on operator attestations	check_coexistence_plan, the wind-down drill
NIST 800-63 mapping; accessibility	IMPLEMENTED EXTERNAL VALIDATION REQUIRED	Not a conformance claim; the automated third only	the assurance-mapping and accessibility drills

continued on the next page

Capability	Status	Limit stated by the repository	Pinned by
Formal models: C1, C2, C3, status freshness	IMPLEMENTED	Models, not the code; each of three must fail a counterpart configuration	the TLA+ drill
Holder key bound to a credential	IMPLEMENTED	Optional; the proof deliberately does not cover the presented code	check_holder_key_binding
Agent delegation, revocable by the holder	IMPLEMENTED	A plain binding still shows the service the credential	check_agent_grant
Four packages on public registries	IMPLEMENTED	Distribution is not validation; no outside install reported	the publish workflow, the boundary drill
Metis (learning the system)	PROPOSED	Assessment required before code, and an outside need before the assessment	
Myrmex (stigmergic debugging)	PROPOSED	Proposed in this paper	
Blinded or one-time presentations; a native anonymous predicate	PROPOSED	The residue: a plain or interoperable credential shows a verifier stable material	
Hardware token, enrolment kit	PROPOSED	Roadmap rows, held	
A non-author operator, a pilot, certification, external audit, independent witnesses	EXTERNAL VALIDATION REQUIRED	External actors	the scoreboard

Appendix B The metacognitive map

The ladder below is the paper’s answer to the request for an explicit map of Polaris as layers of knowledge and scepticism. Each rung states what the layer knows and what it refuses to take on faith from the rung below. The layers are not exactly the rings of Figure 1, because some rings hold two kinds of knowledge; the rungs follow the implementation. Version 3 adds a rung at the top, for the knowledge the tree cannot check about itself; Figure 17 draws the ladder after the table.

Table 20: The layers, what checks each, and whether each is needed now or is preparation.

Layer	Name	Checked by	Now or later
L0	Human subject	Nothing in the software; the enrolment tiers keep non-enrolment a named state and the referee keeps the undocumented enrollable	Now
L1	Credential core	C3, the signature triggers, the lifecycle trigger, the C3 model	Now
L2	Structural constraints	The audit-of-record and cascade checks, the CHECK-constraint suite, and the drills that mutate all three mechanisms	Now
L3	Cryptographic evidence	The two-witness checks, the canonical oracle, the attack suite, the migration drill, one outside corpus	Now
L4	Verification and status	The detached-verifier, conformance, offline-verification and status checks; the boundary drill; the SDK mutation drill	Now
L5	Privacy and disclosure	The C2 CHECK and model, the redaction suite and its adversary, the linkability adversary, the Athena person checks	Now

continued on the next page

Layer	Name	Checked by	Now or later
L6	Federation	The topology, manifest, feed, isolation and two-instance checks	Now as protocol; later as institutions
L7	Transparency	The log, gossip, publication and report checks	Now as protocol; later as independent witnesses
L8	Institutional services	The registry, gateway, receipt, timestamp and signing checks	Now between instances
L9	Operations	The HA, region, replica, partition, DR, chaos, capacity and hardening checks; the wind-down drill	Now on drills; later on production hardware
L10	System cognition	The Atlas caps and growth grading, the six Athena checks	Atlas and Athena now; Metis, Myrmex later
L11	Societal boundary	C10's structural absence, the non-goals, the vocation, the claim-discipline checks	Now, and permanent
L12	Agent and web environment	The grant, nullifier and holder-key checks; the residue is a stable handle on any presentation that is not zero-knowledge	Now, with the residue named
L13	Other people's software	Nothing in the tree. The scoreboard records what a named outside party did, on a date, and refuses anything the author ran against the author's own code	One wallet, one hosted suite, one corpus now; everything else later, and not ours to supply

	WHAT THE LAYER KNOWS	WHAT IT REFUSES TO TAKE ON FAITH
L0 Human subject	exists outside the software	the software's description of the person
L1 Credential core	one active token, one signature, one person	any state the schema cannot represent
L2 Structural constraints	which states may exist at all	application code promising to behave
L3 Cryptographic evidence	who signed what, checkable by anyone	a single implementation's verdict
L4 Verification and status	authentic, and authoritative right now	authenticity as if it were authorization
L5 Privacy and disclosure	what a verifier is allowed to learn	the client's claim about its own disclosure level
L6 Federation	whom an authority explicitly trusts, in which context	transitive trust, a central authority
L7 Transparency	that history was not rewritten or forked	a single observer's view of the log
L8 Institutional services	that an exchange happened and was authorized	keeping the message to prove it
L9 Operations	that the system survives failure and load	a claim that was not drilled
L10 System cognition	what the system is doing and how power is structured	anything about a person
L11 Societal boundary	what Polaris refuses to become	money, scoring, a required doorway
L12 Agent and web environment	what kind of actor is acting, under whose authority	a stable handle on any presentation that is not zero-knowledge (the residue)
L13 Other people's software	what a stranger's wallet, a hosted suite and a published corpus found	anything the author ran against the author's own code

ZOOM: ATOM, ORGAN, ORGANISM, TOWN, FEDERATION, ECOSYSTEM, THE OUTSIDE

Figure 17: The ladder. From the person, who exists outside the software, to the software of strangers, who exist outside it too. The right-hand column is the refusal each layer makes of the layer below it; the last rung's refusal is the operating contract's, and nothing in the tree can discharge it.

Appendix C Counts at 1.0.0-rc.7, and how they were measured

Every count in this paper was taken from the tree at version 1.0.0-rc.7 (commit 4e74376, 20 September 2026) by the command beside it, not copied from an earlier document. Where the repository’s own stamped number was measured at an earlier version, the version is stated. Where a number is external, the row says whose it is.

Version 3 of this paper was written at 1.0.0-rc.1 and re-measured here. Fourteen of the twenty-six quantities below are unchanged; the rows that moved are marked in the text where they matter. One method changed rather than one number: the route count is the same and the command that produced it no longer is.

Table 21: Counts at 1.0.0-rc.7 (commit 4e74376) and their method. The tree moves after the measurement; a count taken later can differ, and the commit is what makes each one reproducible.

Quantity	Count	Method
Canonical tables in 01_schema.sql	45	The file declares 50 CREATE TABLE statements, of which five are the default partition children of the partitioned event tables. A migrated deployment holds 52 tables.
Triggers in 06_triggers.sql	42	grep -c 'CREATE TRIGGER'
Triggers in the loaded schema	46	The file’s 42, and four declared beside what they guard: one in the migration registry’s file, one in 01_schema.sql, two in 12_v7_constraints.sql. Read from pg_trigger on a loaded and migrated database, excluding internal triggers and the 25 copies PostgreSQL makes on the partitions of partitioned tables
Tables under an append-only trigger	31	The BEFORE UPDATE OR DELETE triggers of 06_triggers.sql and the migrations, by table; the design record names fourteen audit-of-record instances among them
Invariant checks	313	grep -c '^def check_' polaris_checks/checks.py, equal to the registered list; each has a detection test with a positive control in test_checks.py
Mutation drills	12	Files matching scripts/polaris-*mutation-drill*: checks, constraints, triggers, procedures, the zero-knowledge witnesses, the conformance contract, the two SDKs, the OpenID4VP verifier, the application, coverage, the constitution, and moves between modules
Application routes	124	grep -c '@app.route(' across polaris_web/*.py. The count is unchanged from 1.0.0-rc.1; the METHOD is not. On 18 September 2026 app.py was decomposed into ten route modules, and the old method, which read that one file, now returns 13
CI jobs in ci.yml	20	The job keys of the workflow; two more workflows run monthly and weekly, and a manual one publishes
Signed wire formats in the specification	27	Distinct polaris-*/1 strings in docs/reference/WIRE-SPEC.md
Conformance cases	118	The cases of conformance/cases.json, across twenty-five artifact types; the frozen version-1 set holds 44
Verdict fields no published case constraints	44	SURVIVORS_EXPECTED of scripts/polaris-conformance-mutation-drill.py, which the drill holds to its own measurement in both directions: of 89 decision fields across the 27 verifiers the cases reach, 44 survive being forced permissive, and none of them could be constrained by writing a case
Formal models checked in CI	4	meta/tla/*.tla; three carry a counterpart configuration they must fail

continued on the next page

Quantity	Count	Method
Drill scripts	80	Files matching <code>scripts/polaris-*drill.*</code> , the twelve mutation drills among them
Migrations	45	Files matching <code>polaris_sql/migrations/*.up.sql</code> , each with a revert
Operator runbooks and ledgers	21	Markdown files in <code>docs/operator/</code> , the index included
Tagged versions	3	<code>git tag</code> . At 1.0.0-rc.1 this was 295, one per ship from v9.30 (16 May 2026) to v9.466 (15 September 2026). A tag marks a RELEASE and not a ship, so those were removed; what remains is v1.0.0-rc.1, -rc.2 and -rc.3, the three published candidates
Test functions defined	2,189	<code>grep -c 'def test_'</code> across the product, CLI, check-layer, script, card, simulation, SDK and package suites; a definition count, not a run count
Product tests passing, as stamped by the README	1,030	Measured at 1.0.0-rc.7 on the reference machine, <code>pytest -q</code> per suite; three skip without real ML-DSA
Crypto witness tests, as stamped	107 of 112	Measured at 1.0.0-rc.7, unchanged from 1.0.0-rc.1; five need a PKCS#11 module or a real KMS key, and CI runs all but the KMS one
OpenID4VP verifier package tests	244	<code>pytest</code> in <code>packages/polaris-oid4vp</code> ; three are the positive control without which the refusals are vacuous
Packages on public registries	4	<code>polaris-verify</code> , <code>polaris-oid4vp</code> , <code>polaris-sdk-python</code> on PyPI; <code>polaris-sdk-ts</code> on npm; each verified by installing from the live registry. All four served 1.0.0-rc.1 when Version 3 was written. 1.0.0-rc.3, published 18 September 2026, is a pre-release: an installer takes it only when asked (<code>pip install -pre</code> , the npm tag next), and a plain install still resolves 0.1.0, the last stable release. Read off the live registries on 23 September 2026
Hosted conformance modules	11	The OpenID Foundation's service, 15 September 2026: 7 PASSED, 4 REVIEW, 0 failures, 0 warnings; the service's own signed exports
External test vectors for ML-DSA-65	58 of 58	Project Wycheproof, pinned to a commit, under both witnesses on every push
Named outside implementations that have presented	1	walt.id Wallet API v2 at a pinned image digest, 15 September 2026; one path, one format
Fixes caused by an external finding	2	The scoreboard's row: the certificate the wallet refused (15 September 2026), and a version stamp an outside reviewer could not read unambiguously (17 September 2026)
Independent security reviews, pilots, non-author operators	0	The scoreboard's rows, blank

The paper's performance numbers are those of Table 12, each with the version that measured it. The application baseline has not been re-run since v9.191 and is cited with that stamp. The dyno was re-run on 23 September 2026 against the tree at 1.0.0-rc.7, with the prover rebuilt from it, and the re-run found that its proof measurement had measured nothing since v9.354: it still sent the prover an input the prover no longer accepted, printed *not measured here*, and exited cleanly, so its CI step stayed green. The signature rows moved by a few per cent; the proof row fell from 35 ms to 25 ms, and is not like for like, because the statement proved changed at v9.354. A half that does not measure now fails the step. The reference machine throughout the repository is an Apple Silicon laptop with eight cores and sixteen gigabytes of memory.

Appendix D Glossary

Terms come first, in alphabetical order. The abbreviations the text uses follow them, each expanded, and for a standard or an organisation, what it is.

Terms

Anchor set. The published issuer keys a relying party has chosen to trust. Trust in Polaris is always membership in the verifier's own anchor set, never a decision the issuer makes for it.

Anonymity set. The crowd a membership proof hides in: the members of the epoch it was made against. Nothing in the schema floors it above one, and no verdict reports it.

Append-only. A table on which UPDATE and DELETE are refused by a database trigger, so history can only be added to. The audit of record is the named pattern of applying this per table.

Assurance level. What an enrolment can honestly claim about how the person was proven, derived from the evidence recorded beside it and never typed by an operator.

Attestation. A directional, per-context statement that one agency accepts another's credentials, recorded as a row. Never transitive.

Authenticity pack. A credential as a file: the token value, its signature and the issuer's public key. What a holder holds and a stranger verifies.

Canonicalisation. Turning a structure into one exact sequence of bytes (sorted keys, no white-space) so that a signature made on one machine can be checked on another that rebuilds the same bytes.

Consistency proof. A short proof that a smaller Merkle tree is a prefix of a larger one, so that a log only appended between two heads.

Detached verifier. A program that checks Polaris artifacts with no Polaris code, no database and no network: the `polaris-verify` package, and the script that is now a shim onto it.

Disclosure level. What a verification records: zero-knowledge (no token identifier), selective (named attributes), or full (the identity, logged).

Duress-aware. The word the front door uses for the duress code, because the mechanism notices duress and does not resist compulsion; the stronger word is refused by a check.

Epoch. A closed, immutable commitment to the active-token set at a moment, as a Merkle root, against which membership proofs are made.

Equivocation. A log telling two observers two different histories. The proof of it is two signed heads of the same size with different roots.

External noun. A named party outside the repository that did something to it, on a date, with a result that can be pointed at: the only kind of entry the scoreboard accepts.

Federation. Independent authorities, each its own root, recognising one another through explicit attestations, with no central service.

Gossip. Witnesses sharing the log heads they were shown so that a split view is detected by comparison.

High availability. A database that survives the loss of its leader by promoting a replica under a lease, drilled and measured.

Inclusion proof. A short proof that one entry is in a Merkle tree with a given root.

Long-term validation. Deciding that a signature was valid at the instant it was made, from evidence fixed at that instant, so that the decision survives the key's later retirement.

- Merkle tree.** A cryptographic fingerprint of a whole set, built by hashing pairs upward to one root, that lets a small proof show one item belongs to the set.
- ML-DSA.** The lattice-based signature scheme standardised as FIPS 204; parameter sets 65 and 87 are accepted. Post-quantum in its assumptions; the repository claims agility, not settled security.
- Mutation drill.** A program that removes or inverts a guarantee, a constraint, a trigger, a refusal, a check's inputs, and requires the tests that supposedly prove it to go red, with a negative control so that zero survivors is distinguishable from a harness that ran nothing.
- Nullifier.** A value derived from a secret and a scope that lets a verifier detect a second use without linking uses across scopes. In Polaris `Poseidon(secret || scope || epoch)`, a public input of the membership proof, per relying party and per epoch, never global.
- OpenID4VP.** OpenID for Verifiable Presentations, the protocol by which a wallet presents a credential to a verifier; the path on which the first outside wallet spoke to Polaris.
- Operating contract.** The rule adopted on 13 September 2026 that the unit of progress is a surviving external dependency, that a product change needs a named outside reason, and that everything else goes to the lab.
- Pairwise identifier.** A different handle for each relying party, so that two parties cannot join their logs. In Polaris the login token's subject and the presentation handle are derived under the relying party's scope; it bounds what a verifier stores, not what it is shown.
- Partition.** A table split into monthly pieces by timestamp, transparent to queries, so that old months can be detached to the archive.
- PKCE.** A proof-of-possession extension to the authorization-code login flow, so that an intercepted code cannot be exchanged by an attacker.
- Positive control.** The half of a measurement that proves the instrument can see anything: an adversary that must succeed on a linkable population, a mutated tree the check must fail on, a patched verifier the suite must notice.
- Post-quantum.** Cryptography whose security does not rest on problems a quantum computer solves efficiently. In this paper, a statement about algorithms, never about the system.
- Release candidate.** The version `1.0.0-rc.7`: every condition the operating contract names for `1.0.0` holds, and no operator other than the author has run it.
- Replica.** A copy of the database that follows the primary with some lag; safe for reading immutable material, unsafe for deciding current authorization.
- Revocation feed.** A signed list of the hashes of revoked credentials an authority issued, consumed offline.
- Scoreboard.** `lab/EXTERNAL-NOUNS.md`: the file that decides whether the project continues, holding only what named outside parties did, with zero and blank as valid values.
- SD-JWT VC.** A credential format in which the issuer signs digests of the claims and the holder discloses the ones a verifier asked for; the format the first outside wallet presented.
- Signed tree head.** A log's signed commitment to its whole history at a size.
- Standby region.** A second cluster with its own lease store, streaming asynchronously from the first, electing no primary of its own until promoted; its recovery point is measured, not zero.
- Status assertion.** A short-lived, issuer-signed statement of a credential's current status, stapled to a presentation so that authorization is decidable offline.
- Sunset.** The day the old credential stops being accepted, treated as a decision with a floor under it because it is the moment the new one becomes compulsory.

Trust list. A signed statement of every key an instance knows, with each key's status and the instants those statuses took effect.

Trusted referee. A person whose vouching lets somebody with no documents be enrolled, bounded and co-signed past a threshold, and invisible on the credential.

Two-witness principle. No cryptographic verdict is trusted from a single implementation; a second, independently written one must agree or abstain explicitly.

Witness. An independent party that cosigns log heads it found consistent and refuses a fork.

Zero-knowledge proof. A proof that a statement is true which reveals nothing beyond the statement's truth; in Polaris, membership of a token in an epoch.

Abbreviations

Every abbreviation the text uses, spelled out, with what each standard or body is.

API. Application programming interface: the requests one program uses to call another.

CBOR. Concise Binary Object Representation, the binary data format ISO 18013-5 mobile documents use.

CI. Continuous integration: the build, tests and drills run automatically on every push.

CSRF. Cross-site request forgery: an attack in which another site makes a browser send a request as a signed-in user.

ES256. The classical ECDSA signature over curve P-256 that the HAIP profile pins.

FIPS. US Federal Information Processing Standards; FIPS 202 defines SHA-3, FIPS 204 ML-DSA and FIPS 205 SLH-DSA.

HA. High availability.

HAIP. The High Assurance Interoperability Profile of OpenID4VP, which pins the credential format, the signature algorithm and the response mode a wallet and a verifier must use.

HTTP. Hypertext Transfer Protocol, over which web services exchange requests.

ISO/IEC 18013-5. The international standard for the mobile driving licence.

JSON. JavaScript Object Notation, the text format every signed Polaris statement is written in.

KMS. Key management service: a cloud service that holds keys.

NFC. Near-field communication: the short-range radio a card answers a reader over.

NIST SP 800-63. The US National Institute of Standards and Technology's digital identity guidelines, which define assurance levels.

npm. The public package registry for JavaScript and TypeScript.

PIN, PUK. The personal code that unlocks a card, and the unblocking key that clears a card locked by too many wrong PINs.

PKCS#11. The standard programming interface to hardware security modules and tokens that generate and hold keys inside themselves.

PyPI. The Python Package Index, the public registry for Python packages.

QR. A two-dimensional barcode read by a camera; one way a presentation is transferred.

RFC. Request for Comments: the series in which internet standards are published; RFC 6962 describes Certificate Transparency.

SDK. Software development kit: a library a relying party builds into its own program.

SHA3-256. A cryptographic hash function of the SHA-3 family (FIPS 202) giving a 256-bit digest; Polaris signs the SHA3-256 digest of a token value.

- SLH-DSA.** The hash-based signature scheme standardised as FIPS 205, registered as the fallback should the lattice assumptions fall.
- SQL.** The query language of relational databases, in which the Polaris schema, triggers and procedures are written.
- TLA+.** A formal specification language whose model checker explores every interleaving of events to show a property holds.
- TLS.** Transport Layer Security, the protocol of the encrypted connection; the Polaris edge negotiates a hybrid post-quantum key exchange.
- W3C.** The World Wide Web Consortium, which publishes the Verifiable Credentials data model.
- WebAuthn.** The standard for proving possession of a hardware key or a device’s built-in authenticator; operators use it as a second factor after the password.
- X.509.** The standard format of a public-key certificate.
- ZK.** Zero knowledge.

Appendix E From Version 1 to Version 2

Version 1 remains the historical report and is not edited. This appendix records what Version 2 kept, what it corrected, and what it retired, so that a reader of both can reconcile them. Version 3 keeps every decision below unchanged; what it changed against Version 2 is Appendix F.

Kept

- The token-centred entity geometry (Figure 2), reproduced from Version 1 without change and used as the nucleus of every map in this paper.
- The lifecycle state machine (Figure 3), reproduced without change.
- The disclosure-level argument: that the level is a persisted property of the verification record, enforced by a check constraint, and that this is what prevents the verification log from functioning as a surveillance database.
- The argument that post-quantum signing belongs on the first day, through the secrecy horizon of identity data and Mosca’s inequality, and the substrate argument that every higher property of an identity system is derivative of its cryptographic primitives. Both are summarised here and stand in Version 1 in full; what Version 3 narrows is the *claim* made about the signatures, not the argument for choosing them (Section 5).
- The comparison of design properties against deployed systems, which this paper does not repeat; the repository’s front page carries the current table and marks Polaris as neither deployed nor issuing at national scale.
- The threat-model frame (STRIDE), which the repository’s design record now carries with every threat mapped to a control or an explicit acceptance.

Corrected

- Twelve tables became thirty-six at Version 2 and forty-five at Version 3. The three subsidiary entities of Version 1 (device bindings, ledger anchors, the revocation list) remain; the rest of the growth is described ring by ring in Section 4.
- Version 1 stated that the audit tables were append-only by convention and tooling and that the state machine would be enforced by a trigger in a production deployment. Both are now enforced by triggers, and a check fails the build if either stops being.

- Version 1 described the ledger anchor as an optional decentralised-identifier path. The off-chain batch commitment exists and is monitored as a transparency log; publication to an external chain remains an operator action with declared drivers and no live integration.
- Version 1 modelled seven verification contexts with per-context biometric requirements and a physical token with on-device biometric binding. The contexts remain; the physical token is now an encoding, an emulator and a verifier device, and remains unmanufactured.
- The two speculative binding horizons of Version 1’s final appendix (genomic and quantum-observer binding) are not carried forward as design. The repository’s readiness pass quarantined the speculative schema behind a check that it cannot return, and this paper treats the holder-side key, not a new biometric, as the extension that the future environment needs.

Added

Everything from Section 5 onward describes machinery that did not exist when Version 1 was written: real signing and custody, the two witnesses, the detached verifier and the conformance contract, status and offline authorization, the membership proof and its second witness, the wallet protocol and the broker, federation, transparency, the exchange fabric, the operational drills, Atlas and Athena, the history of self-correction, the societal boundaries, and the future-facing sections on agents, personhood, delegation and outside observers.

Appendix F From Version 2 to Version 3

Version 2 is preserved unchanged beside this file. This appendix records what changed between them, and, more usefully, which sentences of Version 2 this version can no longer write.

What the system gained

Between v9.355 and 1.0.0-rc.1, in the order the rings meet them: what an enrolment rests on, the trusted referee and the enrolment code; the card as an encoding, an emulator, a personalization step and a verifier device, with duress measured in time; decisions about authorities, operators, quotas, ceilings and relying parties kept as append-only records; eight mutation drills over the verification layer and the corrections they forced; four formal models checked on every push; a population migration that runs and is measured; the cache rules that let an untrusted network carry a status; a depth-24 epoch pipeline; a measured statement of what two colluding verifiers can do; a standby region; a capacity model that found an integer column, and a cost model; the public transparency report; the pilot wind-down and the coexistence plan; the machine-checked assurance mapping and the automated third of accessibility; two format bridges; four packages on public registries; the OpenID4VP verifier; the operating contract and the scoreboard; and the first three external nouns on it.

Claims of Version 2 that this version withdraws or narrows

Table 22: Sentences of Version 2 that Version 3 does not repeat.		
Version 2 said	Version 3 says	Why
A post-quantum, issuer-unlinkable, compulsion-resistant identity-token system	An issuer-unlinkable, duress-aware identity-token system signed with ML-DSA-65 under an audited algorithm-migration path	Two words weakened by the repository’s own assessments; each is now refused by a check on outward surfaces
continued on the next page		

Version 2 said	Version 3 says	Why
The default signing algorithm is post-quantum from day one	The algorithm is ML-DSA-65 and the default development and CI path signs a labelled placeholder; production fails closed without real signing	Measured: 32 placeholder bytes where a signature is 3,309
The duress path is a tested property; no external party has measured its side channel An actual implementation by an external team has not happened	The mechanism resists the casual coercer, not the informed one beside the holder, and is net-negative against lawful access One unmodified outside wallet has presented over OpenID4VP and been accepted; no implementation of the native protocol by anyone else	The lab's assessment against the contract's three coercers The scoreboard's wallet row; the boundary between the two paths
The specification and the conformance suite exist; the integration is unproven Cross-verifier correlation is a permanent, documented property, then bounded	The OpenID Foundation's hosted suite ran the profile: 7 passed, 4 in review, not certified Bounded on the zero-knowledge path within the epoch's crowd, against an adversary that matches equality; exposed on plain and SD-JWT presentations; the crowd has no floor	The Foundation's own signed exports; REVIEW is not PASSED The linkability lab's three findings
The certificate the verifier signs requests with was correct The check layer catches its own violation	It carried no key usage, and every internal instrument had passed it Seventy-one of seventy-five checks passed on a tree where their property was commented out, and the tests for constraints, triggers, indexes and procedures would often not have noticed their subject missing	An outside wallet refused it The mutation drills; every finding closed and drilled
Thirty-six tables, 26 triggers, 191 checks, 48 conformance cases, 18 CI jobs, 20 formats, 1,048 test functions, 755 and 84 of 89 tests Thirteen audit-of-record instances	Forty-five tables, 42 triggers, 272 checks, 118 cases, 20 jobs, 27 formats, 1,755 test functions, 992 and 107 of 112 tests Fourteen named, thirty-one tables under an append-only trigger	Re-measured at 1.0.0-rc.1; Appendix C The schema's own triggers, counted
The physical token is modelled and not manufactured Multi-region recovery is not built	Encoded, emulated, personalized and read by a reference device; not manufactured A standby region is built and its non-zero recovery point is measured	<code>polaris_card/</code> and its drills The evacuation drill
The roadmap's targets are to be validated	Validated: every throughput target clears by an order of magnitude, and a 32-bit column would have ended the service in five days	The capacity model
The version is v9.355, a count of ships Future work is a list of things to build	The version is 1.0.0-rc.7, moved only by an externally observable change Future work is a list of things outside parties would have to do, questions the lab owes answers to, and items held until somebody outside needs them	The operating contract's versioning rule The operating contract's merge rule and kill line

What was deliberately not done

The dyno and the application baseline were not re-run, and their rows in Table 12 carried their original stamps; the dyno was re-run on 23 September 2026 (Appendix C). The Version 1 and Version 2 documents were not edited. The repository documents that disagree with the code at 1.0.0-rc.7 were not corrected for the paper; they are listed in the author's notes, as Version 2's were, so that the paper follows the code and the disagreement is not lost.

Vocabulary introduced

A fifth status mark, `EXTERNALLY EXERCISED`, for a capability a named outside party has exercised, with the rule that it is given only where the scoreboard has a filled row and never means audited, certified or piloted. A thirteenth rung on the ladder, for the knowledge the tree cannot check about itself. And the operating contract's own words, external noun, scoreboard, release candidate, used the way the repository uses them.